



Regnum Aurum Security Review

Pashov Audit Group

Conducted by: Shaka, btk, 0x37, 0xbepresent, Tigerfrake, Bloqarl, oualidpro, Atharv, Matin, seeques, zuhaibmohd

May 6th 2025 - June 3rd 2025

Contents

1. About Pashov Audit Group	8
2. Disclaimer	8
3. Introduction	8
4. About Regnum Aurum	8
5. Risk Classification	9
5.1. Impact	9
5.2. Likelihood	9
5.3. Action required for severity levels	9
6. Security Assessment Summary	10
7. Executive Summary	11
8. Findings	22
8.1. Critical Findings	22
[C-01] Users can steal lending interest	22
[C-02] getAssetValue() does not check if user is the owner of the asset	23
[C-03] Incorrect boost application leads to inflated rewards in gauges	24
[C-04] Failure to update user integral enables double reward accrual	25
[C-05] Double-counting stored rewards causes reward inflation	27
[C-06] Stale total voting power snapshot leads to incorrect reward distribution	31
[C-07] Users can accumulate rewards after lock expiry due to stale user block	32
[C-08] Incorrect lock duration causes extended voting power retention	33
8.2. High Findings	35
[H-01] Malicious users can extend the reward period	35
[H-02] Rewards from vault in LendingPool improperly collected	36
[H-03] DebtToken total supply is inflated	36
[H-04] LendingPool.withdrawAsset() does not take into account debt interests	37
[H-05] Overwrite in RToken.burn() can cause undesired behavior	38
[H-06] Borrowed amounts can be insured for free	39
[H-07] Timelock withdrawal mechanism in LendingPool can be bypassed	40
[H-08] First deposit front-running attack	40

[H-09] Removing distribution pool in RAACMinter does not update totalWeight	41
[H-10] Treasury deposit functions can be DoSed due to unsafe accounting	42
[H-11] Randomness can be manipulated in RWAVault redemptions	42
[H-12] Underflow forces full interest payment in debt token	44
[H-13] Permanent loss of rewards when distribution fails	46
[H-14] Incorrect reward calculation in _calculatePendingRewards	47
[H-15] Malicious users can distribute more rewards to the same gauge	48
[H-16] Reward rate can be manipulated in first cycle	49
[H-17] Incorrect totalSupply calculation	50
[H-18] Inflation attack in RWA Vault	51
[H-19] Users may fail to withdraw from lending pool	52
[H-20] Users can avoid insurance fee when reborrowing debt	53
[H-21] Users may be forced to pay more interest	54
[H-22] Users can steal rewards from SP	55
[H-23] FeeCollector will be dos	56
[H-24] Unclosed liquidation blocks collateral recovery	57
[H-25] Lock overwrite in createLock causes undercredited withdrawals	60
[H-26] Stale interest accrual allows over-borrowing	62
[H-27] Unrestricted FeeCollector.collectFee enables unauthorized fee extraction	65
[H-28] Deposits during liquidation enable collateral lockup	67
[H-29] GaugeRewardsDistributor distributes rewards to gauges not registered	68
[H-30] getNextCycleStart() returns incorrect value at the end of a cycle	69
[H-31] Rewards boosts distribute more rewards than available	70
8.3. Medium Findings	72
[M-01] Same emission cap for all reward tokens in BaseGauge	72
[M-02] Frequent RToken minting and burning raises accrued interest	72
[M-03] Amounts in RToken.calculateDustAmount() should not be normalized	73
[M-04] healthFactorLiquidationThreshold lacks borrowing safety margin	74
[M-05] Blacklisted users can repay their debt but not close their liquidation	74

[M-06] VaultProxy.depositIntoVault() reverts when small amount of assets are deposited	75
[M-07] VaultProxy.depositIntoVault() does not check for maximum deposit limit	75
[M-08] RToken supply cap can be exceeded	76
[M-09] RWAIndexToken is not compatible with Uniswap's router	76
[M-10] veRAACToken cannot receive fees directly from FeeCollector	77
[M-11] FeeCollector.distributeFees() gives unnecessary approval to target EOA	78
[M-12] RAACToken does not use FeeCollector.collectFee() to collect fees	78
[M-13] Stable token and USD assumed 1:1 in protocol	79
[M-14] Users cannot prevent liquidation during the pause period	79
[M-15] VRF requests are not handled properly	80
[M-16] Inefficient NFT Redemption in RWAVault	81
[M-17] RWAVault.redeemNFT() does not round up shares to be burned	81
[M-18] Vault cannot be changed in LendingPool	82
[M-19] Vault cannot be removed in LendingPool	82
[M-20] Incorrect check for health factor in LendingPool.withdrawAsset()	83
[M-21] Balance in vault is overestimated in lending pool rebalance	84
[M-22] Proposal cancellation delays critical change execution	85
[M-23] Inconsistent state at proposal.endTime in Governance	86
[M-24] Governance does not send ether to TimelockController	87
[M-25] Leftovers from distribution are locked in RAACMinter	87
[M-26] Distribution pool not added correctly in RAACMinter	88
[M-27] RAACNFT tokens cannot be burned	89
[M-28] Chainlink beta functions used for critical data	89
[M-29] veRAACToken maximum supply not enforced in increase()	90
[M-30] Auction.emergencyRefundERC20() cannot be executed	90
[M-31] Supply cap limits StabilityPool asset redeposits	91
[M-32] Balance check too strict prevents valid liquidations in StabilityPool	92
[M-33] Incorrect interface used for sCRVUSD Vault	93
[M-34] Missing liquidity rebalancing in repay function	93
[M-35] Reward distribution can be grieved to zero through dust transfers	94

[M-36] Unbounded array allow an attacker to DoS withdrawals	96
[M-37] Missing interface implementation in StabilityPool	97
[M-38] Broken access control between minter and token contracts	98
[M-39] distributionCycleBalance should not rely on contract balance	99
[M-40] Possible reward rate precision loss	100
[M-41] Improper allocation re-calculation	100
[M-42] Improper next Thursday calculation	101
[M-43] Improper user reward calculation in lending pool	102
[M-44] Proposal's result may change after the voting period	103
[M-45] Pending rewards may fail to be distributed	104
[M-46] May fail to liquidate borrow positions	105
[M-47] Improper emission cap check	106
[M-48] Users may change the fee collector's distribution ratio	107
[M-49] Missing distributor rate consideration impacts reward accrual	108
[M-50] Missing reward checkpoint in syncRewardData()	110
[M-51] Design of minting discounts in RWAVault is flawed	112
[M-52] Missing reward synchronization	113
[M-53] BaseGauge.syncRewardData() can provide incorrect reward rates	114
[M-54] Minimum reward rate of 1 wei/second overestimates available rewards	115
[M-55] Zero cost rounding in auction lets users acquire tokens for free	115
[M-56] Boost calculation in BaseGauge varies with staking token	116
[M-57] BaseGauge.pendingRewards() returns incorrect pending rewards	117
[M-58] Any one can prolong the reward distribution	118
8.4. Low Findings	119
[L-01] StabilityPool ignores crvUSD for liquidation debt coverage	119
[L-02] RAACMinter distribution pool updates ignore pending distributions	119
[L-03] vaultRewards.totalDeposits ignores share price growth	120
[L-04] StabilityPool.getTotalDeposits() includes donations	120
[L-05] No check for active positions when unregistering an adapter in LendingPool	120
[L-06] RToken and DebtToken returns incorrect value for first mint	120
[L-07] RWAVault does not check if 0 shares are minted	121

[L-08] unregister*Adapter() should not check global adapter support	121
[L-09] DebtToken.mint() emits Transfer event twice	122
[L-10] RToken.rescueToken() cannot be called from lending pool	122
[L-11] DebtToken.getUsageIndex() returns always WadRayMath.RAY	122
[L-12] RToken.getLiquidityIndex() returns always WadRayMath.RAY	123
[L-13] RAACToken skips burn tax if fee collector is not set	123
[L-14] RAACNFT.mint() assumes payment token has 18 decimals	123
[L-15] Wrong and misleading NatSpec comments	124
[L-16] No check for paused fee collector before calling collectFee()	124
[L-17] Lack of input validation in configuration values	125
[L-18] Withdraw timelock not cleared when duration is set to 0	125
[L-19] Inconsistent behavior between claimRewards() and claimRewardToken()	125
[L-20] Cancelling proposals in Governance does not cancel them in TimelockController	126
[L-21] RAACReleaseOrchestrator.emergencyRevoke() does not withdraw tokens	126
[L-22] Inconsistent approve() behavior in IVaultAssetAdapter implementations	126
[L-23] StabilityPool does not use upgradeable version of ReentrancyGuard	127
[L-24] Tokens with more than 18 decimals cause underflow in normalization	127
[L-25] RAACNFT.tokenURI() does not revert for invalid _tokenId	128
[L-26] Broken emergency unlock in veRAACToken	128
[L-27] LendingPool contract exceeds the size limit for mainnet deployment	129
[L-28] Unsafe use of transfer() and transferFrom()	129
[L-29] Broken internal balance for fee-on-transfer tokens	130
[L-30] GaugeController uses future timestamps to initialize time variables	131
[L-31] lastRewardTime in GaugeController incorrect on failed distribution	131
[L-32] claimRewardToken() blocks rewards after withdrawal in BaseGauge	131
[L-33] Auction contract permits end-time purchases due to off-by-one error	132

[L-34] RAACToken burns incorrect amount when fees are disabled	132
[L-35] RAACNFT.transferRole() revokes incorrect address	133
[L-36] StabilityPool minter update may lead to lost rewards	133
[L-37] VaultProxy double counts availableLiquidity in withdrawal calculations	134
[L-38] FeeCollector makes unchecked external calls to arbitrary addresses	135
[L-39] Missing update the interest timely	135
[L-40] Improper parameter in function notifyRewardAmount	136
[L-41] Missing sanity check for min rate & max rate in updateEmissionRate	136
[L-42] Missing sanity check for EMERGENCY_DELAY	137
[L-43] Missing emergency unlock logic	138
[L-44] Legacy lock recalculation slashes voting power	139
[L-45] Natspec and getNormalizedDebt implementation discrepancy	139
[L-46] Missing zero-amount check in LendingPool.borrowWithInsurance()	140
[L-47] Queued proposals will not be executed if the timelock is changed	141
[L-48] Altered LendingPool parameters may cause instant liquidations	141
[L-49] RToken fails if burn equals scaled balance increase	141
[L-50] LendingPool depositors might receive less than expected if fee is updated	142
[L-51] Governance owner can bypass timelock	142
[L-52] Incorrect check for total supply limit for veRAACToken	143
[L-53] Long liquidation grace period risks protocol solvency	143
[L-54] Hardcoded borrow cap incompatible with low decimal tokens	144
[L-55] Decimal mismatch between underlyingToken and RToken	145
[L-56] Users must withdraw from the pool to claim rewards	146
[L-57] Improper minDy calculation in LiquidationSwap	146
[L-58] Unbounded distributionPools array growth	147
[L-59] Gauge functions inaccessible due to missing controller implementations	149
[L-60] Proposal's execution can be blocked	150
[L-61] Protocol will lose some profit from the curve vault	151
[L-62] Duplicate reward application in _updateReward	152
[L-63] Indefinite reward accrual via emergencyWithdraw()	153

[L-64] Missing decimals normalization between collateral and debt	156
[L-65] Inactive gauge blocks withdrawals	157
[L-66] WadRayMath rayExp() loses precision for large numbers	159
[L-67] Withdrawals in RWAVault may cause unexpected liquidations	160

1. About Pashov Audit Group

Pashov Audit Group consists of multiple teams of some of the best smart contract security researchers in the space. Having a combined reported security vulnerabilities count of over 1000, the group strives to create the absolute very best audit journey possible - although 100% security can never be guaranteed, we do guarantee the best efforts of our experienced researchers for your blockchain protocol. Check our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Introduction

A time-boxed security review of the **RegnumAurumAcquisitionCorp/core** repository was done by **Pashov Audit Group**, with a focus on the security aspects of the application's smart contracts implementation.

4. About Regnum Aurum

Regnum Aurum (RAAC) is a fractionalization platform that tokenizes real estate into NFTs (RAACNFT) and fractional index tokens (iRAAC), enabling on-chain lending, borrowing, and liquidity against property value. By combining Chainlink-powered appraisals, a hybrid RWA Vault, and veRAAC governance the protocol enables programmable debt positions against real estate assets with on-chain liquidation mechanisms.

5. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

5.1. Impact

- High - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- Medium - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- Low - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

5.2. Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

5.3. Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

6. Security Assessment Summary

review commit hash - d44a2a55b2bf1b5c0b4554e193097d5cc4eb2b96

fixes review commit hash - b2203594b7a11e595bb4be32ca8fd5f45539c547

Scope

The following smart contracts were in scope of the audit:

- FeeCollector
- NFTRoyaltyFeeCollector
- Treasury
- BaseGauge
- GaugeController
- GaugeRewardsDistributor
- RAACGauge
- RWAGauge
- Governance
- TimelockController
- LLamaTemple
- RAACMinter
- RAACReleaseOrchestrator
- BaseChainlinkFunctionsOracle
- BaseVRFv2Consumer
- RAACHousePriceOracle
- RAACPrimeRateOracle
- ERC20AssetAdapter
- ERC721AssetAdapter
- LendingPool
- LendingPoolStorage
- LiquidationProxy
- VaultProxy
- LiquidationStrategyProxy
- LiquidationSwap
- StabilityPool
- StabilityPoolStorage
- ComplianceRegistry
- RAACHousePrices
- WithCompliance
- DEToken
- DebtToken
- LPToken
- RAACNFT
- RAACToken
- RToken
- RWAIndexToken
- verAACToken
- ERC20VaultAdapter
- ERC721VaultAdapter
- RWAVault
- BoostCalculator
- LockManager
- PowerCheckpoint
- PercentageMath
- TimeWeightedAverage
- WadRayMath
- DataTypes
- ReserveLibrary
- StringUtils
- Auction
- AuctionFactory
- ZENO
- ZENOFactory

7. Executive Summary

Over the course of the security review, Shaka, btk, 0x37, 0xbepresent, Tigerfrake, Bloqarl, oualidpro, Atharv, Matin, seeques, zuhaibmohd engaged with Regnum Aurum to review Regnum Aurum. In this period of time a total of **164** issues were uncovered.

Protocol Summary

Protocol Name	Regnum Aurum
Repository	https://github.com/RegnumAurumAcquisitionCorp/core
Date	May 6th 2025 - June 3rd 2025
Protocol Type	RWA tokenization

Findings Count

Severity	Amount
Critical	8
High	31
Medium	58
Low	67
Total Findings	164

Summary of Findings

ID	Title	Severity	Status
[C-01]	Users can steal lending interest	Critical	Resolved
[C-02]	getAssetValue() does not check if user is the owner of the asset	Critical	Resolved
[C-03]	Incorrect boost application leads to inflated rewards in gauges	Critical	Resolved
[C-04]	Failure to update user integral enables double reward accrual	Critical	Resolved
[C-05]	Double-counting stored rewards causes reward inflation	Critical	Resolved
[C-06]	Stale total voting power snapshot leads to incorrect reward distribution	Critical	Acknowledged
[C-07]	Users can accumulate rewards after lock expiry due to stale user block	Critical	Acknowledged
[C-08]	Incorrect lock duration causes extended voting power retention	Critical	Resolved
[H-01]	Malicious users can extend the reward period	High	Resolved
[H-02]	Rewards from vault in LendingPool improperly collected	High	Resolved
[H-03]	DebtToken total supply is inflated	High	Resolved
[H-04]	LendingPool.withdrawAsset() does not take into account debt interests	High	Resolved
[H-05]	Overwrite in RToken.burn() can cause undesired behavior	High	Resolved
[H-06]	Borrowed amounts can be insured for free	High	Resolved
[H-07]	Timelock withdrawal mechanism in LendingPool can be bypassed	High	Resolved
[H-08]	First deposit front-running attack	High	Resolved

[H-09]	Removing distribution pool in RAACMinter does not update totalWeight	High	Resolved
[H-10]	Treasury deposit functions can be DoSed due to unsafe accounting	High	Resolved
[H-11]	Randomness can be manipulated in RWAVault redemptions	High	Acknowledged
[H-12]	Underflow forces full interest payment in debt token	High	Resolved
[H-13]	Permanent loss of rewards when distribution fails	High	Resolved
[H-14]	Incorrect reward calculation in _calculatePendingRewards	High	Resolved
[H-15]	Malicious users can distribute more rewards to the same gauge	High	Resolved
[H-16]	Reward rate can be manipulated in first cycle	High	Acknowledged
[H-17]	Incorrect totalSupply calculation	High	Acknowledged
[H-18]	Inflation attack in RWA Vault	High	Resolved
[H-19]	Users may fail to withdraw from lending pool	High	Resolved
[H-20]	Users can avoid insurance fee when reborrowing debt	High	Resolved
[H-21]	Users may be forced to pay more interest	High	Resolved
[H-22]	Users can steal rewards from SP	High	Resolved
[H-23]	FeeCollector will be dos	High	Resolved
[H-24]	Unclosed liquidation blocks collateral recovery	High	Resolved
[H-25]	Lock overwrite in createLock causes undercredited withdrawals	High	Acknowledged
[H-26]	Stale interest accrual allows over-borrowing	High	Resolved
[H-27]	Unrestricted FeeCollector.collectFee enables unauthorized fee extraction	High	Resolved
[H-28]	Deposits during liquidation enable collateral lockup	High	Resolved

[H-29]	GaugeRewardsDistributor distributes rewards to gauges not registered	High	Resolved
[H-30]	getNextCycleStart() returns incorrect value at the end of a cycle	High	Resolved
[H-31]	Rewards boosts distribute more rewards than available	High	Resolved
[M-01]	Same emission cap for all reward tokens in BaseGauge	Medium	Resolved
[M-02]	Frequent RToken minting and burning raises accrued interest	Medium	Acknowledged
[M-03]	Amounts in RToken.calculateDustAmount() should not be normalized	Medium	Resolved
[M-04]	healthFactorLiquidationThreshold lacks borrowing safety margin	Medium	Resolved
[M-05]	Blacklisted users can repay their debt but not close their liquidation	Medium	Acknowledged
[M-06]	VaultProxy.depositIntoVault() reverts when small amount of assets are deposited	Medium	Resolved
[M-07]	VaultProxy.depositIntoVault() does not check for maximum deposit limit	Medium	Resolved
[M-08]	RToken supply cap can be exceeded	Medium	Resolved
[M-09]	RWAIndexToken is not compatible with Uniswap's router	Medium	Acknowledged
[M-10]	veRAACToken cannot receive fees directly from FeeCollector	Medium	Resolved
[M-11]	FeeCollector.distributeFees() gives unnecessary approval to target EOA	Medium	Resolved
[M-12]	RAACToken does not use FeeCollector.collectFee() to collect fees	Medium	Resolved
[M-13]	Stable token and USD assumed 1:1 in protocol	Medium	Resolved
[M-14]	Users cannot prevent liquidation during the pause period	Medium	Acknowledged

[M-15]	VRF requests are not handled properly	Medium	Resolved
[M-16]	Inefficient NFT Redemption in RWAVault	Medium	Acknowledged
[M-17]	RWAVault.redeemNFT() does not round up shares to be burned	Medium	Resolved
[M-18]	Vault cannot be changed in LendingPool	Medium	Resolved
[M-19]	Vault cannot be removed in LendingPool	Medium	Resolved
[M-20]	Incorrect check for health factor in LendingPool.withdrawAsset()	Medium	Resolved
[M-21]	Balance in vault is overestimated in lending pool rebalance	Medium	Resolved
[M-22]	Proposal cancellation delays critical change execution	Medium	Resolved
[M-23]	Inconsistent state at proposal.endTime in Governance	Medium	Resolved
[M-24]	Governance does not send ether to TimelockController	Medium	Resolved
[M-25]	Leftovers from distribution are locked in RAACMinter	Medium	Resolved
[M-26]	Distribution pool not added correctly in RAACMinter	Medium	Resolved
[M-27]	RAACNFT tokens cannot be burned	Medium	Resolved
[M-28]	Chainlink beta functions used for critical data	Medium	Acknowledged
[M-29]	veRAACToken maximum supply not enforced in increase()	Medium	Resolved
[M-30]	Auction.emergencyRefundERC20() cannot be executed	Medium	Resolved
[M-31]	Supply cap limits StabilityPool asset redeposits	Medium	Resolved
[M-32]	Balance check too strict prevents valid liquidations in StabilityPool	Medium	Resolved
[M-33]	Incorrect interface used for sCRVUSD Vault	Medium	Resolved
[M-34]	Missing liquidity rebalancing in repay function	Medium	Resolved

[M-35]	Reward distribution can be grieved to zero through dust transfers	Medium	Acknowledged
[M-36]	Unbounded array allow an attacker to DoS withdrawals	Medium	Resolved
[M-37]	Missing interface implementation in StabilityPool	Medium	Resolved
[M-38]	Broken access control between minter and token contracts	Medium	Resolved
[M-39]	distributionCycleBalance should not rely on contract balance	Medium	Acknowledged
[M-40]	Possible reward rate precision loss	Medium	Resolved
[M-41]	Improper allocation re-calculation	Medium	Acknowledged
[M-42]	Improper next Thursday calculation	Medium	Resolved
[M-43]	Improper user reward calculation in lending pool	Medium	Resolved
[M-44]	Proposal's result may change after the voting period	Medium	Resolved
[M-45]	Pending rewards may fail to be distributed	Medium	Resolved
[M-46]	May fail to liquidate borrow positions	Medium	Resolved
[M-47]	Improper emission cap check	Medium	Acknowledged
[M-48]	Users may change the fee collector's distribution ratio	Medium	Resolved
[M-49]	Missing distributor rate consideration impacts reward accrual	Medium	Resolved
[M-50]	Missing reward checkpoint in syncRewardData()	Medium	Resolved
[M-51]	Design of minting discounts in RWAVault is flawed	Medium	Acknowledged
[M-52]	Missing reward synchronization	Medium	Resolved
[M-53]	BaseGauge.syncRewardData() can provide incorrect reward rates	Medium	Acknowledged

[M-54]	Minimum reward rate of 1 wei/second overestimates available rewards	Medium	Resolved
[M-55]	Zero cost rounding in auction lets users acquire tokens for free	Medium	Resolved
[M-56]	Boost calculation in BaseGauge varies with staking token	Medium	Resolved
[M-57]	BaseGauge.pendingRewards() returns incorrect pending rewards	Medium	Resolved
[M-58]	Any one can prolong the reward distribution	Medium	Resolved
[L-01]	StabilityPool ignores crvUSD for liquidation debt coverage	Low	Resolved
[L-02]	RAACMinter distribution pool updates ignore pending distributions	Low	Resolved
[L-03]	vaultRewards.totalDeposits ignores share price growth	Low	Resolved
[L-04]	StabilityPool.getTotalDeposits() includes donations	Low	Resolved
[L-05]	No check for active positions when unregistering an adapter in LendingPool	Low	Acknowledged
[L-06]	RToken and DebtToken returns incorrect value for first mint	Low	Resolved
[L-07]	RWAVault does not check if 0 shares are minted	Low	Resolved
[L-08]	unregister*Adapter() should not check global adapter support	Low	Resolved
[L-09]	DebtToken.mint() emits Transfer event twice	Low	Resolved
[L-10]	RToken.rescueToken() cannot be called from lending pool	Low	Resolved
[L-11]	DebtToken.getUsageIndex() returns always WadRayMath.RAY	Low	Resolved
[L-12]	RToken.getLiquidityIndex() returns always WadRayMath.RAY	Low	Resolved

[L-13]	RAACToken skips burn tax if fee collector is not set	Low	Acknowledged
[L-14]	RAACNFT.mint() assumes payment token has 18 decimals	Low	Acknowledged
[L-15]	Wrong and misleading NatSpec comments	Low	Resolved
[L-16]	No check for paused fee collector before calling collectFee()	Low	Acknowledged
[L-17]	Lack of input validation in configuration values	Low	Resolved
[L-18]	Withdraw timelock not cleared when duration is set to 0	Low	Resolved
[L-19]	Inconsistent behavior between claimRewards() and claimRewardToken()	Low	Resolved
[L-20]	Cancelling proposals in Governance does not cancel them in TimelockController	Low	Acknowledged
[L-21]	RAACReleaseOrchestrator.emergencyRevoke() does not withdraw tokens	Low	Resolved
[L-22]	Inconsistent approve() behavior in IVaultAssetAdapter implementations	Low	Resolved
[L-23]	StabilityPool does not use upgradeable version of ReentrancyGuard	Low	Resolved
[L-24]	Tokens with more than 18 decimals cause underflow in normalization	Low	Acknowledged
[L-25]	RAACNFT.tokenURI() does not revert for invalid _tokenId	Low	Resolved
[L-26]	Broken emergency unlock in veRAACToken	Low	Acknowledged
[L-27]	LendingPool contract exceeds the size limit for mainnet deployment	Low	Resolved
[L-28]	Unsafe use of transfer() and transferFrom()	Low	Resolved
[L-29]	Broken internal balance for fee-on-transfer tokens	Low	Resolved
[L-30]	GaugeController uses future timestamps to initialize time variables	Low	Resolved

[L-31]	lastRewardTime in GaugeController incorrect on failed distribution	Low	Resolved
[L-32]	claimRewardToken() blocks rewards after withdrawal in BaseGauge	Low	Resolved
[L-33]	Auction contract permits end-time purchases due to off-by-one error	Low	Acknowledged
[L-34]	RAACToken burns incorrect amount when fees are disabled	Low	Acknowledged
[L-35]	RAACNFT.transferRole() revokes incorrect address	Low	Resolved
[L-36]	StabilityPool minter update may lead to lost rewards	Low	Acknowledged
[L-37]	VaultProxy double counts availableLiquidity in withdrawal calculations	Low	Resolved
[L-38]	FeeCollector makes unchecked external calls to arbitrary addresses	Low	Resolved
[L-39]	Missing update the interest timely	Low	Resolved
[L-40]	Improper parameter in function notifyRewardAmount	Low	Resolved
[L-41]	Missing sanity check for min rate & max rate in updateEmissionRate	Low	Acknowledged
[L-42]	Missing sanity check for EMERGENCY_DELAY	Low	Resolved
[L-43]	Missing emergency unlock logic	Low	Acknowledged
[L-44]	Legacy lock recalculation slashes voting power	Low	Acknowledged
[L-45]	Natspec and getNormalizedDebt implementation discrepancy	Low	Resolved
[L-46]	Missing zero-amount check in LendingPool.borrowWithInsurance()	Low	Resolved
[L-47]	Queued proposals will not be executed if the timelock is changed	Low	Acknowledged

[L-48]	Altered LendingPool parameters may cause instant liquidations	Low	Acknowledged
[L-49]	RToken fails if burn equals scaled balance increase	Low	Resolved
[L-50]	LendingPool depositors might receive less than expected if fee is updated	Low	Acknowledged
[L-51]	Governance owner can bypass timelock	Low	Resolved
[L-52]	Incorrect check for total supply limit for veRAACToken	Low	Acknowledged
[L-53]	Long liquidation grace period risks protocol solvency	Low	Acknowledged
[L-54]	Hardcoded borrow cap incompatible with low decimal tokens	Low	Resolved
[L-55]	Decimal mismatch between underlyingToken and RToken	Low	Resolved
[L-56]	Users must withdraw from the pool to claim rewards	Low	Resolved
[L-57]	Improper minDy calculation in LiquidationSwap	Low	Resolved
[L-58]	Unbounded distributionPools array growth	Low	Resolved
[L-59]	Gauge functions inaccessible due to missing controller implementations	Low	Acknowledged
[L-60]	Proposal's execution can be blocked	Low	Resolved
[L-61]	Protocol will lose some profit from the curve vault	Low	Acknowledged
[L-62]	Duplicate reward application in _updateReward	Low	Resolved
[L-63]	Indefinite reward accrual via emergencyWithdraw()	Low	Resolved
[L-64]	Missing decimals normalization between collateral and debt	Low	Acknowledged
[L-65]	Inactive gauge blocks withdrawals	Low	Resolved

[<u>L-66</u>]	WadRayMath.rayExp() loses precision for large numbers	Low	Resolved
[<u>L-67</u>]	Withdrawals in RWAVault may cause unexpected liquidations	Low	Resolved

8. Findings

8.1. Critical Findings

[C-01] Users can steal lending interest

Severity

Impact: High

Likelihood: High

Description

In RAAC, users can lend crvUSD to earn some interest. Users' lending position will be recorded with RToken. The `rawBalance` for one account is the user's principle. Users' actual amount is `rawBalance` plus accrued interest.

The user's interest will be calculated according to `userIndex`.

The problem here is that RToken can be transferred. When we transfer RToken from one account to another account, we don't process the pending interest and update the liquidity index. Malicious users can transfer RToken from one position with higher `_userState[account].index` to another position with one lower `_userState[account].index`. Then we can earn interest than expected.

```

function balanceOf(address account) public view override
(ERC20, IERC20) returns (uint256) {
    uint256 rawBalance = super.balanceOf(account);
    uint256 liquidityIndex = ILendingPool(_lendingPool).getNormalizedIncome
    ();
    uint256 userIndex = liquidityIndex - _userState[account].index;

    if(userIndex == 0) return rawBalance;
    // principle + interest
    return rawBalance + rawBalance.mul(userIndex);
}

function transfer(address recipient, uint256 amount) public override
(ERC20, IERC20) returns (bool) {
    return super.transfer(recipient, amount);
}

function _update
(address from, address to, uint256 amount) internal override {
    if (from != address(0)) {
        // Update rewards for sender before transfer
        ILendingPool(_lendingPool).updateUserReward(from);
    }
    if (to != address
    //(0) && to != from) { // Only update recipient if different from sender
        // Update rewards for recipient before transfer
        ILendingPool(_lendingPool).updateUserReward(to);
    }
    // actual share amount is
    super._update(from, to, amount);
}

```

Recommendations

When we transfer RToken, we need to accrue the pending interest for two accounts and update users' index.

[C-02] `getAssetValue()` does not check if `user` is the owner of the asset

Severity

Impact: High

Likelihood: High

Description

The `getAssetValue()` function in `ERC721AssetAdapter` receives a `user` parameter, but this value is not used to validate that token is deposited in the contract by the user.

As this function is used in `LendingPool._validateBorrow()`, users can borrow against assets that they do not own.

Recommendations

Validate that the `user` is the depositor of the asset in the `getAssetValue()` and `getWithdrawValue()` functions.

[C-03] Incorrect boost application leads to inflated rewards in gauges

Severity

Impact: High

Likelihood: High

Description

The `BaseGauge` contract implements a reward distribution system where users can stake tokens and earn rewards based on their stake amount and boost multiplier. The boost mechanism is designed to incentivize long-term staking by providing increased rewards to users who lock their tokens. However, there is an issue in how the boost is applied when calculating pending rewards, leading to users receiving significantly fewer rewards than intended.

The issue lies in the `_calculatePendingRewards()` function of `BaseGauge`, which is responsible for determining how many rewards a user has earned since their last update. While the function correctly calculates the base reward share proportional to the user's stake, it fails to properly apply the boost multiplier to this share.

The core of the problem is in how the boost value returned from `_applyBoost()` is used. The boost value is expressed in basis points (e.g., 10000 = 1.0x boost, 20000 = 2.0x boost), but the function directly assigns this basis point value as the user's share without converting it to the actual boosted amount:

```
uint256 boost = _applyBoost(user, userShare);
userShare = boost > 0 ? boost : userShare;
```

This is incorrect because it replaces the user's actual share (which is a large number of tokens) with just the boost multiplier (which is in basis points). For example, if a user should receive 100e18 tokens with a 2.0x boost (20000 basis points), they would instead receive just 20000 tokens.

This contrasts with the correct implementation in `_updateUserReward()` which properly applies the boost:

```
uint256 workingBalance = (workingBalance * boost) / WEIGHT_PRECISION;
```

Proof of Concept

1. Alice stakes 1000e18 tokens in the gauge.
2. The gauge accumulates rewards over time.
3. When Alice checks her pending rewards:
 - Base reward calculation: 100e18 tokens earned.
 - Boost calculation returns 20000 (2.0x boost).
 - Instead of receiving 200e18 tokens (1000 * 2.0).
 - Alice only receives 20000 tokens (the boost basis points value).

Recommendations

Modify the boost application in `_calculatePendingRewards()` to match the calculation in `_updateUserReward()`:

```
- userShare = boost > 0 ? boost : userShare;  
+ userShare = boost > 0 ? (userShare * boost) / WEIGHT_PRECISION : userShare;
```

[C-04] Failure to update user integral enables double reward accrual

Severity

Impact: High

Likelihood: High

Description

In `BaseGauge.sol`, the helper function `updateUserRewards()` credits the full “pending” reward balance but fails to advance the user’s cumulative checkpoint (`integral`). Consequently, when `claimRewards()` later invokes `_updateUserReward()`, it sees the same historical gap and issues a second amount of rewards.

Key problematic lines:

```

File: BaseGauge.sol
359:     function updateUserRewards() external nonReentrant {
360:         // First update basic reward for all reward tokens
361:         for (uint256 i = 0; i < rewardTokens.length; i++) {
362:             address token = rewardTokens[i];
363:             RewardData storage rd = rewardData[token];
364:
365:             // Skip calling _updateReward and directly update the stored
// rewards
366:@>             uint256 pending = _calculatePendingRewards(token, msg.sender);
367:             if (pending > 0) {
368:@>                 rewards[token][msg.sender] += pending;
369:
370:                 // Update the last update time to current time
371:
372:                 userRewardData[token][msg.sender].lastUpdate = block.timestamp;
373:
374:                 userRewardPerTokenPaid[token][msg.sender] = rd.rewardPerTokenStored;
375:             }
376:             emit UserRewardsUpdated(msg.sender, block.timestamp);
377:         }
...
929:     function _updateUserReward(
930:         address token,
931:         address account,
932:         uint256 rewardPerToken,
933:         uint256 decimalAdjustment
934:     ) internal {
935:         UserRewardData storage userData = userRewardData[token][account];
936:         uint256 userBalance = _balances[account];
937:
938:         if (userBalance > 0) {
939:@>             uint256 userIntegralDelta = rewardPerToken - userData.integral;
940:@>             if (userIntegralDelta > 0) {
941:                 uint256 workingBalance = userBalance;
942:                 if (boostAmplificationEnabled) {
943:                     uint256 boost = _applyBoost(account, workingBalance);
944:                     workingBalance =
945:                         (workingBalance * boost) / WEIGHT_PRECISION;
946:                 }
947:                 uint256 newReward = (workingBalance * userIntegralDelta) /
948:                     (1e18 * decimalAdjustment);
949:
950:                 userData.integral = rewardPerToken;
951:                 userData.lastUpdate = block.timestamp;
952:@>                 rewards[token][account] += newReward;
953:
954:                 emit UserRewardDataUpdated
955:                     (account, token, rewardPerToken, block.timestamp);
956:             }
957:         }

```

- **Root cause:** `updateUserRewards()` writes to `rewards[token][user]` and updates only `lastUpdate` & `rewardPerTokenPaid`, **never** touching `userData.integral` code line 367-372.
- **Consequence:** `claimRewards()` -> `_updateUserReward()` believes no checkpoint was taken (code line 940) and mints an entirely new reward amount on top of what was just claimed.

Consider the following scenario:

1. **Setup:** Attacker stakes tokens and waits for reward accrual.
2. **First call** (`updateUserRewards()`):

- Contract computes `pending = _calculatePendingRewards(...)` based on delta timestamp.
 - Line 368 credits `rewards[token][attacker] += pending`.
 - Only `lastUpdate` & `rewardPerTokenPaid` code line 371-372 are updated; `integral` remains stale.
3. **Second call** (`claimRewards()`):
- Internally calls `_updateReward(...)` → `_updateUserReward(...)`.
 - Computes `userIntegralDelta = rewardPerTokenStored - userData.integral` (full span again).
 - Line 947 issues a second `newReward` equal (or nearly equal) to the first tranche.
 - Line 952 credits that amount to `rewards[token][attacker]`.
4. **Result:** Attacker's `rewards[token][attacker]` has been increased twice for the same accrual period.
5. **Repetition:** Steps 2–3 can be repeated indefinitely, draining the reward pool.

Recommendations

Synchronize integral. After line 368 in `updateUserRewards()`, add:

```
userRewardData[token][msg.sender].integral = calculatedRewardPerTokenStored;
```

This zeroes out the checkpoint so that subsequent calls see no residual delta.

[C-05] Double-counting stored rewards causes reward inflation

Severity

Impact: High

Likelihood: High

Description

In `BaseGauge.sol`, the helper `_calculatePendingRewards()` erroneously returns the entirety of a user's **already stored** rewards plus the newly calculated share:


```

File: BaseGauge.sol
754:     function _calculatePendingRewards(
755:         address token,
756:         address user
757:     ) internal view returns (uint256) {
...
764:@>         uint256 storedRewards = rewards[token][user];
...
825:@>         return storedRewards + userShare;
826:     }

```

Then, in every place the function is called, it **adds** this returned value on top of the existing balance rather than **overwriting** it:

```

File: BaseGauge.sol
366:         uint256 pending = _calculatePendingRewards(token, msg.sender);
367:         if (pending > 0) {
368:             rewards[token][msg.sender] += pending;
...
...
985:             uint256 pending = _calculatePendingRewards(
986:                 rewardTokenAddr,
987:                 account
988:             );
989:             if (pending > 0) {
990:                 rewards[rewardTokenAddr][account] += pending;
991:             }
...
...
1012:         uint256 pending = _calculatePendingRewards
1013:             (token, account);
1014:         if (pending > 0) {
1015:             rewards[token][account] += pending;
...
...
1389:         if (hasDistributor) {
1390:             uint256 calculatedRewards = _calculatePendingRewards
1391:                 (token, user);
1392:             return rewards[token][user] + calculatedRewards;

```

Because `pending` already includes `storedRewards`, each call **double-counts** the user's prior balance. Subsequent calls amplify the effect, enabling exponential inflation of a user's rewards. Consider the following scenario:

1. Initial state: `rewards[token][user] = 0`.

2. First call to `updateUserRewards()`:

- `_calculatePendingRewards` returns `0 + 100 = 100`.
- `rewards += 100` → new `rewards = 100`.

3. Second call (no new accrual for test purposes):

- `storedRewards = 100`; `_calculatePendingRewards` returns `100 + 0 = 100`.
- `rewards += 100` → new `rewards = 200`.

4. Third call:

- `storedRewards = 200`; returns `200 + 0 = 200`.
- `rewards += 200` → new `rewards = 400`.

Each extra invocation doubles previously stored rewards, draining the reward pool rapidly.

In the following test, the attacker first stakes a bulk amount of tokens and fast-forwards time by seven days to accrue rewards; they then stake an extra 1 wei to ensure `rewards[token][account]` is non-zero, after which they invoke `updateUserRewards()` three times in same timestamp call inflating their stored reward balance by the same pending amount without advancing the timestamp, and finally call `claimRewards()` (followed by `claimRewardToken`) to drain the gauge's reward reserve.

```
// File: BaseGauge.test.js
describe("0xbepresentRewardDistribution", () => {
  beforeEach(async () => {
    await advancePastPeriod(gauge);
    await gaugeController.vote(await gauge.getAddress(), 5000);
    await distributor.initializeRewardData(await gauge.getAddress(),
    await rewardToken.getAddress(), REWARD_AMOUNT);
    await rewardToken.approve(await distributor.getAddress(), REWARD_AMOUNT);
    await distributor.notifyRewardAmount(await rewardToken.getAddress(), REWARD_AMOUNT);
  });
  it("0xbepresentAttackerCanDoublecountRewards", async () => {
    //
    // 1. Attacker stakes some tokens
    let stake_amount = ethers.parseEther("10");
    console.log("Staked amount:", ethers.formatEther(stake_amount));
    await gauge.connect(user1).stake(stake_amount);
    // Advance time to accumulate some rewards
    await time.increase(DAY * 7);
    //
    // 2. Attacker stakes so `rewards[token][account]` is increased
    await gauge.connect(user1).stake(1);
    let rewardsWhenUserRewardsCall = await gauge.rewards
    (await rewardToken.getAddress(), user1.address);
    console.log("Stake 1 wei. Updated rewards:", ethers.formatEther
    ((rewardsWhenUserRewardsCall)); // Check rewards
    //
    // 3. Attacker call `updateUserRewards`
    //() ` multiple times so pending rewards are increased multiple times
    await gauge.connect(user1).updateUserRewards();
    rewardsWhenUserRewardsCall = await gauge.rewards
    (await rewardToken.getAddress(), user1.address);
    console.log("Rewards after call updateUserRewards
    //():", ethers.formatEther(rewardsWhenUserRewardsCall)); // Check rewards
    await gauge.connect(user1).updateUserRewards();
    rewardsWhenUserRewardsCall = await gauge.rewards
    (await rewardToken.getAddress(), user1.address);
    console.log("Rewards after call updateUserRewards
    //():", ethers.formatEther(rewardsWhenUserRewardsCall)); // Check rewards
    await gauge.connect(user1).updateUserRewards();
    rewardsWhenUserRewardsCall = await gauge.rewards
    (await rewardToken.getAddress(), user1.address);
    console.log("Rewards after call updateUserRewards
    //():", ethers.formatEther(rewardsWhenUserRewardsCall)); // Check rewards
    //
    // 4. Attacker drains the contract all the deposited `rewards`
    // amount`
    await gauge.connect(user1).claimRewards();
    // Check claimed rewards
    const claimedRewards = await rewardToken.balanceOf(user1.address);
    console.log(
      "Totalclaimedrewards:",
      ethers.formatEther
    )
    // Assert that the claimed rewards to be equal to the total rewards
    expect(claimedRewards).to.be.gte(REWARD_AMOUNT);
    // Check that the contract balance is 0
    const contractBalance = await rewardToken.balanceOf
    (await gauge.getAddress());
    expect(contractBalance).to.equal(0);
  });
});
```

Output

```
Staked amount: 10.0
Stake 1 wei. Updated rewards: 233.3333333333330496
Rewards after call updateUserRewards(): 466.6666666666660992
Rewards after call updateUserRewards(): 933.3333333333321984
Rewards after call updateUserRewards(): 1866.6666666666643968
Total claimed rewards: 1000.0
```

Recommendations

Return only the delta in `_calculatePendingRewards()` or overwrite instead of "add" when calling the function.

[C-06] Stale total voting power snapshot leads to incorrect reward distribution

Severity

Impact: High

Likelihood: High

Description

The `veRAACToken` contract implements a reward distribution mechanism where users earn rewards based on their voting power. The `rewardPerToken()` function calculates rewards using `totalVotingPowerAtNotify`, which is a snapshot of the total voting power taken when rewards are notified through `notifyRewardAmount()`:

```
function rewardPerToken(address rewardToken) public view returns (uint256) {
    RewardData memory rData = rewardData[rewardToken];

    uint256 totalVeSupply = rData.totalVotingPowerAtNotify;
    if (totalVeSupply == 0) {
        return rData.rewardPerTokenStored;
    }

    uint256 lastTimeRewardApplicable = _lastTimeRewardApplicable(
        rData.periodFinish
    );
    if (lastTimeRewardApplicable <= rData.lastUpdateTime) {
        return rData.rewardPerTokenStored;
    }

    // Calculate time-weighted rewards
    uint256 timeDelta = lastTimeRewardApplicable - rData.lastUpdateTime;
    uint256 rewardDelta = (timeDelta * rData.rewardRate) / PRECISION;

    // Use total supply from notification time
    uint256 rewardPerTokenDelta = (rewardDelta * PRECISION) / totalVeSupply;
    return rData.rewardPerTokenStored + rewardPerTokenDelta;
}
```

However, the snapshot is not updated when new users lock tokens after rewards are notified. This creates an accounting error where rewards are calculated using an outdated total voting power denominator, leading to over-distribution of rewards.

Consider this scenario:

1. Bob locks 100 tokens for 1 year.

2. Admin calls `notifyRewardAmount()` with 100 USDT rewards, setting `totalVotingPowerAtNotify = 100`.
3. Alice locks 100 tokens for 1 year, bringing actual total voting power to 200.
4. After 1 week:
 - System calculates rewards using stale `totalVotingPowerAtNotify = 100`.
 - Bob and Alice each claim ~99 USDT rewards.
 - Total claimed rewards (198 USDT) exceed notified amount (100 USDT).
5. The second user's withdrawal fails due to insufficient reward token balance.

```
function testBreakRewards() public {
    raacToken.mint(bob, 100e18);
    raacToken.mint(alice, 100e18);

    veToken.addRewardToken(address(usdt));

    vm.startPrank(bob);
    raacToken.approve(address(veToken), 100e18);
    veToken.lock(100e18, 365 days);
    vm.stopPrank();

    usdt.mint(address(this), 100e6);
    usdt.approve(address(veToken), 100e6);
    veToken.notifyRewardAmount(address(usdt), 100e6);

    vm.startPrank(alice);
    raacToken.approve(address(veToken), 100e18);
    veToken.lock(100e18, 365 days);
    vm.stopPrank();

    vm.warp(block.timestamp + 7 days);

    vm.prank(bob);
    veToken.updateReward();

    vm.prank(alice);
    veToken.updateReward();

    console.log(veToken.earned(bob, address(usdt))); // 99e6
    console.log(veToken.earned(alice, address(usdt))); // 99e6
}
```

Test Setup: <https://gist.github.com/0xbtk/0c1f5e75815d2cf02d3437cbded026ff>.

Recommendations

Update the `rewardPerToken()` function to use the current total voting power instead of the snapshot(not sure about this fix, recheck).

[C-07] Users can accumulate rewards after lock expiry due to stale user block

Severity

Impact: High

Likelihood: High

Description

The `veRAACToken` contract incorrectly allows users to continue accumulating rewards even after their token lock period has expired. This occurs because the reward calculation relies on `uData.lastUserBlock` which remains unchanged unless explicitly updated through user actions like locking or unlocking tokens:

```
function getNewUserRewards(
    address user,
    address token
) internal view returns (uint256) {
    UserRewardData memory uData = userRewardData[user][token];
    uint256 userPower = balanceOfAt(user, uData.lastUserBlock);
    uint256 rewardPerTokenStored = rewardPerToken(token);
    return
        (userPower *
         (rewardPerTokenStored - uData.userRewardPerTokenPaid)) /
        PRECISION;
}
```

For example:

1. User locks RAAC tokens, setting `uData.lastUserBlock` to the current block.
2. Lock period expires, reducing user's voting power to 0.
3. User continues to earn rewards based on their historical balance at `lastUserBlock`.
4. Rewards accumulate indefinitely until the user performs an action that triggers `updateReward()`.

This creates a vulnerability where users can earn rewards without maintaining the required locked position, effectively stealing rewards from other users.

Recommendations

Use an epoch-based reward calculation, where a new epoch begins each time `notifyRewardAmount()` is called. Calculate each user's updated rewards based on the `lastDistributionBlock` for that specified epoch.

[C-08] Incorrect lock duration causes extended voting power retention

Severity

Impact: High

Likelihood: High

Description

The `getVotingPowerAtBlock()` function in the `PowerCheckpoint` library incorrectly uses `maxLockDuration` as the `totalTime` parameter instead of the actual lock duration `(lock.end - lock.start)` when calculating the decayed balance using `LockManager.calculateExpectedBalanceAfterDecay()`.

```
return LockManager.calculateExpectedBalanceAfterDecay(  
    lock.amount,  
    state.lockState.maxLockDuration, // @audit incorrect  
    timeElapsed,  
    state.lockState.maxLockDuration  
);
```

This causes the following issues:

1. User locks tokens for a minimum duration (7 days).
2. After the duration ends, `getVotingPowerAtBlock()` calculates decay using maximum lock duration (1 year).
3. The user will have 10x voting power.

Note: `PowerCheckpoint.getPastTotalSupply()` contain the same issue.

Recommendations

Use `lock.end - lock.start` as the `totalTime`:

```
return LockManager.calculateExpectedBalanceAfterDecay(  
    lock.amount,  
    lock.end - lock.start, // Use actual lock duration  
    timeElapsed,  
    state.lockState.maxLockDuration  
);
```

8.2. High Findings

[H-01] Malicious users can extend the reward period

Severity

Impact: Medium

Likelihood: High

Description

In GaugeRewardDistributor contract, anyone is allowed to deposit some rewards. The problem here is that malicious users can deposit one tiny reward to extend the period. This will cause the reward rate to be diluted. Normal users may need more time to get the expected rewards.

```
function _handleRewardNotification(address token, uint256 amount) internal {
    if (timestamp >= rd.periodFinish) {
        rd.rate = amount / DURATION;
        rd.lastUpdateTime = getNextCycleStart();
        rd.periodFinish = rd.lastUpdateTime + DURATION;
    } else {
        uint256 remaining = rd.periodFinish - timestamp;
        uint256 leftover = remaining * rd.rate;
    }
    rd.rate = (amount + leftover) / DURATION;
    rd.periodFinish = getNextCycleStart() + DURATION;
}
```

Also, consider this scenario:

- Current cycle started at day 1 and ends at day 8.
- `_handleRewardNotification` is executed on day 2. `getNextCycleStart()` will return on day 8, so `periodFinish` will be set to day 15.
- `rd.rate` is updated to distribute the current rewards over 7 days, that is, they will be consumed from day 2 to day 9.

Recommendations

Revisit the distribution logic.

E.g.


```
} else {  
    // Middle of existing cycle  
    uint256 remaining = rd.periodFinish - timestamp;  
    uint256 leftover = remaining * rd.rate;  
    - rd.rate = (amount + leftover) / DURATION;  
    + rd.rate = (amount + leftover) / remaining;  
    - rd.periodFinish = getNextCycleStart() + DURATION;
```

[H-02] Rewards from vault in **LendingPool** improperly collected

Severity

Impact: Medium

Likelihood: High

Description

When there is a withdrawal from the vault in the **LendingPool**, the proportion of the withdrawn amount that corresponds to the appreciation in the share price is considered to yield and kept as a reward for the protocol to be collected.

The problem is that **vaultRewards.lastSharePrice** is updated even if it has decreased, which means that the protocol might be accruing rewards when it should not, provoking a loss of funds for the depositors.

Consider the following scenario:

- 1,000 tokens are deposited in the vault, being a share price of 1.
- Share price decreases to 0.9 due to unrealized losses.
- 100 tokens are withdrawn and **lastSharePrice** is updated to 0.9.
- Share price increases to 0.95.
- 500 tokens are withdrawn and it is considered that a 0.05 appreciation in the share price has occurred, so 25 tokens are kept as rewards for the protocol to be collected.
- The protocol has collected 25 tokens as rewards, while the position in the vault is still at a loss.

Recommendations

Update **lastSharePrice** only when the share price increases.

[H-03] **DebtToken** total supply is inflated

Severity

Impact: Medium

Likelihood: High

Description

The total supply of `DebtToken` is calculated by multiplying the total amount of tokens minted by the value of the usage index in the lending pool.

```
function totalSupply() public view override(ERC20, IERC20) returns
(uint256) {
    return super.totalSupply().rayMul(ILendingPool
        (_lendingPool).getNormalizedDebt());
}
```

However, it is not taken into account that `super.totalSupply()` may already contain scaled amounts, as the `mint()` and `burn()` functions normalize the current balance of the user by the usage index.

```
function mint(
    (...)
    if (positionIndex != 0 && positionIndex < poolUsageIndex) {
@>        scaledBalanceIncrease = rawDebtBalance.rayMul
        (poolUsageIndex) - rawDebtBalance.rayMul(positionIndex);
    }

@>        uint256 amountToMint = amountScaled + scaledBalanceIncrease;

    _mint(onBehalfOf, amountToMint.toUint128());
```

As a result, the total supply of `DebtToken` is inflated by the scaled balance increase calculated in the `mint()` and `burn()` functions. These amounts are also normalized by the usage index, which compounds the inflation effect.

Any calculations that rely on the total supply of `DebtToken` will be affected by this inflation, including:

- The validation of the borrow supply cap in the lending pool, which could lead to denial of borrows before the cap is reached.
- The `reserve.totalUsage` value in the lending pool.

Recommendations

It will be required to track separately the amount of `DebtToken` minted by borrowed amounts and normalize that amount by the usage index when calculating the total supply.

[H-04] `LendingPool.withdrawAsset()` does not take into account debt interests

Severity

Impact: High

Likelihood: Medium

Description

In `LendingPool._validateBorrow()` the `getPositionDebt()` function is used to calculate the current debt of the user's position. This function does not take into account the debt accumulated by the increase of the usage index, which makes the debt to be underestimated.

As a result, the user can borrow more assets than they should be able to and leave the position undercollateralized.

Recommendations

```
-      uint256 positionDebt = getPositionDebt(adapter, msg.sender, data);
+      uint256 positionDebt = getPositionScaledDebt
+ (adapter, msg.sender, data);
      if (positionDebt > 0) {
(...)
-      if (maxDebt < getPositionDebt(adapter, msg.sender, data) + amount) {
+      if (maxDebt < getPositionScaledDebt
+ (adapter, msg.sender, data) + amount) {
          revert NotEnoughCollateralToBorrow();
      }
```

[H-05] Overwrite in `RToken.burn()` can cause undesired behavior

Severity

Impact: Medium

Likelihood: High

Description

In `RToken.burn()`, when the amount to be burned for a user is greater than his balance, the amount is overwritten to the user's balance.

```
if(amount > userBalance){
    amount = userBalance;
}
```

This will have the following consequences when `LendingPool.withdraw()` is called:

- The user might receive fewer tokens than expected, if the available balance was overestimated. Given that no value is returned by the `withdraw()` function, the user will assume that the amount withdrawn is the same as the amount requested. This can be especially problematic in the case of contract interactions.
- Anyone with a minimum deposit (e.g. 1 wei) can spam the protocol to withdraw all the deposits from the vault in order to cover his withdrawal.

Consider the following scenario:

1. Alice deposits 1 wei into the `LendingPool`.
2. The total deposits are 10,000 crvUSD, 2,000 of which are in the `RToken` contract and 8,000 in the Curve pool.
3. Alice withdraws 10,000 crvUSD, causing 8,000 crvUSD to be withdrawn from the Curve pool. When `RToken.burn()` is executed, the amount is overwritten to 1 wei and the withdrawal is successful.

Recommendations

Revert when the amount to be burned is greater than the user's balance.

[H-06] Borrowed amounts can be insured for free

Severity

Impact: Medium

Likelihood: High

Description

`LendingPool` allows users to pay for insurance on borrowed assets. This insurance allows users to repay their debt in case of liquidation during a grace period.

However, on repayment, it is not checked if the amount insured can cover the amount being repaid, but only if the position is insured.

Consider the following scenario:

- Alice borrows 34 wei of crvUSD, paying a 3% insurance fee, which is 1 wei.
- Alice borrows 1,000 crvUSD using the `borrow()` function (no insurance).
- Alice's position is set for liquidation.
- Alice repays her debt of 1,000 tokens + 34 wei of crvUSD. She only insured 34 wei, so the insurance for the 1,000 crvUSD has been free.

Recommendations

Set `position.isInsured` to `false` in the `borrow()` function.

[H-07] Timelock withdrawal mechanism in `LendingPool` can be bypassed

Severity

Impact: Medium

Likelihood: High

Description

The lending pool has a configurable timelock duration for withdrawals that is initialized to 30 minutes. This is a common practice to prevent flash loan attacks and other types of atomic operations where a user can deposit and withdraw funds in a single transaction.

However, the timelock mechanism for withdrawals is flawed, as `requestWithdraw()` does not check if the user has enough balance to withdraw.

Users can preemptively request withdrawals for different amounts using different accounts and, after the timelock period, they can deposit and withdraw any of the requested amounts in the same transaction.

Recommendations

Check if the user has enough balance to withdraw in `requestWithdraw()`.

[H-08] First deposit front-running attack

Severity

Impact: High

Likelihood: Medium

Description

The `RWAVault` contract is subject to a common issue of ERC-4626 vaults, where the first deposit is front-run by an attacker.

1. A user will deposit assets into the contract to mint shares.

2. The attacker front-runs the user's transaction by minting one share and donating to the adapter 50% of the assets deposited by the user.
3. The user transaction is executed, and the shares minted are calculated as `depositedAssetValue * supplyBefore / assetsBefore`. As `assetsBefore` (assets in all the adapters) is inflated by the attacker's donation, the division is truncated, and the user mints only 1 share: $x * 1 / (0.5x + 1) = 1$.
4. Both the attacker and the user have 1 share each (50% of the total supply), but the amount deposited by the user represents 2/3 of the total assets.

Recommendations

There are different ways to mitigate this issue, including:

- Use of dead shares: Forces to deposit assets on the contract's deployment.
- Use of virtual deposit: OpenZeppelin's ERC4626 implements this solution and is well documented.

[H-09] Removing distribution pool in `RAACMinter` does not update `totalWeight`

Severity

Impact: High

Likelihood: Medium

Description

`RAACMinter.removeDistributionPool()` allows the updater role to remove a distribution pool. However, the `poolInfo` mapping is deleted before `totalWeight` is updated, so the weight of the pool is not subtracted from `totalWeight`.

```
function removeDistributionPool(address pool) external onlyRole
(UPDATER_ROLE) {
    if (!poolInfo[pool].exists) revert PoolDoesNotExist();
    poolInfo[pool].exists = false;
    @> delete poolInfo[pool];
    totalWeight = totalWeight - poolInfo[pool].weight;
    emit DistributionPoolRemoved(pool);
}
```

This will cause that the rewards distributed to the remaining pools will be lower than expected and some rewards will not be distributed to any pool.

Additionally, the distribution pool is not removed from the `distributionPools` array, keeping an unnecessary iteration in `_processMintedRewards()` with no effect.

Recommendations

```
- delete poolInfo[pool];  
  totalWeight = totalWeight - poolInfo[pool].weight;  
+ delete poolInfo[pool];
```

[H-10] **Treasury** deposit functions can be DoSed due to unsafe accounting

Severity

Impact: High

Likelihood: Medium

Description

The **Treasury** contract stores in the `_totalValue` state variable the total amount of all tokens deposited in the contract. As deposits can be made in any token through the `deposit()` and `distributeRewards()` functions, anyone can deposit a dummy token for an amount equal to `type(uint256).max - _totalValue`, causing every new call to `deposit()` or `distributeRewards()` to revert with an overflow error.

The `distributeRewards()` function is called by the fee collector for fee distribution, and the `deposit()` function is called by the stability pool for the liquidation process. The DoS of the liquidation process is of particular concern, as it is a time-sensitive process that should not be blocked in order to maintain the system collateralization.

Once the DoS is detected by the protocol team, the following actions would be required:

- To resume fee distributions, the treasury address should be changed in **FeeCollector** to a new address that does not have the overflow issue.
- To resume the liquidation process, given that the treasury address cannot be updated, treasury fees should be permanently disabled by setting `liquidationSplit.mintPercentage` to 0.

Recommendations

It is recommended to remove the `_totalValue` state variable, as it does not provide any useful information, given that is mixing the balances of different tokens.

[H-11] Randomness can be manipulated in **RWAVault** redemptions

Severity

Impact: High

Likelihood: Medium

Description

The RWAVault contract implements a Chainlink VRF mechanism to randomly select NFTs for redemption. This mechanism is designed to ensure fairness in the NFT selection process when users want to redeem their vault tokens for NFTs. However, a vulnerability exists in the implementation that allows users to manipulate the randomness to their advantage.

The RWAVault contract works as follows:

1. Users deposit assets into the vault using `depositAsset()` and receive vault tokens in return
2. When a user wants to redeem an NFT, they call `redeemNFT()` which:
 - Uses Chainlink VRF to randomly select an NFT from the vault.
 - Burns the user's vault tokens equivalent to the NFT's value.
 - Transfers the selected NFT to the user.
 - Requests new random words from the VRF consumer for the next redemption.

A user can repeatedly redeem NFTs and immediately deposit them back into the vault until they get their desired NFT. This effectively allows users to "game" the randomness mechanism by:

1. Calling `redeemNFT()` to get a randomly selected NFT.
2. If the NFT is not the one they want, immediately depositing it back via `depositAsset()`.
3. Since each redemption triggers a new VRF request (`vrfConsumer.requestRandomWords()`), the user gets a new random selection.
4. Repeating this process until they receive their desired NFT.

Proof of Concept


```

function testManipulateRandomness() public {
    rwa.mint(bob, 1);
    rwa.mint(alice, 2);

    bytes memory data = abi.encode(uint256(1));
    vm.startPrank(bob);
    rwa.approve(address(erc721VaultAdapter), 1);
    rwaVault.depositAsset(address(erc721VaultAdapter), data, bob);
    vm.stopPrank();

    data = abi.encode(uint256(2));
    vm.startPrank(alice);
    rwa.approve(address(erc721VaultAdapter), 2);
    rwaVault.depositAsset(address(erc721VaultAdapter), data, alice);
    vm.stopPrank();

    vrfConsumer.fulfillRandomWords();
    (, uint256 id) = rwaVault.getNextRandomNFT();
    console.log("Nft Before : ", id);

    // Let's say that bob want RWA number 2 so baddd
    uint256 i;
    for(i; i < 10; ++i) {
        data = abi.encode(uint256(id));
        vm.startPrank(bob);
        rwaVault.redeemNFT();
        rwa.approve(address(erc721VaultAdapter), id);
        rwaVault.depositAsset(address(erc721VaultAdapter), data, bob);
        vm.stopPrank();

        vrfConsumer.fulfillRandomWords();

        (, id) = rwaVault.getNextRandomNFT();
        if (id == 2) break;
    }

    console.log("Loop Num : ", i); // Loop number 4 in my case
    console.log("Nft After: ", id); // Real World Asset number 2

    // Now, bob can get the NFT
    vm.prank(bob);
    rwaVault.redeemNFT();
}

```

Test Setup: <https://gist.github.com/0xbtk/0c1f5e75815d2cf02d3437cbded026ff>

Recommendations

Implement a cooldown period for NFT re-deposits after redemption.

[H-12] Underflow forces full interest payment in debt token

Severity

Impact: Medium

Likelihood: High

Description

The `DebtToken` contract implements a debt accounting system where users can burn tokens to repay their debt, including accrued interest. The contract attempts to implement a flexible repayment mechanism that allows users to partially repay their debt and interest.

The issue occurs in the `burn()` function where the contract attempts to handle cases where a user's repayment amount is less than their accrued interest. The function first calculates `amountAfterPaidInterest = amount - scaledBalanceIncrease` without checking if `amount` is greater than `scaledBalanceIncrease` (the accrued interest). In Solidity versions $\geq 0.8.0$, this causes an arithmetic underflow and reverts the transaction when `amount < scaledBalanceIncrease`.

Proof of Concept

Consider a user with the following position:

1. Initial state:
 - Original debt: 1000 tokens
 - Accrued interest (`scaledBalanceIncrease`): 100 tokens
 - User has 50 tokens available for repayment
2. User attempts to make a partial repayment:

```
function burn(uint256 amount) { // amount = 50
    // ...
    uint256 amountAfterPaidInterest = amount - scaledBalanceIncrease;
    // 50 - 100 causes arithmetic underflow and reverts
    // Never reaches the else block that should handle this case
}
```

3. The transaction reverts due to underflow, even though the contract has logic to handle partial interest payments:

```
} else {
    // This block should handle amount < scaledBalanceIncrease
    // but is never reached due to the underflow
    scaledBalanceIncrease = scaledBalanceIncrease - amount;
    amountToBurn = 0;
}
```

Recommendations

Modify the burn function as follows:

```
// We pay the interest first, directly reducing the scaled increase
uint256 amountToBurn = amount;

// If we were able to pay back the interest, we won't have increase in
// the balance
// Instead, we will burn the difference.
if(amount > scaledBalanceIncrease){
    scaledBalanceIncrease = 0;
    amountToBurn = amount - scaledBalanceIncrease;
} else {
    // We were not able to pay back the interest, so we can only burn in
    // the debt side and mint the rest
    scaledBalanceIncrease = scaledBalanceIncrease - amount;
    amountToBurn = 0;
}
```

[H-13] Permanent loss of rewards when distribution fails

Severity

Impact: High

Likelihood: Medium

Description

The `RAACMinter` contract implements a reward distribution system where RAAC tokens are minted and distributed to various pools. The distribution process involves calculating rewards based on pool weights and then transferring these rewards through the `IRAACMinterRewardsReceiver` interface.

The issue occurs in the interaction between `_processMintedRewards()` and `_depositPendingReward()`. When distributing rewards, the contract first clears the `excessTokens` balance (setting it to 0) and then attempts to distribute the rewards to each pool. If a distribution fails, the contract catches the error but has no mechanism to recover the tokens that were meant for that distribution, as the `excessTokens` have already been cleared and the failed distribution amount is only stored in a memory variable that is discarded after the function execution.

Proof of Concept

1. Initial state:
 - `excessTokens = 1000`.
 - Pool A should receive 400 tokens.
 - Pool B should receive 600 tokens.
2. Distribution process starts:

```
function _processMintedRewards() internal {
    uint256 rewardPerWeight = excessTokens / totalWeight;
    excessTokens = 0; // Cleared before ensuring successful distribution
    for (uint256 i = 0; i < distributionPools.length; i++) {
        DistributionPool memory poolData = poolInfo[pool];
        poolData.pendingRewards += rewardPerWeight * poolData.weight;
        _depositPendingReward(poolData);
    }
}
```

3. Distribution to Pool A fails:

```
function _depositPendingReward(DistributionPool memory poolData) internal {
    try IRAACMinterRewardsReceiver(poolData.pool).deposit(...) {
        poolData.pendingRewards = 0
        // Success case
    } catch {
        emit DistributionFailed(poolData.pool, poolData.pendingRewards);
        // 400 tokens are now permanently lost as excessTokens = 0
        // and pendingRewards is only in memory
    }
}
```

4. Final state:

- Pool A's 400 tokens are permanently lost.
- Pool B received 600 tokens.
- 400 tokens are locked in the contract.

Recommendations

Modify the functions to store the `poolInfo` in storage instead of memory.

[H-14] Incorrect reward calculation in `_calculatePendingRewards`

Severity

Impact: High

Likelihood: Medium

Description

In BaseGauge, stakers will gain some rewards. The reward's calculation is based on the reward rate. In base gauge, we will get the reward rate from the reward distributor.

The problem here is that we may distribute the rewards in the distributor to different gauges. As one gauge, the reward rate will be lower than the reward rate in the distributor.

This will cause the reward calculation in the base gauge to be incorrect.

```

function _calculatePendingRewards(
    address token,
    address user
) internal view returns (uint256) {
    if (rd.distributor != address(0) && rd.distributor != controller) {
        try
            IGaugeRewardsDistributor(rd.distributor).getRewardRate(token)
        returns (uint256 rate) {
            if (rate > 0) {
                currentRate = rate;
            }
        } catch {}
    }
}

```

Recommendations

Calculate the reward rate according to the actual notified reward amount.

[H-15] Malicious users can distribute more rewards to the same gauge

Severity

Impact: Medium

Likelihood: High

Description

In GaugeRewardsDistributor contract, users can distribute reward tokens to gauges. After we distribute rewards to one gauge, we will reset `pendingAllocations[token][gauge]` to zero.

The problem here is that malicious users can re-distribute rewards to the same gauge again. In this case, we will re-allocate the rewards here. Function `_checkAndAddGaugeToCycle` here aims to add one new active gauge into the reward allocation. In function `_checkAndAddGaugeToCycle`, we take this gauge as the new gauge if `pendingAllocations` equals 0. The problem here is that `pendingAllocations` will be zero in the below 2 scenarios:

1. When we start this cycle, the gauge is not one active gauge.
2. This gauge is one active gauge, but we finished the cycle's distribution.

```

function distributeRewards
(address gauge, address token) external nonReentrant {
    _validateGaugeAndToken(gauge, token);

    if
        (!distributionCycleActive[token] || block.timestamp >= currentCycleEnd[token]) {
        // When we distribute rewards in one cycle, we will only distribute
        // to active gauges.
        _startDistributionCycle(token);
    }

    uint256 gaugeShare = _checkAndAddGaugeToCycle(token, gauge);

    _distributeRewardToGauge(gauge, token, gaugeShare);
}

function _checkAndAddGaugeToCycle
(address token, address gauge) internal returns (uint256) {
    uint256 gaugeShare = pendingAllocations[token][gauge];
    if (gaugeShare == 0 && distributionCycleBalance[token] > 0) {
        bool gaugeInCycle = false;
        for (uint256 i = 0; i < activeCycleGaugeCount[token]; i++) {
            if (activeCycleGauges[token][i] == gauge) {
                gaugeInCycle = true;
                break;
            }
        }
        if
            (!gaugeInCycle && activeCycleGaugeCount[token] < activeCycleGauges[token].length) {
            activeCycleGauges[token][activeCycleGaugeCount[token]] = gauge;
            activeCycleGaugeCount[token]++;
        }
        _recalculateAllocations(token);
        gaugeShare = pendingAllocations[token][gauge];
    }

    return gaugeShare;
}

function _distributeRewardToGauge
(address gauge, address token, uint256 gaugeShare) internal {
    if (gaugeShare > 0) {
        // Clear the allocation to prevent double-distribution
        pendingAllocations[token][gauge] = 0;
        // transfer reward to the gauge.
        IERC20(token).safeTransfer(gauge, gaugeShare);
        // notify rewards.
        IBaseGauge(gauge).notifyRewardAmount(token, gaugeShare);

        emit RewardDistributed(gauge, token, gaugeShare);
    }
}

```

Recommendations

Prevent re-distribution for the same gauge.

[H-16] Reward rate can be manipulated in first cycle

Severity

Impact: Medium

Likelihood: High

Description

In GaugeRewardsDistributor contract, users are allowed to add some rewards before the `firstCycleStart`. These rewards will be distributed between `current.timestamp` and `firstCycleStart`. The reward rate is calculated via `amount / firstCycleDuration`.

The problem here is that users can notify rewards multiple times in the first cycle duration. And we don't accrue the reward rate. The malicious users can notify rewards with one tiny amount to let the reward rate to 0 after users' normal notify rewards. Users will fail to get these rewards.

```
function _notifyRewardAmount(address token, uint256 amount) internal {
    // transfer reward token to this contract.
    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
    _handleRewardNotification(token, amount);
}

function _handleRewardNotification(address token, uint256 amount) internal {
    RewardData storage rd = rewardData[token];
    uint256 timestamp = block.timestamp;

    // Special handling for first cycle
    // firstCycleStart is the first next Thursday.
    if (timestamp < firstCycleStart) {
        // delta time between current timestamp and first cycle start.
        uint256 firstCycleDuration = getFirstCycleDuration();
        if (firstCycleDuration > 0) {
            // Calculate rate for partial first cycle
            rd.rate = amount / firstCycleDuration;
            rd.lastUpdateTime = timestamp;
            rd.periodFinish = firstCycleStart;
            return;
        }
    }
}
```

Recommendations

Accrue the rewards in the first cycle.

[H-17] Incorrect totalSupply calculation

Severity

Impact: Medium

Likelihood: High

Description

In a lending pool, we use RToken as the lender's share and use DebtToken as the borrower's debt share. Lenders' raw balance means lenders' principle. Each lender has their own liquidity index. When users want to withdraw, we will calculate the loan interest based on the difference between the current liquidity index and this lender's

index. Debt token is one similar case. It means that users' raw balances no matter RToken or DebtToken are not normalized.

If the raw balance is not normalized, when we multiply the normalization index, the result is incorrect. The result might be larger than the expected result. All functions which depend on `totalSupply` will be impacted.

```
function totalSupply() public view override(ERC20, IERC20) returns
(uint256) {
    return super.totalSupply().rayMul(ILendingPool
        (_lendingPool).getNormalizedIncome());
}
```

Recommendations

In order to calculate the total supply conveniently, we should normalize our share.

[H-18] Inflation attack in RWA Vault

Severity

Impact: Medium

Likelihood: High

Description

If one borrows position is unhealthy, we can liquidate this unhealthy position. In the liquidation process, we will swap one part of the index token into crvUSD token. We will calculate the `minDy` based on the formula $(\text{amount} * (10000 - \text{liquidityPoolFee})) / 10000$. This can work well based on one assumption that the index token's price equals crvUSD price.

The problem is that the index token's price can be manipulated via donation. The first depositor can mint 50 wei to get 49 shares, assume the minting fee is 2%. Then the depositor can donate some assets into one vault adapter to increase `totalAssets`. The index token's price will increase. The later depositors will deposit with one higher share price.


```

function _swap(address sender, uint256 amount) internal {
    IERC20(vaultToken).safeTransferFrom(sender, address(this), amount);

    bool approveSuccessIndexToken = IERC20(vaultToken).approve(address
        (liquidityPool), amount);
    if (!approveSuccessIndexToken) revert ApprovalFailed();

    uint256 minDy = (amount * (10000 - liquidityPoolFee)) / 10000;
    ...
}
function _deposit(
    address adapter,
    bytes calldata data,
    address receiver,
    uint256 mintFeePercentage
) internal returns (uint256) {
    if (receiver == address(0)) revert InvalidAddress();

    uint256 assetsBefore = totalAssets();

    uint256 supplyBefore = IVaultToken(vaultToken).totalSupply();

    uint256 depositedAssetValue = IVaultAssetAdapter(adapter).deposit
        (data, msg.sender);
    uint256 sharesMinted;

    if (supplyBefore == 0 || assetsBefore == 0) {
        sharesMinted = depositedAssetValue; // 1:1 for the first depositor
    } else {
        sharesMinted = (depositedAssetValue * supplyBefore) / assetsBefore;
    }
}
function totalAssets() public view override returns (uint256 totalValue) {
    // loop adapters.
    for (uint256 i = 0; i < adapters.length; i++) {
        totalValue += IVaultAssetAdapter(adapters[i]).totalValue();
    }
    return totalValue;
}
function totalValue() external view returns (uint256) {
    uint256 amount = token.balanceOf(address(this));
    return _assetValue(amount);
}

```

Recommendations

Take the `minDy` as one input parameter, the owner needs to set this `minDy` properly.

[H-19] Users may fail to withdraw from lending pool

Severity

Impact: Medium

Likelihood: High

Description

In a lending pool, lenders can withdraw their crvUSD via `withdraw`. If current buffer is not enough to cover this withdrawal, we need to withdraw some crvUSD from the curve

vault.

We will calculate the `requiredAmount` from the curve vault according to the current liquidity. We will withdraw `requiredAmount` from the curve vault. The problem here is that we will charge some yield from the `requiredAmount`. This will cause the RToken contract not to have enough crvUSD amount to pay this withdrawal.

```
function ensureLiquidity(uint256 amount) external onlyProxy() {
    uint256 availableLiquidity = IERC20(reserve.reserveAssetAddress).balanceOf
    (reserve.reserveRTokenAddress);
    uint256 requiredAmount = amount - availableLiquidity;
    uint256 withdrawFromVaultAmount = requiredAmount;
    if (withdrawFromVaultAmount > maxWithdrawable) {
        withdrawFromVaultAmount = maxWithdrawable;
    }
    if (withdrawFromVaultAmount > 0) {
        withdrawFromVault(withdrawFromVaultAmount);
    }
    ...
}

function withdrawFromVault(uint256 amount) internal {
    uint256 currentSharePrice = vault.pricePerShare(true);
    // withdraw from vault. Now crvUSD is in the Lending pool contract.
    uint256 burnedShares = vault.withdraw(amount, address(this), address
    (this));

    uint256 yield;
    if (currentSharePrice > vaultRewards.lastSharePrice) {
        uint256 priceDiff = currentSharePrice - vaultRewards.lastSharePrice;
        yield = (burnedShares * priceDiff) / 1e18;
        vaultRewards.unclaimedRewards += yield;
    } else {
        yield = 0;
    }
    uint256 transferAmount = amount - yield;
    // Here we will transfer crvUSD into RToken.
    IERC20(reserve.reserveAssetAddress).safeTransfer
    (reserve.reserveRTokenAddress, transferAmount);
}
```

Recommendations

When we calculate the `requiredAmount`, we need to consider the possible `yield` deduction.

[H-20] Users can avoid insurance fee when reborrowing debt

Severity

Impact: Medium

Likelihood: High

Description

In a lending pool, users can repay the debt via `repay` function. After we repay all debt, we will reset `position.rawInsuredBalance`. According to the comment, `This will make sure that if the user takes a loan again, the insurance will be reset and the user will have to insure again.`.

The problem here is that we don't reset `position.isInsured` to false. It means that after we repay all debt for one borrow position, we use take one loan again via `borrow` function. And this new loan will be with insurance because we miss reset `position.isInsured`.

```
function _repay(
    addressadapter,
    bytescalldata,
    uint256amount,
    addressonBehalfOf
) internal {
    if (position.rawInsuredBalance > position.rawDebtBalance) {
@>        position.rawInsuredBalance = position.rawDebtBalance;
    }
}
```

Recommendations

Reset the `position.isInsured` when we repay all debt.

[H-21] Users may be forced to pay more interest

Severity

Impact: Medium

Likelihood: High

Description

In a lending pool, users can repay other users' debt via the function `repayOnBehalf`. Once we repay one part of debt, we will recalculate this borrow position's raw balance(principle). If malicious users repay 1 wei debt for one borrow position, the borrow position's total debt does not change. However, we will add the pending borrow interest into the `raw balance`. The borrow position may have to pay more borrow interest because the raw balance is increased.

For example:

1. Alice's position's raw balance is 1000. Usage index is $1e27$.
2. The usage index increases to $1.1 * 1e27$.
3. Malicious users repay 1 wei for Alice's borrow position. The updated raw balance is

$$1000 + 1000 * (1.1 - 1) = 1100.$$

4. The usage index increases to $1.2 * 1e27$.
5. Alice wants to repay the borrow position, the total debt is $1100 + 1100 * (1.2 - 1.1) = 1210$.
6. If malicious users don't manipulate Alice's position, the total debt is $1000 + 1000 * (1.2 - 1) = 1200$.

```
function borrow(
    addressadapter,
    bytescalldata,
    uint256amount
) external nonReentrant whenNotPaused onlyValidAmount(amount)
    _borrow(adapter, data, amount);
}

function borrowWithInsurance(
    addressadapter,
    bytescalldata,
    uint256amount
) external nonReentrant whenNotPaused onlySupportedAdapter(adapter)

    uint256 uninsuredBalance = getUninsuredBalance(
        adapter, msg.sender, data);
    uint256 insuranceFeeAmount = calculateInsuranceFee(
        adapter, msg.sender, data, amount);
    ...
}

function calculateInsuranceFee(
    addressadapter,
    addressuser,
    bytescalldata,
    uint256amount
) public view returns (uint256)
    uint256 uninsuredBalance = getUninsuredBalance(adapter, user, data);
    // Here we will charge the uninsured balance for insurance fee.
    return (uninsuredBalance + amount).percentMul(
        parameters.insuranceFee);
}
```

Recommendations

Add one minimum repaid debt or users cannot repay debt for one borrow position before the borrow position's owner's approval.

[H-22] Users can steal rewards from SP

Severity

Impact: Medium

Likelihood: High

Description

In the Stable pool, users deposit RToken to get some DETokens. When users withdraw DEToken, users can get some rewards. The users' rewards will be calculated via the formula $\text{userDeposits} * \text{totalRewards} / \text{totalDeposits}$.

The problem here is that we don't record users' claimed rewards. Then malicious users can withdraw 1 wei deToken repeatedly. We can steal rewards in the sp.

```
function withdraw
(uint256 deCRVUSDAmount) external nonReentrant whenNotPaused validAmount(deCRVUSDAmount) {
    _update();
    if (deToken.balanceOf
        (msg.sender) < deCRVUSDAmount) revert InsufficientBalance();
    // convert deToken to RToken amount. deToken: RToken = 1:1
    uint256 rcrvUSDAmount = calculateRcrvUSDAmount(deCRVUSDAmount);
    uint256 raacRewards = calculateRaacRewards(msg.sender);
    if
        (userDeposits[msg.sender] < rcrvUSDAmount) revert InsufficientBalance();
    userDeposits[msg.sender] -= rcrvUSDAmount;

    if (userDeposits[msg.sender] == 0) {
        delete userDeposits[msg.sender];
    }

    deToken.burn(msg.sender, deCRVUSDAmount);
    rToken.safeTransfer(msg.sender, rcrvUSDAmount);
    if (raacRewards > 0) {
        raacToken.safeTransfer(msg.sender, raacRewards);
    }
}

function calculateRaacRewards(address user) public view returns (uint256) {
    uint256 userDeposit = userDeposits[user];
    uint256 totalDeposits = deToken.totalSupply();

    uint256 totalRewards = raacToken.balanceOf(address(this));
    if (totalDeposits < 1e6) return 0;

    return (totalRewards * userDeposit) / totalDeposits;
}
```

Recommendations

Calculate the reward per de token and record the claimed reward.

[H-23] FeeCollector will be dos

Severity

Impact: Medium

Likelihood: High

Description

In FeeCollector, users can claim NFT's underlying via function `claimNFTUnderlying`. We expect to withdraw tokens from `target` and update the `collectedFee`'s accountant. The problem here is that we miss the input `target`'s security check, then malicious users can add one malicious target and we will not transfer any underlying token back via this low-level call. The malicious users can manipulate one very large amount to increase the collected fee a lot.

Malicious users can trigger this function repeatedly to increase `treasuryAmount`, `repairFundAmount`, etc to let one of them reach near to `uint256.maximum`.

The impact is as below:

1. If the lending pool or some other contracts want to collect some fees, the normal operation may be reverted because of the possible overflow.
2. If we want to distribute fees via function `claimNFTUnderlying`, we will fail because we don't have enough tokens to distribute.

```
function claimNFTUnderlying(
    address token,
    address target,
    uint256 amount,
    bytes32 feeType
) external nonReentrant whenNotPaused returns (bool)
    if (!isTokenSupported[token]) revert TokenNotSupported();
    if (amount == 0) revert InvalidFeeAmount();

    (bool success,) = target.call(
        abi.encodeWithSelector(bytes4(keccak256("withdrawUnderlying")), token, address(this), amount)
    );

    if (!success) revert ClaimCollectorUnderlyingFailed();

    _updateCollectedFees(token, feeType, amount);
    emit CollectorUnderlyingClaimed(token, target, amount);
    return true;
}

function _updateCollectedFees(
    address token, bytes32 feeType, uint256 amount) internal {
    FeeType memory feeTypeData = feeTypes[feeType];
    uint256 treasuryAmount = amount.percentMul(feeTypeData.treasuryShare);
    uint256 repairFundAmount = amount.percentMul(
        feeTypeData.repairFundShare);
    uint256 burnAmount = amount.percentMul(feeTypeData.burnShare);
    uint256 veRAACTokenAmount = amount.percentMul(
        feeTypeData.veRAACTokenShare);
    uint256 raacCorpAmount = amount.percentMul(feeTypeData.raacCorpShare);
    uint256 otherAmount = amount.percentMul(feeTypeData.otherShare);

    uint256 totalTax = treasuryAmount + repairFundAmount + burnAmount + veRAACTokenA
    ...
    @> collectedFee.treasuryAmount += treasuryAmount;
    @> collectedFee.repairFundAmount += repairFundAmount;
    collectedFee.burnAmount += burnAmount;
    collectedFee.veRAACTokenAmount += veRAACTokenAmount;
    collectedFee.raacCorpAmount += raacCorpAmount;
    collectedFee.otherAmount += otherAmount;
}
```

Recommendations

Add one sanity check for the input `target`.

[H-24] Unclosed liquidation blocks collateral recovery

Severity

Impact: High

Likelihood: Medium

Description

When a position is liquidated, the user must:

1. Call `LendingPool._repay()` to zero out their debt.
2. Call `LiquidationProxy.closeLiquidation()` to clear `position.isUnderLiquidation`.

If this step is omitted or delayed past the grace period, the position remains flagged under liquidation forever—even though `rawDebtBalance == 0`.

```
File: LiquidationProxy.sol
78:         if
      (block.timestamp > position.liquidationStartTime + parameters.liquidationGracePeriod) {
79:@>             revert GracePeriodExpired();
80:         }
81:
82:         // The liquidation can be closed onyl if the asset is insured. Since
      // repay is blocked, this will be automatically, but just in case.
83:         if (!position.isInsured) revert NotInsured();
84:
85:         // update state
86:         ReserveLibrary.updateReserveState(reserve, rateData);
87:
88:         uint256 positionDebt = getPositionScaledDebtProxy
      (adapter, userAddress, data);
89:
90:@>         if (positionDebt > 0) revert DebtNotZero();
91:
92:@>         position.isUnderLiquidation = false;
```

This state prevents:

- Collateral withdrawal via `withdrawAsset()` (even when debt is zero), which reverts at:

```
File: LendingPool.sol
385:     function withdrawAsset(
        address adapter,
        bytes calldata data
    ) external nonReentrant whenNotPaused onlySupportedAdapter(adapter
386:         CollateralPosition memory position = getPosition
        (adapter, msg.sender, data);
387: @>     if
        (position.isUnderLiquidation) revert CannotWithdrawUnderLiquidation();
```

- StabilityPool finalization, since `LiquidationStrategyProxy.liquidateBorrower()` reverts early when `debt == 0`:

```
File: LiquidationStrategyProxy.sol
40:     function liquidateBorrower(
        address poolAdapter,
        address vaultAdapter,
        address user,
        bytes calldata data
    ) external onlyProxy {
    ...
48:         // Get the user's debt from the LendingPool.
49: @>         if (lendingPool.getPositionDebt
        (poolAdapter, user, data) == 0) revert InvalidAmount();
```

The following test walks through a user borrowing with insurance, then triggers liquidation by dropping the house price and calling `initiateLiquidation()`; it then advances time by the full 72-hour grace period, has the user repay their entire debt without calling `closeLiquidation()`, and finally, after one more second, attempts `closeLiquidation()` and verifies it reverts with `GracePeriodExpired`, demonstrating that the position remains stuck under liquidation despite zero debt.


```
// File: LendingPool.test.js
it("0xbepresentCloseLiquidationTimeout", async function () {
  const borrowAmount = ethers.parseEther("80");
  const insuranceFee = await lendingPool.calculateInsuranceFee
    (assetAdapter.target, user1.address, assetData, borrowAmount);
  await crvusd.connect(user1).approve(feeCollector.target, insuranceFee);
  await lendingPool.connect(user1).borrowWithInsurance
    (assetAdapter.target, assetData, borrowAmount);
  expect((await lendingPool.getPosition
    (assetAdapter.target, user1.address, assetData)).isInsured).to.be.true
  await time.increase(16000);
  await ethers.provider.send("evm_mine");
  await lendingPool.connect(user1).updateState();
  // Decrease house price and initiate liquidation
  await raacHousePrices.setHousePrice(1, ethers.parseEther("90"));
  await erc20PriceOracle.setLatestPrice(ethers.parseEther("0.9"))
  await lendingPool.connect(user2).initiateLiquidation
    (assetAdapter.target, user1.address, assetData);
  //
  // 1. User1 repays full debt
  // (it repays in the last time of grace period) but not call `closeLiquidation`
  await ethers.provider.send("evm_increaseTime", [72 * 60 * 60]);
  const userScaledDebt = await lendingPool.getPositionScaledDebt
    (assetAdapter.target, user1.address, assetData);
  const repaymentAmount = userScaledDebt + ethers.parseEther("1");
  await crvusd.connect(user1).approve
    (lendingPool.target, repaymentAmount);
  await lendingPool.connect(user1).repay
    (assetAdapter.target, assetData, repaymentAmount);
  //
  // 2. Time pass more than grace period and user calls `closeLiquidation`
  // but the tx will be reverted. Since the liquidation is not closed, the user is still un
  await ethers.provider.send("evm_increaseTime", [1]);
  await ethers.provider.send("evm_mine");
  await lendingPool.connect(user1).updateState();
  await expect(lendingPool.connect(user1).closeLiquidation
    (assetAdapter.target, assetData))
    .to.be.revertedWithCustomError(lendingPool, "GracePeriodExpired");
});
```

Recommendations

Update `closeLiquidation()` so that it checks the user's current debt before enforcing the grace-period expiration: if `getPositionScaledDebtProxy(...)` returns zero, the function should bypass the grace-period check and allow the position to be closed even after the deadline.

```
function closeLiquidation
  (address adapter, bytes calldata data) external onlyProxy {
  ...
  - if
  - (block.timestamp > position.liquidationStartTime + parameters.liquidationGracePeriod) {
  + if
  + (postionDebt > 0 && block.timestamp > position.liquidationStartTime + parameters.liquidationGra
    revert GracePeriodExpired();
  }
```

[H-25] Lock overwrite in `createLock` causes undercredited withdrawals

Severity

Impact: High

Likelihood: Medium

Description

In `LockManager.createLock`, the per-user lock record is blindly overwritten on every invocation, without checking for a pre-existing lock. As a result, calling `veRAACToken.lock()` twice (even across different blocks) will replace the first `Lock` struct with the second, while the protocol still debits the user both times. This causes `getLockPosition()` to report only the **latest** amount and makes `withdraw()` return only that amount.

```
// File: LockManager.sol
275:     function createLock(
276:         LockState storage state,
277:         address user,
278:         uint256 amount,
279:         uint256 duration
280:     ) internal returns (uint256 end) {
...
...
287:
288:     uint256 start = block.timestamp;
289:     end = start + duration;
290:
291:     Lock memory newLock = Lock({
292:         amount: amount,
293:         start: start,
294:         end: end,
295:         exists: true
296:     });
297:
298:@>     state.locks[user] = newLock;
299:@>     state.totalLocked += amount;
```

The following test shows how the `veRAACToken.lock()` function is called twice with time difference. The same amount is paid twice, but only one amount is recorded in `locked positions`. As a result, during the `withdraw`, it's not possible to retrieve both amounts since only one was registered.

```
// File: veRAACToken.test.js
it("0xbepresentlockoverwritten", async () => {
  const amount = ethers.parseEther("1000");
  const duration = 365 * 24 * 3600; // 1 year

  let raacTokenBalanceBefore = await raacToken.balanceOf(
    users[0].address);
  //
  // 1. Call lock two times with time difference
  await veRAACToken.connect(users[0]).lock(amount, duration);
  await time.increase(duration + 1); // time diff
  await veRAACToken.connect(users[0]).lock(amount, duration);
  //
  // 2. Check if the raacToken balance is decreased two times
  let raacTokenBalanceAfter = await raacToken.balanceOf(
    users[0].address);
  expect(raacTokenBalanceBefore - raacTokenBalanceAfter).toEqual(
    (BigInt(amount) * 2n));
  //
  // 3. Lock position has wrong amount because it was charged twice
  // but the lockPosition.amount has only one amount not two.
  let lockedPosition = await veRAACToken.getLockPosition(
    users[0].address);
  expect(lockedPosition.amount).toEqual(amount);
  await time.increase(duration + 1);
  //
  // 6. The withdraw wrongly returns only one amount which is wrong
  // because the user locked two times.
  await expect(veRAACToken.connect(users[0]).withdraw())
    .to.emit(veRAACToken, "Withdrawn")
    .withArgs(users[0].address, amount);
  raacTokenBalanceAfter = await raacToken.balanceOf(users[0].address);
  expect(raacTokenBalanceBefore - raacTokenBalanceAfter).toEqual(
    (BigInt(amount)));
});
```

Recommendations

Disallow multiple locks per user:

```
// At top of veRAACToken.lock():
function lock(
  uint256 amount,
  uint256 duration
) external nonReentrant whenNotPaused {
+   LockManager.Lock memory userLock = _lockState.locks[msg.sender];
+   if (userLock.amount != 0) revert LockFound();
```

[H-26] Stale interest accrual allows over-borrowing

Severity

Impact: High

Likelihood: Medium

Description

In the `LendingPool._borrow()` function, the call to `LendingPool._validateBorrow()` occurs **before** `ReserveLibrary.updateReserveState` (which accrues interest and updates scaled debt indices). As a result, `_validateBorrow` sees an **under-reported** debt balance:

```
File: LendingPool.sol
422:     function _borrow
    (address adapter, bytes calldata data, uint256 amount) internal {

424:         if
    (position.isUnderLiquidation) revert CannotBorrowUnderLiquidation();
425:
426:         // Will revert if the user does not have enough collateral for the
    // borrow
427:@>     _validateBorrow(adapter, data, amount);
428:
429:         // Update reserve state before borrowing
430:@>     ReserveLibrary.updateReserveState(reserve, rateData);
...
1033:    function _validateBorrow
    (address adapter, bytes calldata data, uint256 amount) internal view {
1034:        // check that the amount does not exceed borrow
1035:        require(IDebtToken(reserve.reserveDebtTokenAddress).totalSupply
    //( ) + amount <= parameters.borrowCap, "borrow cap reached"); // totalsupply because raw total sup
1036:        // For the borrow, we need to ensure that the user has enough
    // collateral to cover the new debt
1037:        uint256 collateralValue = IAssetAdapter(adapter).getAssetValue
    (msg.sender, data);
1038:        if (collateralValue == 0) revert NoCollateral();
1039:
1040:        // We calculate the max debt that the user can have
    //(based on the collateral value and the liquidation threshold)
1041:        uint256 maxDebt = collateralValue.percentMul
    (parameters.liquidationThreshold);
1042:
1043:        // We ensure that the position has enough collateral to cover the
    // new debt or revert
1044:@>        if (maxDebt < getPositionDebt
    (adapter, msg.sender, data) + amount) {
1045:            revert NotEnoughCollateralToBorrow();
1046:        }
1047:    }
```

The `ReserveLibrary.updateReserveState()` ->
`ReserveLibrary.updateReserveInterests()` function uses the elapsed time since
`reserve.lastUpdateTimestamp` to accrue both liquidity and debt indices:

```

File: ReserveLibrary.sol
121:     function updateReserveInterests
    (ReserveData storage reserve, ReserveRateData storage rateData) internal {
122:@>         uint256 timeDelta = block.timestamp - uint256
    (reserve.lastUpdateTimestamp);
123:         if (timeDelta < 1) {
124:             return;
125:         }
126:
127:         uint256 oldLiquidityIndex = reserve.liquidityIndex;
128:         if (oldLiquidityIndex < 1) revert LiquidityIndexIsZero();
129:
130:         // Update liquidity index using linear interest
131:@>         reserve.liquidityIndex = calculateLiquidityIndex(
132:             rateData.currentLiquidityRate,
133:@>             timeDelta,
134:             reserve.liquidityIndex
135:         );
136:
137:         // Update usage index (debt index) using compounded interest
138:@>         reserve.usageIndex = calculateUsageIndex(
139:             rateData.currentUsageRate,
140:@>             timeDelta,
141:             reserve.usageIndex
142:         );
143:
144:         // Update the last update timestamp
145:         reserve.lastUpdateTimestamp = uint40(block.timestamp);
146:
147:         emit ReserveInterestsUpdated
    (reserve.liquidityIndex, reserve.usageIndex);
148:     }

```

Because `_validateBorrow` reads debt via the old `reserve.usageIndex` (also see report [Underestimated position debt in collateral and withdrawal validation](#)), any positive `timeDelta` (≥ 1 second) causes actual debt to exceed the “raw” principal used for validation. A borrower can thus slip in extra debt until the next reserve update, bypassing intended collateral limits.

Consider the following scenario:

1. Initial state:

- UserA has borrowed to `99 rToken` of a `100 rToken maxDebt` based on the collateral.
- Reserve’s last interest `reserve.usageIndex` is at 1.00.

2. Time passes:

- Interest accrues, raising the user’s true debt to `105 rToken`, but `reserve.usageIndex` remains stale until `updateReserveState` is called.

3. Borrow invocation:

- UserA calls `_borrow(adapter, data, 1 rToken)`.
- `_validateBorrow` reads `getPositionDebt() = 99 rToken` (since raw principal is unchanged). Validation passes (`99 + 1  $\leq$  100`).

4. Post-validation update:

- `updateReserveState` accrues interest, now user's scaled debt = 105 rToken + 1 new debt = 106 rToken.
- Borrow proceeds, minting an extra 1 rToken.

5. Result:

- User A holds `106 rToken` debt against `100 rToken` `maxDebt limit` -> under-collateralized position, requiring emergency liquidation or risking protocol loss.

Recommendations

Ensure interest is accrued first, then perform collateral checks:

```
422:     function _borrow
      (address adapter, bytes calldata data, uint256 amount) internal {
    ...
426:         // 1) Reserve state update to accrue interest
427:         ReserveLibrary.updateReserveState(reserve, rateData);
428:
429:         // 2) Debt validation using up-to-date scaled debt
430:         _validateBorrow(adapter, data, amount);
```

[H-27] Unrestricted `FeeCollector.collectFee` enables unauthorized fee extraction

Severity

Impact: Medium

Likelihood: High

Description

The `LendingPool.borrowWithInsurance` flow requires users to pre-approve the insurance fee to the `feeCollector`, then with the help of function `LendingPool::_collectInsuranceFee`, the fees are transferred to `feeCollector`.

```

File: LendingPool.sol
464:     function borrowWithInsurance(
        addressadapter,
        bytescalldatadata,
        uint256amount
    ) external nonReentrant whenNotPaused onlySupportedAdapter(adapter
    ...

471:@>         if (IERC20(reserve.reserveAssetAddress).allowance(
            IERC20
        ).allowance
        (msg.sender, address(feeCollector
472:@>         if (IERC20(reserve.reserveAssetAddress).balanceOf
        (msg.sender) < insuranceFeeAmount) revert InsufficientBalance();
473:
474:         _borrow(adapter, data, amount);
475:
476:         // Transfer insurance fee to fee collector
477:@>         _collectInsuranceFee(insuranceFeeAmount);
    ...
482:     }
    ...
    ...
1192:     function _collectInsuranceFee(uint256 amount) internal {
1193:         if (address(feeCollector) == address
            (0)) revert AddressCannotBeZero();
1194:         if (amount == 0) revert InvalidAmount();
1195:
1196:@>         IBaseCollector(feeCollector).collectFee
            (reserve.reserveAssetAddress, msg.sender, amount, keccak256("INSURANCE_FEE"));
1197:     }

```

However, the `FeeCollector.collectFee` function is public and unrestricted:

```

File: FeeCollector.sol
324:     function collectFee(
        addresstoken,
        addresstarget,
        uint256amount,
        bytes32feeType
    ) external nonReentrant whenNotPaused returns (bool
325:         if
        (amount == 0 || amount > MAX_FEE_AMOUNT) revert InvalidFeeAmount();
326:         if (feeTypes[feeType].feeType == bytes32
        (0)) revert FeeTypeDoesNotExist();
327:         if (!isTokenSupported[token]) revert TokenNotSupported();
328:
329:         // Transfer tokens from target to this contract
330:@>         IERC20(token).safeTransferFrom(target, address(this), amount);
331:
332:         // Update collected fees for this token
333:         _updateCollectedFees(token, feeType, amount);
334:
335:         emit FeeCollected(feeType, amount);
336:         return true;
337:     }

```

Because there is **no access control** on `collectFee`, an attacker can invoke and siphon off the victim's approved tokens via frontrun attack whether the victim ever calls `borrowWithInsurance`. Consider the next scenario:

1. Victim pre-approves `FeeCollector`.
2. Victim calls `LendingPool.borrowWithInsurance` but attacker drains (frontrun) allowance using the `FeeCollector.collectFee()`.
3. Victim loses approved tokens without borrowing.

Recommendations

Instead of requiring pre-approval, perform a **pull** from the user's balance within the `borrowWithInsurance()` function. Similar to `LendingPool._collectLendingFee()`.

[H-28] Deposits during liquidation enable collateral lockup

Severity

Impact: High

Likelihood: Medium

Description

The function `StabilityPool.liquidateBorrower(...)` ultimately calls the adapter's

```
File: LiquidationStrategyProxy.sol
40:     function liquidateBorrower(
        addresspoolAdapter,
        addressvaultAdapter,
        addressuser,
        bytescallldata
    ) external onlyProxy {
    ...
55:         // If the user data parameter is wrong, especially in ERC20 amount
    // where amountn != collateral value
56:@>     if (!IAAssetAdapter(poolAdapter).validateLiquidationData
        (user, data)) revert CollateralAndParamterDataMismatch();
```

which enforces that the user's stored balance exactly equals the expected **liquidation amount** (liquidator's data parameter):

```
File: ERC20AssetAdapter.sol
175:     function validateLiquidationData
        (address user, bytes calldata data) external view returns(bool) {
176:         uint256 amount = abi.decode(data, (uint256));
177:@>     return _balances[user] == amount;
178:     }
```

However, the borrower can invoke `depositAsset` and increase their `_balances[user]` at any time—even while under liquidation:

```
File: LendingPool.sol
365:     function depositAsset(
        addressadapter,
        bytescallldata
    ) external nonReentrant whenNotPaused onlySupportedAdapter(adapter
366:         if (!vaultOpened[msg.sender]) revert VaultPositionNotOpened();
367:         // update state
368:         ReserveLibrary.updateReserveState(reserve, rateData);
369:@>     IAAssetAdapter(adapter).deposit(msg.sender, data);
370:     }
```



```

File: ERC20AssetAdapter.sol
67:     function deposit
    (address user, bytes calldata data) external override onlyLendingPool {
68:         uint256 amount = abi.decode(data, (uint256));
69:         require(amount > 0, "ERC20Adapter: invalid amount");
70:
71:         // Effects: update balance
72: @>         _balances[user] += amount;
73:
74:         // Interaction: transfer tokens
75:         token.safeTransferFrom(user, address(this), amount);
76:
77:         emit TokenDeposited(address(token), user, amount);
78:     }

```

By front-running the liquidator with a minimum amount of deposit, `validateLiquidationData` will revert, aborting liquidation and locking collateral indefinitely. Test:

```

// File: StabilityPool.test.js
it("0xbepresent02 frontrun liquidateBorrower", async function() {
    await stabilityPool.setLiquidationVaultTokenSplit
        (2_00, 2_00, 90_00, 6_00)
    await time.increase(86400);
    await ethers.provider.send("evm_mine");
    await lendingPool.initiateLiquidation
        (poolAdapter.target, user3.address, assetEncodedData);
    await lendingPool.connect(user3).updateState();
    //
    // 1. Attacker frontrun and deposits 1 asset
    let attackerAssetEncodedData = encodeERC20Token(1);
    await baseAsset.mint(user3.address, 1);
    await baseAsset.connect(user3).approve(poolAdapter.target, 1);
    await lendingPool.connect(user3).depositAsset
        (poolAdapter.target, attackerAssetEncodedData);
    //
    // 2. Liquidation is reverted
    await expect(
        stabilityPool.liquidateBorrower(
            poolAdapter.target,
            vaultAdapter.target,
            user3.address,
            assetEncodedData
        )
    ).to.be.revertedWithCustomError
        (stabilityPool, "CollateralAndParamterDataMismatch");
})

```

Recommendations

Block deposits during liquidation in `LendingPool.depositAsset()`, add a check:

```

if (position.isUnderLiquidation) revert DepositsDisabledDuringLiquidation();

```

This prevents any balance changes once liquidation has begun.

[H-29] `GaugeRewardsDistributor` distributes rewards to gauges not registered

Severity

Impact: Medium

Likelihood: High

Description

When `GaugeRewardsDistributor.distributeRewards()` is called and a new distribution cycle is started, the token balance is distributed to all active gauges in the `GaugeController` contract.

```
address[] memory activeGauges = gaugeController.getActiveGauges();
for (uint256 i = 0; i < activeGauges.length; i++) {
    allActiveGauges[count] = activeGauges[i];
    count++;
    if (count >= MAX_ACTIVE_GAUGES) break; //No more than 100 gauges
}

// Store active gauges for this cycle
activeCycleGauges[token] = allActiveGauges;
```

This is incorrect, as the tokens should only be distributed to the gauges that have been registered to receive rewards of the specific token using the `addGaugeRewardToken()` function.

As a result, the distributor will distribute part of the tokens to gauges that were not meant to receive them, distributing fewer tokens to the intended gauges. Additionally, some gauges might not support the token, so will not be able to receive them.

Recommendations

```
for (uint256 i = 0; i < activeGauges.length; i++) {
+     if (gaugeRewardTokens[activeGauges[i]][token]) {
        allActiveGauges[count] = activeGauges[i];
        count++;
        if (count >= MAX_ACTIVE_GAUGES) break; //No more than 100 gauges
+     }
}
```

[H-30] `getNextCycleStart()` returns incorrect value at the end of a cycle

Severity

Impact: Medium

Likelihood: High

Description

When a distribution cycle ends `GaugeRewardsDistributor.getNextCycleStart()` is used to set the new cycle start.

However, when this function is called exactly at the end of a cycle, the value returned is `block.timestamp + DURATION` instead of `block.timestamp`, so `rd.periodFinish` is set to `block.timestamp + DURATION * 2`, extending the cycle by one more period (1 week).

This will have important consequences, such as an overestimation of the leftover rewards, both in `GaugeRewardsDistributor` and `BaseGauge`, breaking the rewards accumulation logic.

```
File: GaugeRewardsDistributor.sol
    } else {
        // Middle of existing cycle
        uint256 remaining = rd.periodFinish - timestamp;
        uint256 leftover = remaining * rd.rate;
```

```
File: BaseGauge.sol
    if (rewardPeriodActive) {
        // Calculate remaining rewards in the current period
        uint256 remaining = rd.periodFinish - block.timestamp;
        uint256 leftover = remaining * rd.rate;
```

Recommendations

```
function getNextCycleStart() public view returns (uint256) {
    if (block.timestamp < firstCycleStart) {
        return firstCycleStart;
    }

    uint256 timeElapsedSinceFirst = block.timestamp - firstCycleStart;
    -   uint256 completedCycles = timeElapsedSinceFirst / DURATION;
    +   uint256 completedCycles =
    + (timeElapsedSinceFirst + DURATION - 1) / DURATION;
    -   return firstCycleStart + (completedCycles + 1) * DURATION;
    +   return firstCycleStart + completedCycles * DURATION;
}
```

[H-31] Rewards boosts distribute more rewards than available

Severity

Impact: Medium

Likelihood: High

Description

`BaseGauge` applies a boost over the rewards of the users, but these extra rewards received by the users are not backed by new deposits of reward tokens.

As a result, more rewards than the available tokens are distributed over the period. This will cause some users might not be able to claim their rewards.

Recommendations

The boost system should be either removed or redesign to ensure that the gauge rewards are always backed by the available tokens.

8.3. Medium Findings

[M-01] Same emission cap for all reward tokens in `BaseGauge`

Severity

Impact: Low

Likelihood: High

Description

Gauges have a maximum emission per period, which is enforced in the `notifyRewardAmount` function.

However, there can be multiple reward tokens and the same emission cap is applied to all of them. This is especially problematic when the reward tokens value if very different or they use a different decimal precision.

Recommendations

Consider implementing a separate emission cap for each reward token, or apply the cap only to the `rewardToken` if this is the intended behavior.

[M-02] Frequent `RToken` minting and burning raises accrued interest

Severity

Impact: Low

Likelihood: High

Description

`RToken` scales the user balance on `mint()` and `burn()` functions by applying the difference between the last user index and the current index.

This means that the more often `RToken` is minted or burned, the more interest is accrued due to the compounding effect. This incentivizes users to mint or burn small amounts of `RToken` frequently to maximize their interest. It is important to note that the cost of the

gas fees for minting and burning `RToken` has to be taken into account when deciding the optimal minting and burning strategy. Ultimately, this will benefit the users that hold a greater amount of `RToken`, as the profitability of minting often will be greater for them.

Recommendations

Replace the linearly increasing interest rate accrual with a function that takes into account the compounding effect over time.

[M-03] Amounts in `RToken.calculateDustAmount()` should not be normalized

Severity

Impact: Low

Likelihood: High

Description

`RToken.calculateDustAmount()` calculates the dust amount that can be withdrawn by the owner of the lending pool. The amount is calculated as the difference between the actual balance of the underlying asset held by the contract and the total supply of `RToken` multiplied by the liquidity index for normalization.

```
function calculateDustAmount() public view returns (uint256) {
    // Calculate the actual balance of the underlying asset held by this
    // contract
    uint256 contractBalance = IERC20(_assetAddress).balanceOf(address
        (this)).rayMul(ILendingPool(_lendingPool).getNormalizedIncome());

    // Calculate the total real obligations to the token holders
    uint256 currentTotalSupply = totalSupply();

    // Calculate the total real balance equivalent to the total supply
    uint256 totalRealBalance = currentTotalSupply.rayMul(ILendingPool
        (_lendingPool).getNormalizedIncome());
    // All balance, that is not tied to rToken are dust
    //(can be donated or is the rest of exponential vs linear)

    return contractBalance <= totalRealBalance ? 0 : contractBalance - totalRealBala
}
```

There is no reason to normalize the amounts, as they are already normalized, meaning the that potential difference between both amounts can be inflated, leading to a higher or lower dust amount than expected.

Recommendations

Avoid normalizing the amounts in the calculation of the dust amount.

[M-04] `healthFactorLiquidationThreshold` lacks borrowing safety margin

Severity

Impact: Medium

Likelihood: Medium

Description

`LendingPool` uses `healthFactorLiquidationThreshold` parameter to determine a safety margin between the collateralization ratio of a position and the liquidation threshold.

This parameter is initialized on deployment to `1e18`. This means that borrowers will be liquidated when the value of their collateral threshold (collateral value * liquidation threshold) is lower than the value of their debt. As users can borrow an amount equivalent to their collateral threshold, this means that a slight change in the value of the collateral or a slight increase in the value of the debt due to interest accrual will make the position immediately liquidatable.

Recommendations

Initialize `healthFactorLiquidationThreshold` to a value lower than `1e18`, so that there is a gap between the maximum borrowing power and the liquidation threshold.

It is also recommended to add a validation in the `setParameter` function to ensure that the value of `healthFactorLiquidationThreshold` is always lower than `1e18`.

[M-05] Blacklisted users can repay their debt but not close their liquidation

Severity

Impact: High

Likelihood: Low

Description

The `LendingPool.closeLiquidation()` function has a `notBlacklisted` modifier that prevents a blacklisted user from closing the liquidation of his position.

However, blacklisted users are allowed to repay their debt. If a user is blacklisted and repays his debt, he will not be able to close the liquidation of their position, so he would have paid the insurance, repaid his debt, and still be liquidated.

Recommendations

Either remove the `notBlacklisted` modifier from the `closeLiquidation()` function or add it to the `repay()` function. In the last case, it should also be considered if the insurance fee should be refunded to the user.

[M-06] `VaultProxy.depositIntoVault()` reverts when small amount of assets are deposited

Severity

Impact: Medium

Likelihood: Medium

Description

`VaultProxy.depositIntoVault()` function reverts when 0 shares are minted. When the current assets buffer is slightly above the desired buffer, a tiny amount of assets can be deposited into the vault, resulting in 0 shares being minted. This can cause a DoS on deposits, withdrawals, and borrowing.

Recommendations

Avoid depositing when the amount of assets is very small, specially if it will result in 0 shares being minted.

[M-07] `VaultProxy.depositIntoVault()` does not check for maximum deposit limit

Severity

Impact: Medium

Likelihood: Medium

Description

`VaultProxy.depositIntoVault()` function does not check for the maximum deposit limit in the vault. If this limit is reached, the deposit will fail, causing a DoS on deposits, withdrawals, and borrowing.

Recommendations

Add a check for the maximum deposit limit in the `VaultProxy.depositIntoVault` function.

[M-08] `RToken` supply cap can be exceeded

Severity

Impact: Medium

Likelihood: Medium

Description

`LendingPool.deposit()` validates that the supply cap of the `RToken` is not exceeded by the incoming deposit amount.

In the validation, the deposit amount is added to the current supply of the `RToken` and compared to the maximum supply. However, this is incorrect, as the minted amount can be greater than the deposit amount. As a result, the total supply of the `RToken` can exceed the maximum supply.

Recommendations

```
-      _validateDepositSupplyCap(depositAmount);  
-  
      // Perform the deposit through ReserveLibrary  
      uint256 mintedAmount = ReserveLibrary.deposit  
        (reserve, rateData, depositAmount, msg.sender);  
+      _validateDepositSupplyCap(mintedAmount);
```

[M-09] `RWAIndexToken` is not compatible with Uniswap's router

Severity

Impact: Low

Likelihood: High

Description

The protocol expects to create a pool for the RWAIndexToken/crvUSD pair and it contemplates Uniswap v3/v4 as the best option.

An important consideration is that the Uniswap periphery contracts are not prepared to handle tokens with fees on transfer, which is the case of `RWAIndexToken`.

Recommendations

There are different options depending on the interests of the protocol:

- Remove the swap fee from `RWAIndexToken`. Easiest to implement, but protocol will lose fees.
- Create a custom router that properly manages the swap fees. The downside is that users will need to be aware of the existence of this contract and use it instead of the official Uniswap router.
- Create a wrapped version of `RWAIndexToken`. Probably the best option for the protocol.

[M-10] `veRAACToken` cannot receive fees directly from `FeeCollector`

Severity

Impact: Low

Likelihood: High

Description

`FeeCollector.distributeFees()` calls `distributeRewards()` on the target address so that the target can pull the tokens.

One of the target addresses is `veRAACAddress`. However, the `veRAACToken` contract does not implement `distributeRewards()`, but expects instead the tokens to be transferred directly to it and the `notifyRewardAmount()` function to be called.

As a result, distribution of fees to `veRAACToken` will fail and it will be required to use instead a proxy contract that will forward the tokens to `veRAACToken` and call `notifyRewardAmount()`.

Recommendations

Either implement `distributeRewards()` in `veRAACToken` or handle the special case in `FeeCollector.distributeFees()` to call `notifyRewardAmount()`.

[M-11] `FeeCollector.distributeFees()` gives unnecessary approval to target EOA

Severity

Impact: High

Likelihood: Low

Description

`FeeCollector.distributeFees()` approves the target address for the amount of the fees distributed so that the target can pull the tokens when `distributeRewards()` is called.

However, if the contract size is zero (it is not a contract), the tokens are transferred to the target address directly. In this case, the approval is not only unnecessary but allows a malicious EOA to transfer additional tokens to itself for the amount approved.

While it is expected that the target address will be a trusted contract, it is not recommended to approve it when the target is not a contract.

Recommendations

Move the approval inside the `if (isContract)` block.

[M-12] `RAACToken` does not use `FeeCollector.collectFee()` to collect fees

Severity

Impact: Medium

Likelihood: Medium

Description

`RAACToken` collects fees on burn and transfer. These fees are transferred to the fee collector using the internal `_transfer()` and `_update()` functions.

However, the `FeeCollector` contract requires that the fees be collected using its `collectFee()` function. Otherwise, `FeeCollector` will not be aware of the fees collected and distribute them properly. Instead, the fees will require to be recovered using the `emergencyWithdraw()` function.

Recommendations

- Use the `collectFee()` function in the `FeeCollector` contract to collect fees.
- Add `to == feeCollector` to the list of conditions when the fee collection can be skipped to prevent running an infinite loop.

[M-13] Stable token and USD assumed 1:1 in protocol

Severity

Impact: High

Likelihood: Low

Description

`RAACNFT.mint()` uses the price returned by `RAACHousePrices.tokenToHousePrice()` to determine the amount of tokens that should be paid for minting the NFT.

It is expected that the payment token is crvUSD, but the `RAACHousePrices` returns the price in USD. As the conversion between crvUSD and USD is not necessarily 1:1, especially in moments of high volatility, where crvUSD can momentarily depeg from USD, this can lead to a situation where the user pays more or less than expected.

Similarly, `ERC721VaultAdapter` and `ERC721AssetAdapter` use the `RAACHousePrices.getLatestPrice()` function to calculate the value of the NFT, which is meant to be in terms of the collateral/deposit token, expected to be crvUSD. But the price returned is in USD.

Recommendations

Use Chainlink price feeds to get the price of crvUSD in USD and calculate the amount of crvUSD to value NFTs in crvUSD. This way, the conversion between crvUSD and USD is always 1:1, and the user will always pay the expected amount of crvUSD for minting the NFT.

[M-14] Users cannot prevent liquidation during the pause period

Severity

Impact: High

Likelihood: Low

Description

The `LendingPool` has a pause mechanism that can be used to prevent calls to certain functions in the pool. These functions include the `deposit()` and `repay()` functions.

This means that during the pause period, users cannot act to enhance the health of their positions to prevent liquidation.

For example, if the protocol is paused and the collateral value drops significantly, users might want to deposit more collateral, but they will not be able to do so and, when the protocol is unpaused, they might be instantly liquidated without any chance to act.

Similarly, users that have paid insurance in exchange for the ability to repay their debt during the grace period might not be able to do so if the protocol is paused, and it can be possible that the grace period expires before the protocol is unpaused, having their position liquidated.

Recommendations

Consider removing the pause mechanism for actions that will enhance the health of the positions.

Alternatively, add a grace period after the pause is lifted, during which users cannot be liquidated, but can act to enhance the health of their positions.

[M-15] VRF requests are not handled properly

Severity

Impact: Low

Likelihood: High

Description

As stated in the [Chainlink's VRF documentation](#), in order to ensure randomness, it is important to prevent the ability of users to choose from different VRF requests. And to do so it is recommended:

Use `requestId` to match randomness requests with their fulfillment in order

`BaseVRv2Consumer` stores the `requestId`, but does not use it to verify it matches the `requestId` received on fulfillment.

Do not allow re-requesting or cancellation of randomness

This is not enforced in `BaseVRFv2Consumer`, so when `allowVRFRequest` is set to `true` in `RWAVault`, and more than 3 days have passed since the last request was fulfilled, multiple requests can be made without waiting for the previous one to be fulfilled.

Recommendations

Do not allow requesting a new VRF request until the previous one has been fulfilled and check on fulfillment that the `requestId` matches `lastRequestId`.

[M-16] Inefficient NFT Redemption in `RWAVault`

Severity

Impact: Low

Likelihood: High

Description

`RWAVault` offers the possibility to redeem an NFT from the vault by burning shares. However, this system has some important limitations:

- The NFT is chosen randomly, which means that users cannot choose the amount of shares they want to burn.
- Only users with significant amounts of shares can redeem NFTs, as it is needed to cover the full value of the NFT with shares.
- Not all shares are redeemable, as some of the share's value is backed by ERC20 tokens, which cannot be redeemed.

This will affect the liquidity of the shares, as users might not be able to convert them back to the deposited assets.

Recommendations

Some solutions may pass for allowing the redemption of ERC20 tokens in exchange for shares or allowing the user to choose the NFT they want to redeem.

[M-17] `RWAVault.redeemNFT()` does not round up shares to be burned

Severity

Impact: Low

Likelihood: High

Description

In the calculation of the shares to be burned in `RWAVault.redeemNFT()`, the shares are not rounded up, meaning that the user may receive an NFT valued more than the shares they are burning.

As redemptions can only be done in the form of NFTs and these are expected to be valued in high amounts, burning one share less at a time should not suppose a great impact on the protocol. However, if the value of each share were to be inflated (there is another issue describing a first depositor inflation attack), the effect of the attack would be significantly increased by this issue.

Recommendations

Round up the shares to be burned on redemption.

[M-18] Vault cannot be changed in `LendingPool`

Severity

Impact: Medium

Likelihood: Medium

Description

When the owner of the `LendingPool` sets a new vault, the available balance in the current vault is withdrawn using a delegate call to the vault proxy's `withdrawFromVault()` function. However, this function is declared as `internal` in the vault proxy, which means that the `LendingPool` contract will not be able to call it and the transaction will revert.

Recommendations

Change the visibility of the `withdrawFromVault()` function in the vault proxy to `public`.

[M-19] Vault cannot be removed in `LendingPool`

Severity

Impact: Medium

Likelihood: Medium

Description

`LendingPool.setVault()` allows the owner to pass the zero address as a parameter to remove the current vault. However, if this is the case, the transaction will revert to the asset check.

```
// Allow address(0) to be set to remove the vault
address oldVault = address(vault);
vault = IBaseVault(newVault);

// Verify vault asset matches our reserve asset
if (vault.asset() != reserve.reserveAssetAddress && newVault != address
(0)) revert InvalidVaultAsset();
```

As the new value is set before the check, the `asset()` function will be called on the zero address, which will revert the transaction.

Recommendations

```
- if (vault.asset() != reserve.reserveAssetAddress && newVault != address
- (0)) revert InvalidVaultAsset();
+ if (newVault != address(0) && vault.asset
+ () != reserve.reserveAssetAddress) revert InvalidVaultAsset();
```

[M-20] Incorrect check for health factor in

`LendingPool.withdrawAsset()`

Severity

Impact: Medium

Likelihood: Medium

Description

`LendingPool` uses `healthFactorLiquidationThreshold` parameter to determine a safety margin between the collateralization ratio of a position and the liquidation threshold.

Users are allowed to borrow an amount of assets up to their collateral threshold (collateral value * liquidation threshold). However, in `withdrawAsset()` it is checked that their health factor is above `healthFactorLiquidationThreshold`, which is incorrect, as they will be left without any margin for liquidation and a slight change will lead to liquidation.

Consider the following example:

- `liquidationThreshold` is 80%.
- `healthFactorLiquidationThreshold` is 0.9e18 (offers a safety margin).
- Alice collateral value is 1,000.
- Alice collateral threshold is 800 ($1,000 * 80\%$).
- Alice borrows 800 (maximum allowed).
- With no changes in the collateral value, Alice will be liquidated if the debt value increases up to ~889. However, Alice was forced to open the position with a higher health factor, to prevent instant liquidation.
- Alice is now allowed to withdraw up to the liquidation threshold, which is 89 tokens. She should not be allowed to withdraw anything just after borrowing if she used her maximum collateralization ratio.

Recommendations

```
uint256 newHealthFactor = _previewHealthFactor
    (newCollateralValue, positionDebt);
-     if (newHealthFactor < parameters.healthFactorLiquidationThreshold) {
+     if (newHealthFactor < 1e18) {
        revert WithdrawalWouldLeaveUserUnderCollateralized();
    }
```

Additionally, it would be recommended renaming the `liquidationThreshold` variable, as in reality, it represents the collateralization ratio, so it can be misleading.

[M-21] Balance in vault is overestimated in lending pool rebalance

Severity

Impact: Medium

Likelihood: Medium

Description

In `VaultProxy.rebalanceLiquidity()` when the current buffer is lower than the required buffer, it is calculated if the balance in the vault can cover the shortage and the maximum amount available to withdraw, limited by the shortage amount, is withdrawn from the vault.

However, the available balance for withdrawal is calculated as the total assets in the vault, instead of the maximum amount that can be withdrawn from by the contract.

```

} else if (currentBuffer < desiredBuffer) {
    uint256 shortage = desiredBuffer - currentBuffer;
    // Check how much we can actually withdraw
    @> uint256 vaultBalance = vault.totalAssets();

    uint256 withdrawAmount = shortage > vaultBalance ? vaultBalance : shortage;
    if (withdrawAmount > 0) {
        // Withdraw what we can from the vault
        withdrawFromVault(withdrawAmount);
    }
}

```

As a result, the contract may attempt to withdraw more than it is allowed to, which will cause the transaction to revert. This will lead to deposits, withdrawals, and borrowing failed in the lending pool.

Recommendations

```

-    uint256 vaultBalance = vault.totalAssets();
+    uint256 vaultBalance = vault.maxWithdraw(address(this));

```

[M-22] Proposal cancellation delays critical change execution

Severity

Impact: Medium

Likelihood: Medium

Description

Governance proposals can be cancelled before execution by the proposer or by anyone if the proposer loses the required voting power.

This could lead to the execution of a proposal being delayed by a proposer.

Consider the following scenario:

- A user creates a proposal for a vital change in the protocol.
- Users vote massively in favor of the proposal.
- Just before the proposal is executed, the proposer cancels it.
- A new proposal has to be created, voted on scheduled, and executed again, delaying the change, which could be critical for the protocol.
- The new proposal has the same risk of being canceled by the new proposer.

The same scenario could happen unintentionally if the proposer loses the required voting power.

Recommendations

Allow the cancellation of proposals only before the voting period starts.

[M-23] Inconsistent state at `proposal.endTime` in `Governance`

Severity

Impact: High

Likelihood: Low

Description

The `Governance` contract allows submitting new votes when timestamp is equal to `proposal.endTime`.

```
function castVote
(uint256 proposalId, bool support) external override returns (uint256) {
(...)
    if (block.timestamp > proposal.endTime) {
        revert VotingEnded(proposalId, proposal.endTime, block.timestamp);
    }
}
```

However, the `state()` function only considers the proposal to be active if the timestamp is less than `proposal.endTime`.

```
function state(uint256 proposalId) public view override returns
(ProposalState) {
(...)
    if (block.timestamp < proposal.endTime) return ProposalState.Active;
}
```

This inconsistency can be problematic if votes are cast at the end of the voting period.

Imagine the following scenario:

- Current `block.timestamp` is `proposal.endTime`.
- For votes are greater than 50% and a quorum is reached.
- `execute()` is called and the proposal is queued.
- In the same block, new votes are cast, and now against is greater than 50%.
- The proposal will be executed even if the majority of votes are against it.

Recommendations

```
-    if (block.timestamp < proposal.endTime) return ProposalState.Active;
+    if (block.timestamp <= proposal.endTime) return ProposalState.Active;
```

[M-24] **Governance** does not send ether to **TimelockController**

Severity

Impact: Low

Likelihood: High

Description

`TimelockController.executeBatch()` is meant to receive ether when the operations have a value greater than 0. However, the **Governance** contract does not send any ether to it.

Given that **TimelockController** does not have a `receive()` function, in order to execute operations with value it will be required to batch the call with a previous transaction that will force sending ether to it.

Recommendations

Make `Governance.execute()` payable and forward `msg.value` to `TimelockController.executeBatch()`.

It will also be advisable adding a check to ensure that not more ether than required is sent and return the excess to the sender.

[M-25] Leftovers from distribution are locked in **RAACMinter**

Severity

Impact: Low

Likelihood: High

Description

`RAACMinter._processMintedRewards()` processes the minted rewards and distributes them to the distribution pools. The amount of rewards distributed to each pool is calculated as

```
(excessTokens / totalWeight) * poolData.weight
```

This can lead to some tokens not being distributed due to rounding errors, which will accumulate over time.

Imagine this scenario:

- We have two pools with weights of 5,000 each.
- 10,001 RAAC tokens are minted (excessTokens).
- Each pool would receive $(19_000 / 10_000) * 5_000 = 5_000$ RAAC tokens.
- 9,000 RAAC tokens are left undistributed and locked in the contract.

Recommendations

Add leftover tokens from distribution to the `excessTokens` variable.

[M-26] Distribution pool not added correctly in `RAACMinter`

Severity

Impact: Medium

Likelihood: Medium

Description

When a pool distribution pool is added in `RAACMinter` through the `manageDistributionPool()` function, `poolData.pool` is not set to the pool address.

```
if (poolData.exists) {  
    // Already exist, directly update total weight  
    totalWeight = totalWeight - poolData.weight + weight;  
} else {  
    // New pool, just add + update total weight  
    distributionPools.push(pool);  
    poolData.exists = true;  
    totalWeight += weight;  
}
```

This will cause a revert for approval to zero address in `_depositPendingReward()`.

```
function _depositPendingReward(DistributionPool memory poolData) internal {  
    if (poolData.pendingRewards > 0) {  
        @>  
        raacToken.approve(poolData.pool, poolData.pendingRewards);  
    }
```

This function is called by `tick()`, which in turn is called on deposit, withdrawal, and liquidation from the stability pool. As a result, these actions will be DoSed until the `RAACMinter` updater role removes the pool.

Recommendations

```
poolData.exists = true;  
+     poolData.pool = pool;  
    totalWeight += weight;
```

[M-27] **RAACNFT** tokens cannot be burned

Severity

Impact: Low

Likelihood: High

Description

RAACNFT has a `setAllowBurning()` function that can be called by the admin role to allow burning NFTs. While the `_update()` and `_increaseBalance()` functions have been overridden to allow burning, this is not enough, as the `transferFrom()` function reverts if the recipient is the zero address.

Recommendations

Provide a new external function that calls the `_burn()` internal function or override the `transferFrom()` function to allow burning NFTs.

[M-28] **Chainlink** beta functions used for critical data

Severity

Impact: High

Likelihood: Low

Description

The protocol uses Chainlink Functions to set the house price for RAACNFTs and to set the prime rate in the lending pool. However, Chainlink Functions are currently in beta, and the [documentation](#) states the following in that regard:

Chainlink Functions is available on mainnet only as a BETA preview to ensure that this new platform is robust and secure for developers. While in BETA, **developers must follow best practices and not use the BETA for any mission-critical application or secure any value.** Chainlink Functions is likely to evolve and improve. Breaking changes might occur while the service

is in BETA. Monitor these docs to stay updated on feature improvements along with interface and contract changes.

Recommendations

Given that the house price and prime rate are critical components of the protocol, it would be advisable adding a validation step for the data returned by Chainlink Functions before the update these values. This validation could consist of checking for a certain deviation from the previous value and requiring a manual intervention if the deviation is too high.

[M-29] `veRAACToken` maximum supply not enforced in `increase()`

Severity

Impact: Medium

Likelihood: Medium

Description

`veRAACToken` has a maximum supply of 15 million tokens. While the `lock()` function ensures that the maximum supply is not exceeded, the `increase()` function does not have a similar check. This could lead to the creation of more tokens than intended.

Recommendations

Add a check in the `increase()` function to ensure that the total supply does not exceed the maximum supply.

[M-30] `Auction.emergencyRefundERC20()` cannot be executed

Severity

Impact: Low

Likelihood: High

Description

`Auction` contracts are deployed by `AuctionFactory` and the factory address is set as the initial owner of the auction.

The `emergencyRefundERC20()` function can only be called by the owner of the auction. However, the factory contract does not provide a way of calling this function or transferring ownership of the auction to another address. This means that `emergencyRefundERC20()` can never be called.

Recommendations

Implement a way to transfer ownership of the auction to another address and/or implement a way for the factory to call `emergencyRefundERC20()` on the auction.

[M-31] Supply cap limits `StabilityPool` asset redeposits

Severity

Impact: High

Likelihood: Low

Description

The `LiquidationStrategyProxy` contract handles the liquidation of undercollateralized positions through the stability pool. After a successful liquidation, the contract attempts to deposit any remaining CRVUSD back into the lending pool to receive rTokens:

```
uint256 finalCRVUSDBalance = crvUSDToken.balanceOf(address(this));
if (finalCRVUSDBalance > 0) {
    lendingPool.deposit(finalCRVUSDBalance);
}
```

However, this deposit operation can fail if the lending pool's supply cap has been reached. When this occurs, the entire liquidation transaction will revert, preventing the stability pool from executing its core function of liquidating unhealthy positions.

Consider this sequence:

1. A position becomes undercollateralized.
2. The stability pool attempts to liquidate the position.
3. The liquidation succeeds and the stability pool receives CRVUSD.
4. The attempt to deposit CRVUSD back into the lending pool fails due to supply cap.
5. The entire liquidation transaction reverts.
6. The undercollateralized position remains in the system, accumulating more bad debt.

This creates a situation where the protocol's primary defense against bad debt (liquidations) becomes ineffective precisely when it's most needed - during periods of high utilization when the supply cap is reached.

Recommendations

Omit checking for the supply cap when the sender is the StabilityPool.

[M-32] Balance check too strict prevents valid liquidations in `StabilityPool`

Severity

Impact: Medium

Likelihood: Medium

Description

The `LiquidationStrategyProxy` implements liquidation logic for the stability pool, but contains an overly restrictive balance validation that can prevent legitimate liquidations from executing. The contract checks if the stability pool has sufficient CRVUSD balance to cover the entire position debt before proceeding with the liquidation:

```
if (rToken.balanceOf(address(this)) < scaledPositionDebt)
    revert InsufficientBalance();
```

This validation fails to account for the fact that the stability pool may have sufficient funds split between CRVUSD and rTokens that could together cover the position debt. Consider the following scenario:

1. A position has a scaled debt of 100 CRVUSD.
2. The stability pool has:
 - 50 CRVUSD in direct balance.
 - 90 rTokens (redeemable for CRVUSD).
3. The liquidation attempt fails because `initialCRVUSDBalance < scaledPositionDebt`.
4. However, the liquidation should succeed since withdrawing 50 rTokens would provide enough CRVUSD to cover the debt.

This unnecessary restriction reduces the efficiency of the liquidation mechanism by blocking valid liquidations where the stability pool has adequate combined liquidity across both CRVUSD and rToken balances. This could lead to positions remaining undercollateralized for longer than necessary, potentially increasing bad debt in the system.

Recommendations

Remove this check:

```
if (rToken.balanceOf(address(this)) < scaledPositionDebt)
    revert InsufficientBalance();
```

[M-33] Incorrect interface used for sCRVUSD Vault

Severity

Impact: Low

Likelihood: High

Description

The `VaultProxy` contract implements deposit functionality that interacts with Curve vaults through a standardized interface. Within the `depositIntoVault()` function, after depositing assets into the vault, it calls `vault.pricePerShare()` with a boolean parameter to retrieve the current share price:

```
uint256 currentSharePrice = vault.pricePerShare(false);
```

However, the protocol plans to integrate with the sCRVUSD vault, which implements a different interface where `pricePerShare()` takes no parameters:

```
def pricePerShare() -> uint256:
```

When `VaultProxy` attempts to call `pricePerShare(bool)` on the sCRVUSD vault contract, the transaction will fail because no such function exists.

Recommendations

Update the `VaultProxy` contract to implement the correct interface for sCRVUSD vault.

[M-34] Missing liquidity rebalancing in repay function

Severity

Impact: Low

Likelihood: High

Description

The `_repay()` function in the `LendingPool` contract fails to rebalance liquidity after funds are transferred from the borrower. When users repay their loans, the repaid funds are transferred to the `RToken` contract but remain idle instead of being properly allocated between the buffer and the yield-generating vault according to the protocol's liquidity buffer ratio.

```
function _repay(  
    address adapter,  
    bytes calldata data,  
    uint256 amount,  
    address onBehalfOf  
) internal {
```

Unlike the `deposit()` function which correctly calls `_rebalanceLiquidity()` after updating the reserve state, the repay function omits this step. This oversight results in inefficient capital utilization as repaid funds sit idle in the contract without generating yield, directly impacting protocol revenue.

Proof of Concept

1. User A deposits 1000 tokens into the protocol.
 - The `deposit()` function correctly rebalances liquidity, sending excess funds to the vault.
2. User B borrows 500 tokens from the protocol.
3. User B later repays the 500 tokens.
 - The `_repay()` function transfers the tokens to the RToken contract.
 - No rebalancing occurs, so all 500 tokens remain in the RToken contract.

Recommendations

To fix this issue, add a call to `_rebalanceLiquidity()` at the end of the `_repay()` function.

[M-35] Reward distribution can be grieved to zero through dust transfers

Severity

Impact: High

Likelihood: Low

Description

The `GaugeRewardsDistributor` contract manages reward distributions to various gauges in the protocol. When initializing reward data for a token, the contract checks the current token balance and adjusts the distribution amount accordingly. However, the current implementation contains a vulnerability that allows malicious users to grief the reward distribution system by front-running initialization transactions with dust transfers.

```
uint256 tokenBalance = IERC20(token).balanceOf(address(this));

// If we don't have enough tokens, use what we have
uint256 distributionAmount = amount;
if (tokenBalance < amount) {
    distributionAmount = tokenBalance;
}

// If we don't have any tokens locally, try to get them from the sender
if (distributionAmount == 0 && amount > 0) {
    IERC20(token).transferFrom(msg.sender, address(this), amount);
    distributionAmount = amount;
}
```

The core issue lies in how the contract handles cases where the current token balance is less than the intended distribution amount. Instead of requiring the full amount to be transferred, it adjusts the distribution amount to the current balance. This adjusted amount is then used to calculate the reward rate by dividing it by the distribution duration. Due to integer division, if the adjusted amount is small enough, the resulting rate will be zero, effectively nullifying the reward distribution.

Proof of Concept

1. Admin prepares to initialize rewards with 1000e18 tokens.
2. Attacker monitors mempool for `initializeRewardData()` transaction.
3. Attacker front-runs with:
 - Transfers 1 wei of reward token to the distributor.
4. Admin's transaction executes:
 - Checks balance: 1 wei.
 - Sets `distributionAmount = 1`.
 - Calculates rate: `1 / DURATION = 0`.
5. Result: Reward rate is set to 0.

Recommendations

Replace the current balance check logic with:

```

function initializeRewardData(address token, uint256 amount) external {
    uint256 tokenBalance = IERC20(token).balanceOf(address(this));

    -    // If we don't have enough tokens, use what we have
    -    uint256 distributionAmount = amount;
    -    if (tokenBalance < amount) {
    -        distributionAmount = tokenBalance;
    -    }

    -    // If we don't have any tokens locally, try to get them from the sender
    -    if (distributionAmount == 0 && amount > 0) {
    -        IERC20(token).transferFrom(msg.sender, address(this), amount);
    -        distributionAmount = amount;
    -    }

    +    uint256 distributionAmount = amount;
    +    if (tokenBalance < amount) {
    +        IERC20(token).transferFrom(
    +            msg.sender,
    +            address(this),
    +            amount - tokenBalance
    +        );
    +    }
    + }

```

Or:

```

uint256 tokenBalance = IERC20(token).balanceOf(address(this));

+    uint256 amountToTransfer = tokenBalance < amount ? amount - tokenBalance : 0;

-    // If we don't have enough tokens, use what we have
-    uint256 distributionAmount = amount;
-    if (tokenBalance < amount) {
-        distributionAmount = tokenBalance;
-    }

-    // If we don't have any tokens locally, try to get them from the sender
-    if (distributionAmount == 0 && amount > 0) {
-        IERC20(token).transferFrom(msg.sender, address(this), amount);
-        distributionAmount = amount;
-    }

+    if (amountToTransfer > 0) {
+        IERC20(token).transferFrom(msg.sender, address(this), amountToTransfer);
+    }

```

[M-36] Unbounded array allow an attacker to DoS withdrawals

Severity

Impact: High

Likelihood: Low

Description

The `BaseGauge` contract maintains a list of active users in the `_userList` array, which is used to track all users with non-zero balances. When users stake tokens through the

`stake()` function, they are added to this list via `_addToUserList()`. When they withdraw their full balance, they are removed via `_removeFromUserList()`.

The `_removeFromUserList()` function must iterate through the entire array to find and remove a user:

```
function _removeFromUserList(address user) internal {
    if (_isActiveUser[user]) {
        for (uint256 i = 0; i < _userList.length; i++) {
            if (_userList[i] == user) {
                _userList[i] = _userList[_userList.length - 1];
                _userList.pop();
                break;
            }
        }
        _isActiveUser[user] = false;
    }
}
```

This creates a vulnerability where an attacker can perform a DoS attack by:

1. Creating many addresses.
2. Staking dust amounts from each address.
3. Causing the `_userList` array to grow so large that legitimate users cannot withdraw their full balances because `_removeFromUserList()` will exceed the block gas limit.

Recommendations

To fix this issue:

- Add a minimum stake amount requirement.
- Remove the `_userList` tracking entirely or replace it with a mapping.

[M-37] Missing interface implementation in `StabilityPool`

Severity

Impact: Low

Likelihood: High

Description

The `RAACMinter` contract implements a reward distribution system where RAAC tokens are minted and distributed to various pools, including the `StabilityPool`. The distribution mechanism relies on recipient pools implementing the `IRAACMinterRewardsReceiver` interface, which defines methods for receiving token

rewards. However, the `StabilityPool` contract does not implement this required interface, causing all reward distributions to fail.

The issue occurs in the `_depositPendingReward()` function of the `RAACMinter` contract, which attempts to call the `deposit()` function on recipient pools through the `IRAACMinterRewardsReceiver` interface. Since the `StabilityPool` does not implement this interface, every distribution attempt falls into the catch block.

```
function _depositPendingReward(DistributionPool memory poolData) internal {
    if (poolData.pendingRewards > 0) {
        raacToken.approve(poolData.pool, poolData.pendingRewards);
        try IRAACMinterRewardsReceiver(poolData.pool).deposit(address
            (raacToken), poolData.pendingRewards) {
            emit RewardsDistributed(poolData.pool, poolData.pendingRewards);
            poolData.pendingRewards = 0;
        } catch {
            // Always enters this block for StabilityPool
            emit DistributionFailed(poolData.pool, poolData.pendingRewards);
        }
    }
}
```

Recommendations

Implement the `IRAACMinterRewardsReceiver` interface in the `StabilityPool` contract.

[M-38] Broken access control between minter and token contracts

Severity

Impact: Medium

Likelihood: Medium

Description

The `RAACMinter` contract is designed to manage various aspects of the RAAC token, including tax rates and fee collection. The issue stems from the `RAACToken` contract using OpenZeppelin's `Ownable` pattern for access control on functions like `setSwapTaxRate()`, `setBurnTaxRate()`, and `setFeeCollector()`. The `RAACMinter` attempts to call these functions directly but he's not the owner:

1. The minter cannot execute its intended functions.
2. Transferring token ownership to the minter would be insufficient as the minter lacks implementation of other owner-required functions like `manageWhitelist()` and `setTaxRateIncrementLimit()`.
3. The minter has no mechanism to transfer ownership back to the original owner.

```

contract RAACToken is Ownable {
    function setBurnTaxRate(uint256 rate) external onlyOwner { ... }
    function setSwapTaxRate(uint256 rate) external onlyOwner { ... }
    function setFeeCollector(address collector) external onlyOwner { ... }
}

contract RAACMinter {
    function setBurnTaxRate(uint256 _burnTaxRate) external onlyRole
    (UPDATER_ROLE) {
        raacToken.setBurnTaxRate(_burnTaxRate); // Reverts: caller is not owner
    }
}

```

Recommendations

Add an `onlyMinterOrOwner()` modifier to the RAACToken contract, and apply it to the following functions: `setSwapTaxRate`, `setBurnTaxRate`, and `setFeeCollector`.

[M-39] `distributionCycleBalance` should not rely on contract balance

Severity

Impact: Medium

Likelihood: Medium

Description

In Gauge Reward distributor, the admin role can initialize the reward to one specific gauge. If we have enough balance in the distributor, we will transfer rewards to the gauge.

The problem here is that after we start one cycle, the balance in the distributor contract will be taken as the pending allocation. If the admin transfers the remaining balance, it may cause some gauges may fail to distribute rewards.

```

function initializeRewardData(
    addressgauge,
    addresstoken,
    uint256amount
) external onlyRole(DISTRIBUTOR_ADMIN
    uint256 tokenBalance = IERC20(token).balanceOf(address(this));

    // If we don't have enough tokens, use what we have
    uint256 distributionAmount = amount;
    if (tokenBalance < amount) {
        distributionAmount = tokenBalance;
    }
    if (distributionAmount == 0 && amount > 0) {
        IERC20(token).transferFrom(msg.sender, address(this), amount);
        distributionAmount = amount;
    }
}

```


Recommendations

Add one sanity check that we have to leave enough balance to the pending allocations.
Or, track the rewards added to the cycle instead of relying on the balance in the contract.

[M-40] Possible reward rate precision loss

Severity

Impact: Medium

Likelihood: Medium

Description

In GaugeRewardDistributor contract, anyone is allowed to deposit some rewards. The problem here is when we re-calculate the reward rate, we will use the formula `(amount + leftover) / DURATION`.

Currently, the `DURATION` is `7 days` (604800). If the reward token is one low decimal token, e.g. USDC/USDT. The precision loss will be around 0.6 U. And malicious users can trigger this repeatedly. The normal users will lose their rewards.

```
function _handleRewardNotification(address token, uint256 amount) internal {
    if (timestamp >= rd.periodFinish) {
        rd.rate = amount / DURATION;
        rd.lastUpdateTime = getNextCycleStart();
        rd.periodFinish = rd.lastUpdateTime + DURATION;
    } else {
        uint256 remaining = rd.periodFinish - timestamp;
        uint256 leftover = remaining * rd.rate;
        rd.rate = (amount + leftover) / DURATION;
        rd.periodFinish = getNextCycleStart() + DURATION;
    }
}
```

Recommendations

Add one precision protection when we calculate the reward rate.

[M-41] Improper allocation re-calculation

Severity

Impact: High

Likelihood: Low

Description

Function `_checkAndAddGaugeToCycle` aims to add one new active gauge into the cycle reward allocation. If we add one new active gauge, we're allowed to distribute some rewards to the new gauge via re-allocation.

In the re-allocation, we will use the formula `distributionCycleBalance / activeCount`. The problem here is that it's possible that several gauges have already distributed rewards according to the previous allocation. This will cause we will fail to distribute rewards for the last gauge because we don't have enough rewards to distribute.

```
function _checkAndAddGaugeToCycle
(address token, address gauge) internal returns (uint256) {
    uint256 gaugeShare = pendingAllocations[token][gauge];
    if (gaugeShare == 0 && distributionCycleBalance[token] > 0) {
        bool gaugeInCycle = false;
        for (uint256 i = 0; i < activeCycleGaugeCount[token]; i++) {
            if (activeCycleGauges[token][i] == gauge) {
                gaugeInCycle = true;
                break;
            }
        }
        if
            (!gaugeInCycle && activeCycleGaugeCount[token] < activeCycleGauges[token].length) {
                activeCycleGauges[token][activeCycleGaugeCount[token]] = gauge;
                activeCycleGaugeCount[token]++;
            }
        _recalculateAllocations(token);
        gaugeShare = pendingAllocations[token][gauge];
    }
    return gaugeShare;
}

function _recalculateAllocations(address token) internal {
    uint256 activeCount = activeCycleGaugeCount[token];
    if (activeCount == 0) return;
    uint256 cycleBalance = distributionCycleBalance[token];
    uint256 sharePerGauge = cycleBalance / activeCount;
    uint256 totalAllocated = 0;
    for (uint256 i = 0; i < activeCount - 1; i++) {
        address gauge = activeCycleGauges[token][i];
        pendingAllocations[token][gauge] = sharePerGauge;
        totalAllocated += sharePerGauge;
    }

    // Last gauge gets the remainder
    address lastGauge = activeCycleGauges[token][activeCount - 1];
    pendingAllocations[token][lastGauge] = cycleBalance - totalAllocated;
}
```

Recommendations

If we allow adding of some new gauges in one cycle, the allocation should be calculated via `remaining rewards / remaining gauges`.

[M-42] Improper next Thursday calculation

Severity

Impact: Medium

Likelihood: Medium

Description

When we deploy GaugeRewardsDistributor contract, we will set the first cycle start as next Thursday's midnight. The problem here is that if today is Thursday, the result will be one historical timestamp. This breaks our expectations.

```
constructor(address _gaugeController) {
    if (_gaugeController == address(0)) revert InvalidAddress();
    gaugeController = IGaugeController(_gaugeController);

    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    _grantRole(DISTRIBUTOR_ADMIN, msg.sender);

    // Initialize first cycle start to next Thursday 0:00 to ensure it's
    // always a Thursday midnight
    firstCycleStart = _getNextThursdayMidnight(block.timestamp);
}
function _getNextThursdayMidnight(uint256 timestamp) internal pure returns
(uint256) {
    uint256 daysSinceEpoch = timestamp / 1 days;

    uint256 daysSincePreviousThursday = daysSinceEpoch % 7;

    @>    uint256 daysUntilThursday = daysSincePreviousThursday == 0 ? 0 : 7 - daysSincePreviousTh

    return (daysSinceEpoch + daysUntilThursday) * 1 days;
}
```

Recommendations

Revisit the `_getNextThursdayMidnight`'s logic.

One of the solutions:

```
function _getNextThursdayMidnight(uint256 timestamp) internal pure returns
(uint256) {
    uint256 daysSinceEpoch = timestamp / 1 days;

    uint256 daysSincePreviousThursday = daysSinceEpoch % 7;

+       if (daysSincePreviousThursday == 0 && (timestamp % 1 days) == 0) {
+       return timestamp; // Already at Thursday midnight
+       }
-
-       uint256 daysUntilThursday = daysSincePreviousThursday == 0 ? 0 : 7 - daysSincePreviousTh
+       uint256 daysUntilThursday = daysSincePreviousThursday == 0 ? 7 : 7 - daysSincePreviousTh

    return (daysSinceEpoch + daysUntilThursday) * 1 days;
}
```

[M-43] Improper user reward calculation in lending pool

Severity

Impact: Medium

Likelihood: Medium

Description

In a lending pool, users can add some reward tokens into the lending pool via `depositRewardTokens`. We will calculate the `rewardPerShare`. After that, users can claim rewards according to the `rewardPerShare`.

The problem here is that we calculate the `rewardPerShare` via `totalLiquidity`. However, when users claim their rewards, we will use `RToken`'s raw balance. This is different. When `crvUSD` token is borrowed, the reserve's `totalLiquidity` will be decreased, but the `RToken`'s raw balance will keep the same.

This will cause that users may fail to claim rewards.

```
function _updateReward(uint256 addedAmount) internal {
    if(rewardData.rewardToken == IERC20(address
        (0))) revert NoActiveRewardToken();
    uint256 totalSupply = reserve.totalLiquidity;
    if (totalSupply == 0) return;

    if(addedAmount > 0) {
        // update rewards per share.
        @> uint256 rewardPerShare = (addedAmount * 1e18) / totalSupply;
        rewardData.rewardPerShare += rewardPerShare;
        rewardData.lastUpdateTime = block.timestamp;
    }
}

function updateUserReward(address user) external override {
    require(
        msg.sender==reserve.reserveRTokenAddress,
        "OnlyRTokencanupdaterewards"
    );

    uint256 userBalance = getEligibleUserDeposits(user);
    if(userBalance > 0) {
        // balance * reward per share = pending rewards.
        uint256 pending = (userBalance *
            (rewardData.rewardPerShare - rewardData.userRewardPerShare[user])) / 1e18;
        if(pending > 0) {
            rewardData.unclaimedRewards[user] += pending;
        }
    }
    rewardData.userRewardPerShare[user] = rewardData.rewardPerShare;
}
```

Recommendations

When we calculate `rewardPerShare`, we should use `RToken`'s `totalSupply` not `totalLiquidity`.

[M-44] Proposal's result may change after the voting period

Severity

Impact: Medium

Likelihood: Medium

Description

In Governance, users can propose one proposal via `propose` function. If the proposal is voted on and succeeds, we can queue and execute this proposal.

We will take one proposal as `pass` if we match the below conditions:

1. `forVotes` > `againstVotes`.
2. `currentQuorum` >= `requiredQuorum`.

The problem here is that `quorum()` will change with the time. So we may manipulate one voting result after we finish the voting process.

```
function state(uint256 proposalId) public view override returns
(ProposalState) {
    ProposalCore storage proposal = _proposals[proposalId];
    if (proposal.startTime == 0) revert ProposalDoesNotExist(proposalId);
    if (proposal.canceled) return ProposalState.Canceled;
    if (proposal.executed) return ProposalState.Executed;
    if (block.timestamp < proposal.startTime) return ProposalState.Pending;
    if (block.timestamp < proposal.endTime) return ProposalState.Active;

    // After voting period ends, check quorum and votes
    ProposalVote storage proposalVote = _proposalVotes[proposalId];

    uint256 currentQuorum = proposalVote.forVotes + proposalVote.againstVotes;
    uint256 requiredQuorum = quorum();

    // Check if quorum is met and votes are in favor
    if
    (currentQuorum < requiredQuorum || proposalVote.forVotes <= proposalVote.againstVotes) {
        return ProposalState.Defeated;
    }
}

function quorum() public view override returns (uint256) {
    return (_veToken.totalSupply() * quorumNumerator) / QUORUM_DENOMINATOR;
}

function totalSupply() public view override(ERC20, IERC20) returns
(uint256) {
    return _lockState.getTotalVotingPowerAt(block.timestamp);
}
```

Recommendations

`quorum()` should be one fixed value for one proposal.

[M-45] Pending rewards may fail to be distributed

Severity

Impact: High

Likelihood: Low

Description

In RAACMinter, we will mint RAAC tokens with the rate `emissionRate`. The minted rewards will be distributed to multiple distribution pools. Based on the current implementation, we can process the possible distribution failure. According to the comment, `If distribution fails, poolData.pendingRewards will accrue until next distribution`.

In next tick, we will go on minting rewards and try to distribute minting rewards and pending rewards.

The problem here is that if our mint amount reaches `MAX_SUPPLY` in this tick, we fail to distribute for some reason. We will have some pending rewards, and we expect to distribute these rewards in next tick. But we will not distribute again, because `amountToMint` equals zero.

```
function tick() external nonReentrant whenNotPaused {
    // Do not error as it will be called from outside the contract
    if (raacToken.totalSupply() >= MAX_SUPPLY) return;

    uint256 currentBlock = block.number;
    uint256 blocksSinceLastUpdate = currentBlock - lastUpdateBlock;
    if (blocksSinceLastUpdate > 0) {
        //
        uint256 amountToMint = emissionRate * blocksSinceLastUpdate;

        if (raacToken.totalSupply() + amountToMint > MAX_SUPPLY) {
            amountToMint = MAX_SUPPLY - raacToken.totalSupply();
        }
        if (amountToMint > 0) {
            _mint(amountToMint);
            _processMintedRewards();
        }
    }
}
```

Recommendations

If `amountToMint` equals 0 and we have some pending rewards, try to process the pending rewards.

[M-46] May fail to liquidate borrow positions

Severity

Impact: High

Likelihood: Low

Description

In SP, owners can liquidate one unhealthy borrow position. In SP, we may have some existing crvUSD and RToken.

We will use the initial crvUSD balance in SP at first. If the initial crvUSD balance is not enough to cover this debt, we will withdraw RToken to get back some more crvUSD token. The problem here is that we have one check `rToken.balanceOf(address(this)) < scaledPositionDebt`. Actually, if we have enough crvUSD token in SP, we should liquidate this borrow position successfully, even if we don't have any RToken here. This strict limitation will block normal liquidation.

The problem here is that if we don't have enough rToken in SP, we will fail to liquidate this borrow position. The liquidation process is one key function considering the collateral price's volatile.

There are two possible scenarios:

1. There are not enough reward incentives, we don't have too many rToken deposits.
2. If current lending pool's util rate is high, we may fail to withdraw our RToken because lending pool does not have enough crvUSD to be withdrawn.

```
function liquidateBorrower(
    addresspoolAdapter,
    addressvaultAdapter,
    addressuser,
    bytescallldata
) external onlyProxy {
    uint256 scaledPositionDebt = lendingPool.getPositionScaledDebt
        (poolAdapter, user, data);
    uint256 initialCRVUSDBalance = crvUSDToken.balanceOf(address(this));
    if ( rToken.balanceOf(address
        (this)) < scaledPositionDebt) revert InsufficientBalance();
}

    uint256 rTokenAmount = initialCRVUSDBalance >= scaledPositionDebt ? 0 : scaledPositionDe

// We unwind the position
if (rTokenAmount > 0) {
    // actually, withdraw parameter should be the underlying token amount,
    // not RToken amount.
    lendingPool.withdraw(rTokenAmount);
}
@>    if (crvUSDBalance < scaledPositionDebt) revert InsufficientBalance();
```

Recommendations

Revisit the liquidation logic. We need to make sure the liquidation process always works well.

[M-47] Improper emission cap check

Severity

Impact: Low

Likelihood: High

Description

In base gauge, we have one reward period, weekly or monthly. We have one function `setMaxEmission` to set the maximum cap for one period.

The problem here is that it's possible that we notify rewards multiple times in one reward period. The check `amount > periodState.emission` will make sure that each reward notify amount cannot exceed `periodState.emission`. This will cause that one period's emission may exceed `emission`. This breaks our expectations.

```
/**
 * @notice Sets max emission for the period
 * @param emission New max emission amount
 */
function setMaxEmission(uint256 emission) external onlyRole
(CONTROLLER_ROLE) {
    periodState.emission = emission;
    emit EmissionUpdated(emission);
}
function notifyRewardAmount(
    address token,
    uint256 amount
) external override onlyRole(CONTROLLER_ROLE) {
    if (amount > periodState.emission) {
        revert RewardCapExceeded();
    }
}
```

Recommendations

In the base gauge, we will record one period's distribution amount via `periodState.distributed`. We should make sure `periodState.distributed` cannot exceed `emission`.

[M-48] Users may change the fee collector's distribution ratio

Severity

Impact: Medium

Likelihood: Medium

Description

In FeeCollector, users can claim NFT's underlying via function `claimNFTUnderlying`. We expect to withdraw some token from `target` via function `withdrawUnderlying`. And we

will update the withdrawn fee for different receipt according to the related `feeType`.

If we claim an underlying token from one fixed `target`, the `feeType` should be one fixed one. Different fee receivers will receive fees according to this specific fee type's distribution ratio. The problem here is that users can manipulate the `feeType`. For example, one user holds some veRAAC tokens, and he prefers to choose the `feeType` which will distribute more for veRAAC holders.

```
function claimNFTUnderlying(
    addresstoken,
    addresstarget,
    uint256amount,
    bytes32feeType
) external nonReentrant whenNotPaused returns (bool)
    if (!isTokenSupported[token]) revert TokenNotSupported();
    if (amount == 0) revert InvalidFeeAmount();

    // Call the target contract to withdraw underlying NFTs
    (bool success,) = target.call(
        abi.encodeWithSelector(bytes4(keccak256("withdrawUnderlying")), token, address(this), amount)
    );

    if (!success) revert ClaimCollectorUnderlyingFailed();

    // Update the collected fees to reflect the withdrawal
    _updateCollectedFees(token, feeType, amount);
    emit CollectorUnderlyingClaimed(token, target, amount);
    return true;
}
```

Recommendations

Add one sanity check for the input `feeType`.

[M-49] Missing distributor rate consideration impacts reward accrual

Severity

Impact: Medium

Likelihood: Medium

Description

The function `_calculatePendingRewards` (lines 767–773) correctly fetches an override rate from `IGaugeRewardsDistributor` when present:

```

File: BaseGauge.sol
754:     function _calculatePendingRewards(
755:         address token,
756:         address user
757:     ) internal view returns (uint256) {
758:         RewardData storage rd = rewardData[token];
759:
760:         if (_balances[user] == 0) {
761:             return 0;
762:         }
763:
764:         uint256 storedRewards = rewards[token][user];
765:         uint256 currentRate = rd.rate;
766:
767: @>         if (rd.distributor != address(0) && rd.distributor != controller) {
768:             try
769: @>             IGaugeRewardsDistributor(rd.distributor).getRewardRate
              (token)
770:             returns (uint256 rate) {
771:                 if (rate > 0) {
772:                     currentRate = rate;
773:                 }
774:             } catch {}
775:         }

```

However, the on-chain accrual in `_updateUserReward` (lines 939–952) applies only the passed-in `rewardPerToken` delta—without ever querying the distributor override rate:

```

929:     function _updateUserReward(
930:         address token,
931:         address account,
932:         uint256 rewardPerToken,
933:         uint256 decimalAdjustment
934:     ) internal {
935:         UserRewardData storage userData = userRewardData[token][account];
936:         uint256 userBalance = _balances[account];
937:
938:         if (userBalance > 0) {
939:             uint256 userIntegralDelta = rewardPerToken - userData.integral;
940:             if (userIntegralDelta > 0) {
941:                 uint256 workingBalance = userBalance;
942:                 if (boostAmplificationEnabled) {
943:                     uint256 boost = _applyBoost(account, workingBalance);
944:                     workingBalance =
945:                         (workingBalance * boost) / WEIGHT_PRECISION;
946:                 }
947:                 uint256 newReward = (workingBalance * userIntegralDelta) /
948:                     (1e18 * decimalAdjustment);
949:
950:                 userData.integral = rewardPerToken;
951:                 userData.lastUpdate = block.timestamp;
952:                 rewards[token][account] += newReward;
953:
954:                 emit UserRewardDataUpdated
955:                     (account, token, rewardPerToken, block.timestamp);
956:             }
957:         }

```

As a result, any rate changes pushed through the distributor are only visible via off-chain calls to `_calculatePendingRewards`, but are entirely ignored when users actually claim or accrue rewards. This divergence can mislead UIs and integrators, and undermine the protocol’s reward-control mechanisms.

Recommendations

In the function that updates user's integral `_updateUserReward()`, replicate the distributor lookup (code line 769) so that both view and state paths share the same rate source.

[M-50] Missing reward checkpoint in

`syncRewardData()`

Severity

Impact: Medium

Likelihood: Medium

Description

When `BaseGauge.syncRewardData(address token)` is invoked, it updates the on-chain `rate` and `periodFinish` without first accruing and storing the rewards accumulated `rewardPerTokenStored` at the *old* rate via `_updateRewardPerToken()` function. As a result, the next reward-calculation (via `_updateReward()`) retroactively applies the *new* rate over the entire elapsed interval, mis-pricing the rewards.

```

File: BaseGauge.sol
334:     function syncRewardData(address token) external {
335:         _ensureTokenDistributorSet(token);
336:
337:         IGaugeRewardsDistributor distributor = IGaugeRewardsDistributor(
338:             rewardData[token].distributor
339:         );
340:
341:         // Get current data from distributor
342:         uint256 newRate = distributor.getRewardRate(token);
343:         uint256 newPeriodFinish = distributor.getPeriodFinish(token);
344:
345:         // Update reward data
346:@> RewardData storage rd = rewardData[token];
347:@> rd.rate = newRate;
348:@> rd.periodFinish = newPeriodFinish;
349:
350:         // Only update lastUpdateTime if it hasn't been set yet
351:         if (rd.lastUpdateTime == 0) {
352:             rd.lastUpdateTime = block.timestamp;
353:         }
354:     }
...
893:     function _updateRewardPerToken(
894:         RewardData storage rd,
895:         address token
896:     ) internal returns (uint256, uint256) {
897:         if (totalSupply == 0) {
898:             return (rd.rewardPerTokenStored, getDecimalAdjustment(token));
899:         }
900:
901:         uint256 currentTime = block.timestamp;
902:         uint256 decimalAdjustment = getDecimalAdjustment(token);
903:
904:         if (currentTime > rd.lastUpdateTime) {
905:             uint256 duration = currentTime < rd.periodFinish
906:                 ? currentTime - rd.lastUpdateTime
907:                 : rd.periodFinish - rd.lastUpdateTime
908:                 ? rd.periodFinish - rd.lastUpdateTime
909:                 : 0;
910:
911:             if (duration > 0) {
912:                 // Following Curve's integral calculation
913:@> rd.rewardPerTokenStored +=
914:                 (duration * rd.rate * 1e18) / totalSupply;
915:             }
916:@> rd.lastUpdateTime = currentTime;
917:         }
918:
919:         return (rd.rewardPerTokenStored, decimalAdjustment);
920:     }

```

Consider the following scenario:

1. Initial state:

- `rewardData[token].rate = 1e18` (1 token/sec).
- `rewardData[token].lastUpdateTime = T0`.

2. Time passes: until `T1 = T0 + 1 hour`.

3. Distributor rate changes from `1e18` → `2e18` tokens/sec.

4. Anyone calls `syncRewardData(token)` at `T1`:

- Lines 347–348 overwrite `rewardData[token].rate` and `rewardData[token].periodFinish`.
- **No checkpoint:** `rewardData[token].rewardPerTokenStored` remain at `T0`.

5. **Later**, anyone invokes `_updateReward()`:

- `_updateRewardPerToken()` computes `delta = now - T0` (≈ 1 hour)
- Applies **new** rate `2e18` over that full hour to calculate `rewardData[token].rewardPerTokenStored` (code line BaseGauge#L913), instead of the initial `1e18` rate.
- User extracts double the intended rewards.

Recommendations

Checkpoint rewards before updating the rate. Call `_updateRewardPerToken(token)` to accrue all pending rewards under the old rate and advance `lastUpdateTime`.

```
function syncRewardData(address token) external {  
    _ensureTokenDistributorSet(token);  
    + _updateRewardPerToken(token);  
}
```

[M-51] Design of minting discounts in **RWAVault** is flawed

Severity

Impact: Medium

Likelihood: Medium

Description

`RWAVault.depositAsset()` offers a discount on minting fees to the `msg.sender` if they have NFTs locked in the `LLamaTemple` contract.

Given that the deposit can be made on behalf of another user, anyone can create a proxy contract that will provide discounted minting to other users, extracting fees from the protocol.

Consider the following scenario:

- Someone creates a contract that deposits some NFTs in the `LLamaTemple` contract. The deposited NFTs are entitled to a 50% discount on minting fees.
- The contract exposes a function that allows users to deposit assets in `RWAVault` on their behalf with a discount of 25% on minting fees, so users that do not have NFTs locked in `LLamaTemple` will prefer minting through this contract.
- As a result, for each minting, the contract profits 25% of the minting fee, the user pays 25% less, and the protocol loses 50% of the minting fee.

Even if `depositAsset()` was to remove the ability to deposit on behalf of another user, given that the index token is transferrable, the external contract could just mint the token to themselves and transfer it to the user, so the same issue would still apply.

As a result, there is no incentive to lock NFTs in the `LLamaTemple` contract, as users can always mint through a proxy contract that will provide some discount.

Recommendations

Either remove the discounts mechanism or apply the discount only when `msg.sender == receiver` and add some timelock for transfers when tokens are minted with a discount.

[M-52] Missing reward synchronization

Severity

Impact: Medium

Likelihood: Medium

Description

The `RAACMinter` contract implements a reward distribution system where RAAC tokens are minted and distributed to various pools based on their weights and emission rates. The contract uses a `tick()` function to process and distribute accumulated rewards at specific intervals. However, several functions modify the reward distribution parameters without first processing pending rewards, leading to reward theft and inflation.

The core issue lies in the lack of reward synchronization before state changes. The contract maintains a `lastUpdateBlock` variable to track when rewards were last processed but fails to call `tick()` before modifying critical parameters in functions that affect reward calculation and distribution.

- `updateEmissionRate()` changes the rate without processing pending rewards, causing inflation of old rewards.
- `addDistributionPool()`, `removeDistributionPool()`, and `manageDistributionPool()` modify pool weights without distributing accumulated rewards first.
- New pools can claim rewards that were earned before their addition.

Proof of Concept

Scenario 1

1. Initial state: `emissionRate = 100`, `lastUpdateBlock = 100`.
2. At block 200, admin calls `updateEmissionRate(200)`:

```
function updateEmissionRate(uint256 _newRate) external onlyRole
(UPDATER_ROLE) {
    if
        (_newRate == 0 || _newRate > MAX_BENCHMARK_RATE) revert InvalidBenchmarkRate();
    uint256 oldRate = emissionRate;
    emissionRate = _newRate;
    emit EmissionRateUpdated(oldRate, _newRate);
}
```

4. The 100 blocks of pending rewards are now calculated at a rate 200 instead of 100.
5. Result: 10,000 extra tokens minted than should have been (100 blocks × 100 rate difference).

Scenario 2

1. Pool A and B each have 50% weight at block 100.
2. At block 200, without calling `tick()`, admin adds Pool C with 50% weight:

```
function addDistributionPool
(address pool, uint256 weight) external onlyRole(UPDATER_ROLE) {
    if (pool == address(0)) revert ZeroAddress();
    if (poolInfo[pool].exists) revert PoolAlreadyExists();

    poolInfo[pool] = DistributionPool
        ({weight: weight, exists: true, pool: pool, pendingRewards: 0});

    distributionPools.push(pool);
    totalWeight += weight;
}
```

4. Pools C now receives 1/3 of the rewards from blocks 100-200.
5. Result: Pool C steals rewards that should have gone to A and B.

Recommendations

Add reward synchronization to all state-changing functions (`updateEmissionRate`, `addDistributionPool`, `removeDistributionPool`, `manageDistributionPool`).

[M-53] `BaseGauge.syncRewardData()` can provide incorrect reward rates

Severity

Impact: Medium

Likelihood: Medium

Description

`BaseGauge.syncRewardData()` syncs the reward data from the distributor associated with the token.

However, it is not taken into account that there can be multiple distributors for the same token. So the new rate will only reflect the data of one of the distributors.

Additionally, a distributor can also distribute rewards to multiple gauges, so its rate should be divided by the number of gauges that it distributes to.

Recommendations

Remove the `syncRewardData()` function and rely on `notifyRewardAmount()` to update the distribution rates.

[M-54] Minimum reward rate of 1 wei/second overestimates available rewards

Severity

Impact: Medium

Likelihood: Medium

Description

When `BaseGauge.notifyRewardAmount()` is called for a new period and the resulting rate is 0, this is set to 1.

```
rd.rate = newRate > 0 ? newRate : 1; // Minimum rate of 1 wei/second
```

This will cause rewards over the available tokens to be distributed over the period. This will cause some users might not be able to claim their rewards.

Recommendations

Revert the transaction if the new rate is 0.

[M-55] Zero cost rounding in auction lets users acquire tokens for free

Severity

Impact: Medium

Likelihood: Medium

Description

The auction contract's cost calculation is vulnerable to integer division rounding when dealing with small amounts. The current implementation:

```
File: Auction.sol
87:     function buy(uint256 amount) external whenActive nonReentrant {
88:         if (amount == 0) revert InvalidAmount();
89:         if (amount > state.totalRemaining) revert InsufficientZenoRemaining
        ();
90:
91:         uint256 price = getPrice();
92:         uint256 decimals = IERC20Metadata(address(token)).decimals();
93: @>         uint256 cost = price * amount / (10 ** decimals);
```

When a user attempts to buy a very small amount of ZENO tokens (e.g., 1 wei), the multiplication `price * amount` results in a number smaller than `10^decimals` (for USDC, 10^6). This causes the division to round down to zero, allowing users to acquire tokens without paying any USDC. Test:

```
// File: Integration.test.js
it("0xbepresentMinimumAmountAuction", async function () {
    // Increase time to auctionStartTime
    // Increase EVM time by 1 hour (3600 seconds)
    await ethers.provider.send("evm_increaseTime", [3600*1.5]);
    // Mine a single block
    await ethers.provider.send("evm_mine", []);
    //
    // 1. Attacker buy 1 zeno token but the cost is 0
    let amountToBuy = ethers.parseUnits("1", 0);
    const balanceBeforeBuy = await usdc.balanceOf(addr1.address);
    await expect(auction1.connect(addr1).buy(amountToBuy)).to.emit(
        auction1,
        "ZENOPurchased"
    );
    // check usdc balance after buy. It is the same as before, because the cost is
    // 0
    const balanceAfterBuy = await usdc.balanceOf(addr1.address);
    expect(balanceAfterBuy).to.be.equal(balanceBeforeBuy);
    //
    // 2. User has 1 ZENO token with zero usdc cost
    const userBalance = await zenol.balanceOf(addr1.address);
    console.log(`User zeno Balance: ${userBalance}`);
    expect(await zenol.balanceOf(addr1.address)).to.equal(amountToBuy);
});
```

Recommendations

Add a minimum cost check:

```
require(cost > 0, "Cost too small");
```

[M-56] Boost calculation in `BaseGauge` varies with staking token

Severity

Impact: Low

Likelihood: High

Description

`BaseGauge` applies a boost to the account's balance based on the amount of veToken hold.

```
// If veBalance >= balance, give full boost
if (veBalance >= balance) {
    workingBalance = balance; // Full balance (100% boost)
} else {
    // Partial boost based on veToken ratio
    uint256 boostPortion = (balance * veBalance * 6000) / (balance * 10000);
    workingBalance += boostPortion;
}
```

However, instead of using some configuration ratio, the boost is applied in a 1:1 relation between the balance of the deposited staking token and the veToken balance. Depending on what token is used as the staking token, its value can be very different from the veToken value, or they can have a different number of decimals, so applying a 1:1 can result in the boosts having very different effects depending on the token used.

Recommendations

Use a configuration ratio for boost calculations, which can be customized depending on the staking token used.

[M-57] `BaseGauge.pendingRewards()` returns incorrect pending rewards

Severity

Impact: Low

Likelihood: High

Description

In `BaseGauge.pendingRewards()` calculates the pending rewards based on their working balance, however, `_updateUserReward()` uses the regular balance when calculating the rewards for the user.

This provokes that the value returned by `pendingRewards()` will not be equal to the real pending rewards for the user.

Recommendations

Adjust the `pendingRewards()` function to use the same calculation as `_updateUserReward()`.

[M-58] Any one can prolong the reward distribution

Severity

Impact: Medium

Likelihood: Medium

Description

The `notifyRewardAmount` function is designed so that if it's called before the current `rewardDuration` ends, it adds the remaining rewards to the new amount and recalculates the distribution rate and `periodFinish`.

This introduces an issue where a malicious user could repeatedly call the function with just 1 wei, effectively extending the reward distribution period indefinitely.

Consider this:

1. A legitimate reward distribution is active.
2. Before `periodFinish`, an attacker calls `notifyRewardAmount()` with 1 wei:

```
if (block.timestamp < rData.periodFinish) {  
    uint256 remaining = rData.periodFinish - block.timestamp;  
    remaining = (remaining * rData.rewardRate) / PRECISION;  
    reward = reward + remaining; // Adds 1 wei to remaining rewards  
}  
rData.periodFinish = block.timestamp + REWARD_DURATION;
```

3. This causes:
 - Remaining rewards are added to the 1 wei deposit.
 - A new `periodFinish` is set.
 - Distribution rate is recalculated for the new period.
4. The attacker can repeat this process indefinitely with minimal cost.

Recommendations

Require minimum reward amount or add a whitelist for the `notifyRewardAmount` function.

8.4. Low Findings

[L-01] `StabilityPool` ignores `crvUSD` for liquidation debt coverage

`LiquidationStrategyProxy.liquidateBorrower()` takes into account the `crvUSD` balance in the contract when calculating the amount of `rToken` needed to be withdrawn from the lending pool to cover the debt. However, the `crvUSD` balance is not taken into account when checking if the total amount of funds is enough to cover the debt.

```
uint256 initialCRVUSDBalance = crvUSDToken.balanceOf(address(this));

@> if ( rToken.balanceOf(address
    (this)) < scaledPositionDebt) revert InsufficientBalance();

    // We need to get the amount of rToken that is needed to cover the debt,
    // or 0 if the debt is covered

    uint256 rTokenAmount = initialCRVUSDBalance >= scaledPositionDebt ? 0 : scaledPo
```

Consider the following scenario:

- `scaledPositionDebt` = 100.
- `rToken` balance = 99.
- `crvUSD` balance = 1.
- There are enough funds to cover the debt, but the transaction will revert.

It is recommended to consider the `crvUSD` balance when checking if the total amount of funds is enough to cover the debt.

[L-02] `RAACMinter` distribution pool updates ignore pending distributions

When a distribution pool is added in `RAACMinter` by the updater role, if `tick()` is not called in the same block, the new distribution pool will obtain emissions corresponding to blocks previous from its addition.

The opposite can happen on removal, where the pool being removed will not receive emissions from the last update until the block where it was removed.

It is recommended to make the `tick()` function public and call it on every update of the distribution pools.

[L-03] `vaultRewards.totalDeposits` ignores share price growth

Whenever tokens are withdrawn from the vault in the `LendingPool` contract, `vaultRewards.totalDeposits` is updated by subtracting the withdrawn amount from the total deposits. However, this does not take into account the appreciation of the shares, which might cause a greater amount of tokens to be present in the vault.

Consider the following scenario:

- 1,000 tokens are deposited in the vault, being the share price 1 (`totalDeposits = 1,000`).
- Share price increases to 1.1, so now the lending pool has 1,100 tokens in the vault.
- 1,000 tokens are withdrawn and `totalDeposits` is updated to 0 (`totalDeposits -= 1,000`), while there are still 100 tokens in the vault.

Take into account the share price when updating the total deposits in the vault.

[L-04] `StabilityPool.getTotalDeposits()` includes donations

`StabilityPool.getTotalDeposits()` returns the contract's raw balance of `RToken`. This is incorrect, as this value will include not accounted deposits, like donations or amounts sent by mistake.

It is recommended to return instead `deToken.totalSupply()`, as this value represents the total deposits in the pool.

[L-05] No check for active positions when unregistering an adapter in `LendingPool`

Unregistering an adapter in `LendingPool` when it has active positions can lead to deposited funds being locked, as users will not be able to repay their debts and withdraw their assets.

It is recommended to check if the adapter has active positions before unregistering it.

[L-06] `RToken` and `DebtToken` returns incorrect value for first mint

`RToken` and `DebtToken` mint functions return as a first parameter a boolean value that indicates if it was the first mint by the user.

This value will be true when the user's register index is equal to the current liquidity index or usage index in the lending pool. This is incorrect, as it only indicates if the indexes are in sync, but not if it is the first mint by the user. For example, if multiple tokens are minted in the same transaction, all of them might return true, which is incorrect.

This value is currently being ignored in the codebase, but if it is used after changes are made, it could lead to incorrect behavior. It is recommended to either remove this value from the return parameters or properly implement the logic to check if it is the first mint by the user.

[L-07] `RWAVault` does not check if 0 shares are minted

When an NFT is deposited into `RWAVault`, the contract calls `ERC721VaultAdapter.deposit()` to handle the deposit into the adapter. This function returns the price oracle price for the NFT, which is then used to calculate the amount of index tokens to mint.

If for any reason, the tokenId was not registered in the oracle, it will return 0, which means that the user will deposit the NFT and not receive any index tokens in return.

It is recommended to add a check to ensure that at least one index token is minted when depositing assets in `RWAVault`.

[L-08] `unregister*Adapter()` should not check global adapter support

The `unregisterRedeemableERC721Adapter()` and `unregisterDepositAdapter()` have the `onlySupportedAdapter` modifier, meaning it is required that the adapter to be unregistered is supported. This should not be the case, as the owner should be able to unregister a redeemable or depositable adapter if it has been already unregistered from the supported adapters. If that is the case, the owner will be forced to re-register the adapter, unregister it from redeemable and/or depositable adapters, and then unregister it from the supported adapters again.

Consider removing the `onlySupportedAdapter` modifier from the `unregisterRedeemableERC721Adapter()` and `unregisterDepositAdapter()` functions to check if they are supported as redeemable or depositable is enough.

[L-09] `DebtToken.mint()` emits `Transfer` event twice

`DebtToken.mint()` emits the `Transfer` event twice, as `_mint()` executes `_update()` which also emits the `Transfer` event.

```
_mint(onBehalfOf, amountToMint.toUint128());  
  
    emit Transfer(address(0), onBehalfOf, amountToMint);
```

This might cause the same amount to be registered twice in off-chain systems that rely on the `Transfer` event to track changes in token balances.

It is recommended to remove the second `emit Transfer` statement.

[L-10] `RToken.rescueToken()` cannot be called from lending pool

The `rescueToken()` function in the `RToken` contract is protected by the `onlyLendingPool`. However, the `LendingPool` contract does not implement a call to this function. As a result, if it is required to rescue tokens from the `RToken` contract, the owner will need to momentarily change the lending pool address to another contract from which the function can be called.

It is recommended to replace the `onlyLendingPool` modifier with `onlyOwner` in the `rescueToken()` function or implement a call to this function in the `LendingPool` contract.

[L-11] `DebtToken.getUsageIndex()` returns always `WadRayMath.RAY`

In `DebtToken` the internal `_usageIndex` variable is initialized to `WadRayMath.RAY`. This value is never updated, as the usage index is tracked by the lending pool, so is not required in the `DebtToken` contract.

However, its value is returned by the `getUsageIndex()` function, so this function will always return `WadRayMath.RAY`. While the function is not used in the protocol, it can be misleading for users querying the contract.

It is recommended to remove this function and all internal references to `_usageIndex` in the `DebtToken` contract.

[L-12] `RToken.getLiquidityIndex()` returns always `WadRayMath.RAY`

In `RToken` the internal `_liquidityIndex` variable is initialized to `WadRayMath.RAY`. This value is never updated, as the liquidity index is tracked by the lending pool, so is not required in the `RToken` contract.

However, its value is returned by the `getLiquidityIndex()` function, so this function will always return `WadRayMath.RAY`. While the function is not used in the protocol, it can be misleading for users querying the contract.

It is recommended to remove this function and all internal references to `_liquidityIndex` in the `RToken` contract.

[L-13] `RAACToken` skips burn tax if fee collector is not set

`RAACToken` contract applies a swap tax rate and a burn tax rate on the transfer of tokens. These are not applied under certain conditions.

```
if (baseTax == 0 || from == address(0) || to == address(0) || whitelistAddress[from] || whitelistAddress[to] || feeCollector == address(0)) {
```

In case the fee collector is not set, the application of burn tax is unnecessarily skipped.

It is recommended to only skip the application of the swap tax rate if the fee collector is not set.

```
+     uint256 baseTax = burnTaxRate + feeCollector == address(0) ? 0 : swapTaxRate;
+     // Skip tax for whitelisted addresses or when fee collector disabled
-     if (baseTax == 0 || from == address(0) || to == address(0) || whitelistAddress[from] || whitelistAddress[to] || feeCollector == address(0)) {
+     if (baseTax == 0 || from == address(0) || to == address(0) || whitelistAddress[from] || whitelistAddress[to]) {
```

[L-14] `RAACNFT.mint()` assumes payment token has 18 decimals

`RAACNFT.mint()` uses the price returned by `RAACHousePrices.tokenToHousePrice()` to determine the amount of tokens that should be paid for minting the NFT.

The price returned by `RAACHousePrices` is normalized to 18 decimals, so if the payment token has a different number of decimals, the amount paid will differ in orders of magnitude from the expected amount.

It is recommended to either add a check on deployment to ensure that the payment token has 18 decimals or adjust the amount paid by the decimals of the payment token.

[L-15] Wrong and misleading NatSpec comments

Some NatSpec comments in the codebase are incorrect or misleading. This can lead to confusion for developers and users interacting with the contract, as NatSpec comments are usually used in tooling applications like network explorers or documentation generation.

File: RAACNFT.sol

```
/**
 * @notice Mint a new NFT
 * @param _tokenId The ID of the token to mint
 *
 * @param _amount The amount of tokens to mint 🚩 it is max amount of payment tokens pa
 */
function mint(
    uint256 _tokenId,
    uint256 _amount
) public override nonReentrant notBlacklisted(msg.sender
```

File: NFTRoyaltyFeeCollector.sol

```
/**
 *
 * @notice Collect the internal fee 🚩 it is not
 * @param _token The token to send to the fee collector (address
 * (0) = ETH is converted to WETH)
 * @param recipient To whom the tokens will be sent to
 */
function emergencyWithdraw
    (address _token, address recipient) external onlyOwner nonReentrant {
```

[L-16] No check for paused fee collector before calling `collectFee()`

`RAACNFT`, `RWAIndexToken`, `RWAVault`, and `LendingPool` contracts have calls to `FeeCollector.collectFee()` so that this contract can collect the corresponding fees.

Before the fee collector is called, it is checked if it is set and if the fee is greater than 0, however, it is not checked if the contract is paused, which will cause the transaction to revert.

In order to prevent a DoS on the different operations involved, it is recommended to check if the fee collector is paused before calling the `collectFee()` function and, if so,

skip the fee collection.

[L-17] Lack of input validation in configuration values

Some configuration values lack proper sanity checks to prevent erroneous values from being set.

- `GaugeController.setTypeWeight()` does not check if the sum of the weights of both gauge types is lower or equal than `MAX_WEIGHT`.
- `LendingPool.setParameter()` does not check if the new lending supply cap is lower than the borrowing cap.
- `RAACNFT.setTradeFee()` does not check if the new fee is greater than 100_00.
- `RWAIndexToken.setFees()` does not check if the new fee is greater than `FEE_DENOM`.

[L-18] Withdraw timelock not cleared when duration is set to 0

In `LendingPool`, when the withdrawal timelock duration is set to 0, `_executeWithdrawTimelock()` returns early without clearing the user's timelock.

Consider the following scenario:

- Alice requests a withdrawal when the timelock duration is 1 day.
- After 1 day, Alice withdraws her funds. At this point, the timelock duration has been changed to 0, so her withdrawal timelock is not cleared.
- Timelock duration is changed again, this time to 1 week.
- Alice can withdraw her funds instantly without having to wait for a week.

It is recommended to clear the timelock also when the duration is set to 0.

[L-19] Inconsistent behavior between

`claimRewards()` and `claimRewardToken()`

`BaseGauge` provides two functions for claiming rewards. `claimRewards()` claims rewards for all tokens, while `claimRewardToken()` claims rewards for a single token. In both cases, the rewards are first updated, but there are discrepancies in the claim process.

- `claimRewards()` reverts if the distributor for a token is not set, while `claimRewardToken()` does not revert.
- `claimRewardToken()` reverts if the claimable amount is lower than the gauge balance, while `claimRewards()` does not revert.
- `claimRewardToken()` misses updating `userClaimedRewards`.

Consider using the internal function `_claimReward()` in `claimRewardToken()` to ensure consistent behavior and to correctly update the claimed rewards by user.

[L-20] Cancelling proposals in `Governance` does not cancel them in `TimelockController`

`Governance` proposals can be cancelled. However, when they are scheduled in the `TimelockController`, they are not cancelled there.

It is recommended to cancel operations in the `TimelockController` as well so that the state of the operation is consistent across both contracts.

[L-21] `RAACReleaseOrchestrator.emergencyRevoke()` does not withdraw tokens

`RAACReleaseOrchestrator.emergencyRevoke()` can be called by the emergency role to revoke the vesting schedule of a beneficiary.

If there are unreleased tokens, these tokens are meant to be withdrawn from the contract. However, the tokens are transferred to the contract itself, which means that they remain in the contract.

```
if (unreleasedAmount > 0) {
    raacToken.transfer(address(this), unreleasedAmount);
    emit EmergencyWithdraw(beneficiary, unreleasedAmount);
}
```

In order to recover the tokens, it will be required that the admin role calls `updateCategoryAllocation()` and reallocates them.

[L-22] Inconsistent `approve()` behavior in `IVaultAssetAdapter` implementations

`ERC20VaultAdapter` has a function `approve()` that is meant to be called through `delegatecall`. This function receives a `_token` parameter, but the immutable variable `token` is used instead.

```
function approve(
    address_token,
    addressspender,
    bytescalldatadata
) external returns (bool)
    require(address
        (this) != _self, "Function must be called through delegatecall");
    return IERC20(token).approve(spender, abi.decode(data, (uint256)));
}
```

It is also important to note that in `ERC721VaultAdapter`, which uses the same interface, the `_token` parameter is used instead of the immutable variable `token`.

Currently this function is only called from `LiquidationStrategyProxy` with the vault token passed as `_token`, so it is not a problem. However, it is recommended to update the function to prevent unexpected behavior from other contracts using this function or future changes.

If the intention is that the immutable variable `token` should always be used, remove the `_token` parameter. Otherwise, use the `_token` parameter for the approval.

[L-23] `StabilityPool` does not use upgradeable version of `ReentrancyGuard`

Upgradeable contracts should inherit from the upgradeable versions of the OpenZeppelin contracts and their respective `__{ContractName}__init` function that should be called on initialization.

However, the `StabilityPool` contract does not use the upgradeable version of `ReentrancyGuard`. While the outcome of this missed initialization is just a higher gas cost for the first execution of the `nonReentrant` modifier, it is recommended to inherit from `ReentrancyGuardUpgradeable` and call the initialization function, as it could prevent future issues if the contract is modified.

[L-24] Tokens with more than 18 decimals cause underflow in normalization

In different parts of the code, the amount of tokens is normalized to 18 decimals. However, this process will revert with an underflow error if the token has more than 18 decimals.

File: ERC20VaultAdapter.sol

```
uint8 decimals = IERC20Metadata(address(token)).decimals();  
uint256 normalized = amount * (10 ** (18 - decimals));
```

File: BaseGauge.sol

```
return 10 ** (18 - tokenDecimals);
```

File: ERC20AssetAdapter.sol

```
uint8 decimals = IERC20Metadata(address(token)).decimals();  
uint256 normalized = amount * (10 ** (18 - decimals));  
(...)  
uint8 decimals = IERC20Metadata(address(token)).decimals();  
uint256 normalized = amount * (10 ** (18 - decimals));
```

It is recommended adding a check to ensure that tokens added to the system do not have more than 18 decimals.

[L-25] `RAACNFT.tokenURI()` does not revert for invalid `_tokenId`

The specification of [ERC-721](#) states the following regarding the `tokenURI` function

Throws if `_tokenId` is not a valid NFT.

However, the `tokenURI()` function in `RAACNFT` does not check if `_tokenId` is a valid NFT, and therefore it is possible to call the function with an invalid `_tokenId` and get a response.

Consider adding a check to ensure that `_tokenId` is a valid NFT before returning the token URI.

[L-26] Broken emergency unlock in `veRAACToken`

The `veRAACToken` contract has a system to schedule emergency actions under a time lock. Scheduling the `EMERGENCY_UNLOCK_ACTION` action allows the owner to call `executeEmergencyUnlock()` after 3 days. This function will set `emergencyUnlockEnabled` to true and emit the `EmergencyUnlockEnabled` event. However, the contract does not provide a way for users to unlock their tokens.

It is recommended to implement a mechanism that allows users to unlock their tokens in case of an emergency or remove the `EMERGENCY_UNLOCK_ACTION` action altogether.

[L-27] `LendingPool` contract exceeds the size limit for mainnet deployment

Currently, the `LendingPool` contract exceeds the size limit for mainnet deployment. The deployed size is 24.092 KiB, while the limit is 24.000 KiB.

```
gh
  repo clone RegnumAurumAcquisitionCorp/core regnum-aurum && cd regnum-aurum && git checkout d44
(...)

Compiled 202 Solidity files successfully (evm target: paris).

|
|   Solc version: 0.8.20      •   Optimizer enabled: true      •   Runs: 1
|
|   Contract Name            •   Deployed size (KiB)
|   (change) •   Initcode size (KiB) (change) |
|
(...)

|   LendingPool              •               24.092
|   () •               25.852 () |

Warning: 1 contracts exceed the size limit for mainnet deployment
(24.000 KiB deployed, 48.000 KiB init).
```

[L-28] Unsafe use of `transfer()` and `transferFrom()`

Some tokens do not implement the ERC20 standard properly but are still accepted by most protocols that accept ERC20 tokens. This includes tokens that do not return a boolean value from the `transfer()` and `transferFrom()` functions. When these sorts of tokens are cast to `IERC20`, their function signatures do not match, and, therefore, the calls made revert.

File: `NFTRoyaltyFeeCollector.sol`

```
IERC20(_token).transfer(recipient, _balance);
```

File: `Treasury.sol`

```
IERC20(token).transferFrom(from, address(this), amount);
(...)
IERC20(token).transfer(recipient, amount);
```

File: `GaugeRewardsDistributor.sol`

```
IERC20(token).transferFrom(msg.sender, address(this), amount);
```

It is recommended to use `safeTransfer()` and `safeTransferFrom()` functions from OpenZeppelin's `SafeERC20` library.

[L-29] Broken internal balance for fee-on-transfer tokens

In different parts of the protocol, when transferring tokens it is assumed that the amount of tokens received by the contract is the same as the amount of tokens sent to it, which is not always the case for fee-on-transfer tokens. This can cause the internal balance of the contract to be inconsistent with the actual balance.

File: ERC20VaultAdapter.sol

```
token.safeTransferFrom(from, address(this), amount);
emit TokenDeposited(address(token), from, amount);

return _assetValue(amount);
```

File: ERC20AssetAdapter.sol

```
_balances[user] += amount;

// Interaction: transfer tokens
token.safeTransferFrom(user, address(this), amount);
```

File: FeeCollector.sol

```
IERC20(token).safeTransferFrom(target, address(this), amount);

// Update collected fees for this token
_updateCollectedFees(token, feeType, amount);
```

File: BaseGauge.sol

```
totalSupply += amount;
_balances[user] = _balances[user] + amount;
stakingToken.safeTransferFrom(user, address(this), amount);
} else {
    if (amount > _balances[user]) revert InsufficientStakedBalance();
    totalSupply -= amount;
    _balances[user] = _balances[user] - amount;
    stakingToken.safeTransfer(user, amount);
```

File: Treasury.sol

```
IERC20(token).transferFrom(from, address(this), amount);
_balances[token] += amount;
_totalValue += amount;
```

In most cases, the tokens transferred are whitelisted or configurable, so it should be enough to avoid the use of fee-on-transfer tokens in those cases. However, in the case of the `Treasury` contract any ERC20 token can be deposited, so for this contract it is recommended to either create a whitelist of tokens to avoid the use of fee-on-transfer tokens or to check the actual amount of tokens received by the contract after the transfer.

[L-30] GaugeController uses future timestamps to initialize time variables

The `_initializeGauge()` function in GaugeController initializes `lastUpdateTime` and `lastRewardTime` with `nextPeriod` instead of the current timestamp. While this doesn't currently impact functionality, using a future timestamp for these tracking variables could cause issues in future implementations or integrations that rely on accurate historical timing data.

Recommendation: Initialize `lastUpdateTime` and `lastRewardTime` with `currentTime` instead of `nextPeriod` to maintain accurate historical timing data:

```
lastUpdateTime = currentTime;
lastRewardTime = currentTime;
```

[L-31] `lastRewardTime` in GaugeController incorrect on failed distribution

The GaugeController updates `lastRewardTime` even when reward distribution fails, leading to incorrect tracking of reward distribution timing.

Recommendation: Move the `lastRewardTime` update inside the try block so it only occurs when the distribution succeeds:

```
try gauge.notifyRewardAmount(rewardToken, amount) {
  lastRewardTime = block.timestamp;
  emit RewardDistributed(address(gauge), rewardToken, amount);
} catch {
  emit DistributionFailed(address(gauge), rewardToken, amount);
}
```

[L-32] `claimRewardToken()` blocks rewards after withdrawal in BaseGauge

The `claimRewardToken()` function in BaseGauge prevents users from claiming their earned rewards if they have already withdrawn their staked tokens, due to a check requiring `_balances[msg.sender] > 0`.

```
if (
    _balances[msg.sender] == 0 ||
    rewardData[token].distributor == address(0)
) {
    return 0;
}
```


Recommendation: Remove the `_balances[msg.sender] == 0` check from `claimRewardToken()` and only retain the distributor check.

[L-33] Auction contract permits end-time purchases due to off-by-one error

The `Auction` contract's `whenActive` modifier uses an incorrect comparison operator that allows users to purchase tokens exactly at the auction's end time (`block.timestamp == state.endTime`):

```
modifier whenActive() {
    require(block.timestamp >= state.startTime, "Auction not started");
    require(block.timestamp <= state.endTime, "Auction ended");
    _;
}
```

This off-by-one error means the auction remains active for one additional second after its intended end time, potentially allowing purchases at the reserve price when they should be rejected.

Recommendation: Update the end time check in the `whenActive` modifier:

```
modifier whenActive() {
    require(block.timestamp >= state.startTime, "Auction not started");
    - require(block.timestamp <= state.endTime, "Auction ended");
    + require(block.timestamp < state.endTime, "Auction ended");
    _;
}
```

[L-34] `RAACToken` burns incorrect amount when fees are disabled

The `RAACToken` contract's `burn()` function incorrectly deducts fees even when the fee collection is disabled (i.e., when `feeCollector` is set to `address(0)`). This means users will burn fewer tokens than intended, as the fee amount is still subtracted but never collected.

For example:

- User attempts to burn 1000 tokens.
- `burnTaxRate` is 5%.
- `feeCollector` is set to `address(0)` to disable fees.
- Current implementation: Burns 950 tokens (1000 - 5% fee) even though fees are disabled.

Recommendation: Add a check for disabled fees in the burn calculation:

```
function burn(uint256 amount) public virtual {
-   uint256 fee = amount.percentMul(burnTaxRate);
-   uint256 amountToBurn = amount - fee;
+   uint256 fee;
+   uint256 amountToBurn = amount;

+   if (feeCollector != address(0)) {
+       fee = amount.percentMul(burnTaxRate);
+       amountToBurn = amount - fee;
+   }

    _burn(_msgSender(), amountToBurn);

    if ( fee > 0 ) {
        _transfer(msg.sender, feeCollector, taxAmount);
    }
}
```

[L-35] **RAACNFT.transferRole()** revokes incorrect address

The `transferRole()` function in the `RAACNFT` contract incorrectly transfers the minter role from the `msg.sender` (admin) to the new minter, instead of transferring it from the current minter.

```
function transferRole(bytes32 role, address newMinter) external onlyRole
(ADMIN_ROLE) {
    _revokeRole(role, msg.sender);
    _grantRole(role, newMinter);
    emit MinterRoleTransferred(role, msg.sender, newMinter);
}
```

This implementation means that when the admin calls the function while he have the minter role, the function revoke his role>

Recommendation: Modify the function to explicitly transfer the role from the old minter to the new minter:

```
- function transferRole(address newMinter) external onlyRole
- (DEFAULT_ADMIN_ROLE) {
+ function transferRole(address oldMinter, address newMinter) external onlyRole
+ (DEFAULT_ADMIN_ROLE) {
-     _revokeRole(MINTER_ROLE, msg.sender);
+     _revokeRole(MINTER_ROLE, oldMinter);
+     _grantRole(MINTER_ROLE, newMinter);
}
```

[L-36] **StabilityPool** minter update may lead to lost rewards

The `setMinter()` function in the `StabilityPool` contract fails to call `_update()` before changing the minter address. Since `_update()` is responsible for processing pending rewards through `minter.tick()`, any unclaimed rewards from the previous minter will be lost when the minter address is updated.

```
function setMinter(address _minter) external onlyOwner {
    minter = IRAACMinter(_minter);
}
```

This could result in users missing out on earned rewards that were pending at the time of the minter change.

Recommendation: Call `_update()` before changing the minter address to ensure all pending rewards are processed.

[L-37] `VaultProxy` double counts availableLiquidity in withdrawal calculations

The `VaultProxy` contract incorrectly calculates the required withdrawal amount by subtracting `availableLiquidity` twice from the requested `amount`. This leads to an underestimation of the required withdrawal amount and allows the execution to proceed further than it should before reverting:

```
uint256 availableLiquidity = IERC20(reserve.reserveAssetAddress)
    .balanceOf(reserve.reserveTokenAddress);

if (availableLiquidity < amount) {
    uint256 requiredAmount = amount - availableLiquidity;
    uint256 maxWithdrawable = vault.maxWithdraw(address(this));

    uint256 totalAvailable = availableLiquidity + maxWithdrawable;
    if (totalAvailable < requiredAmount) {
        revert("Not enough liquidity to fulfill the withdrawal");
    }
}
```

For example:

- `amount` = 70 tokens.
- `availableLiquidity` = 10 tokens.
- `maxWithdrawable` = 50 tokens.
- Current calculation: `requiredAmount = amount - availableLiquidity = 70 - 10 = 60`.
- Total available = `availableLiquidity + maxWithdrawable = 10 + 50 = 60`.
- Code proceeds even though total available (60) < requested amount (70).

Recommendation: Remove the initial subtraction of `availableLiquidity` from the required amount calculation:

```

-         if (totalAvailable < requiredAmount) {
+         if (totalAvailable < amount) {
            revert("Not enough liquidity to fulfill the withdrawal");
        }
    }

```

or

```

-         uint256 totalAvailable = availableLiquidity + maxWithdrawable;
-         if (totalAvailable < requiredAmount) {
+         if (maxWithdrawable < requiredAmount) {
            revert("Not enough liquidity to fulfill the withdrawal");
        }
    }

```

[L-38] **FeeCollector** makes unchecked external calls to arbitrary addresses

The **FeeCollector** contract makes external calls to addresses without validating if they are trusted or malicious. While the impact may be limited, it's still a security best practice to restrict external calls to known, trusted contracts:

```

(bool success, bytes memory data) = target.call(
    abi.encodeWithSelector(bytes4(keccak256("claimCollectorRewards()")))
);

(bool success, ) = target.call(
    abi.encodeWithSelector(
        bytes4(
            keccak256("withdrawUnderlying(address,address,uint256)")
        ),
        token,
        address(this),
        amount
    )
);

```

Recommendation: Implement a whitelist of trusted target addresses that can be called by the **FeeCollector** or add access control to the functions.

[L-39] Missing update the interest timely

The admin can trigger oracle to update the prime rate. We will update the primeRate at first then we will update the borrow rate, and lending rate via the update primeRate. The problem is that we miss updating the liquidity index and usage index timely.

```

function setPrimeRate(
    ReserveDataStorage reserve,
    ReserveRateDataStorage rateData,
    uint256 newPrimeRate
) internal {
    if (newPrimeRate < 1) revert PrimeRateMustBePositive();

    uint256 oldPrimeRate = rateData.primeRate;
    // If the prime rate changes exceed our range, we will revert.
    if (oldPrimeRate > 0) {
        uint256 maxChange = oldPrimeRate.percentMul(500); // Max 5% change

        uint256 diff = newPrimeRate > oldPrimeRate ? newPrimeRate - oldPrimeRate :
            oldPrimeRate - newPrimeRate;
        if (diff > maxChange) revert PrimeRateChangeExceedsLimit();
    }

    rateData.primeRate = newPrimeRate;
    updateInterestRatesAndLiquidity(reserve, rateData, 0, 0);

    emit PrimeRateUpdated(oldPrimeRate, newPrimeRate);
}

```

Recommendation: Update the liquidity index, and usage index to accrue the interest for the previous time slot and then update the prime rate.

[L-40] Improper parameter in function

`notifyRewardAmount`

In gauge, we will be notified of rewards via function `notifyRewardAmount`. In function `notifyRewardAmount`, we have one modifier `updateReward`. In this modifier, we should define whether we will update one account's pending rewards. But we use one incorrect parameter `token`.

```

modifier updateReward(address account) virtual {
    // In modifier, we will
    _updateReward(address(0), account);
    _;
}

function notifyRewardAmount(
    address token,
    uint256 amount
) external override onlyRole(CONTROLLER_ROLE) {
    if (amount == 0) {
        emit RewardNotified(token, 0, 0);
        return;
    }
}

```

Recommendation: Use `address(0)` as the modifier's parameter.

[L-41] Missing sanity check for min rate & max rate in `updateEmissionRate`

In RAAC Minter, the emission rate can be updated via function `updateEmissionRate`. We will set the emission rate's minimum rate and maximum rate.

The problem here is that when we update the emission rate, we miss the check for `minEmissionRate` and `maxEmissionRate`. This will cause the actual emission rate may not in the expected range.

```
function updateEmissionRate(uint256 _newRate) external onlyRole
(UPDATER_ROLE) {
    if
        (_newRate == 0 || _newRate > MAX_BENCHMARK_RATE) revert InvalidBenchmarkRate();
    uint256 oldRate = emissionRate;
    emissionRate = _newRate;
    emit EmissionRateUpdated(oldRate, _newRate);
}
function setMinEmissionRate(uint256 _minEmissionRate) external onlyRole
(UPDATER_ROLE) {
    if (_minEmissionRate >= maxEmissionRate) revert InvalidMinEmissionRate
    ();
    uint256 oldRate = minEmissionRate;
    minEmissionRate = _minEmissionRate;
    emit MinEmissionRateUpdated(oldRate, _minEmissionRate);
}
function setMaxEmissionRate(uint256 _maxEmissionRate) external onlyRole
(UPDATER_ROLE) {
    if (_maxEmissionRate <= minEmissionRate) revert InvalidMaxEmissionRate
    ();
    uint256 oldRate = maxEmissionRate;
    maxEmissionRate = _maxEmissionRate;
    emit MaxEmissionRateUpdated(oldRate, _maxEmissionRate);
}
```

Recommendation: Add the sanity check in function `updateEmissionRate`.

[L-42] Missing sanity check for `EMERGENCY_DELAY`

In timelock controller, the emergency role can schedule one emergency action and execute this emergency action after the `EMERGENCY_DELAY`. The problem here is that when we execute our emergency action, we miss checking the `EMERGENCY_DELAY`. It means that the emergency role can execute the emergency action immediately. This breaks the expectation.

```

function scheduleEmergencyAction(bytes32 id) external onlyRole
(EMERGENCY_ROLE) {
    _emergencyActions[id] = true;
    emit EmergencyActionScheduled(id, block.timestamp);
}
function executeEmergencyAction(
    address[] calldata targets,
    uint256[] calldata values,
    bytes[] calldata calldatas,
    bytes32 predecessor,
    bytes32 salt
) external payable onlyRole(EMERGENCY_ROLE) nonReentrant {
    bytes32 id = hashOperationBatch
    (targets, values, calldatas, predecessor, salt);
    if (!_emergencyActions[id]) revert EmergencyActionNotScheduled
    (id);
    delete _emergencyActions[id];
    for (uint256 i = 0; i < targets.length; i++) {
        (
            bool success,
            bytes memory returndata
        ) = targets[i].call{value: values[i]}(calldatas[i]
        if (!success) {
            if (returndata.length > 0) {
                assembly {
                    let returndata_size := mload(returndata)
                    revert(add(32, returndata), returndata_size)
                }
            }
            revert CallReverted(id, i);
        }
    }
    emit EmergencyActionExecuted(id);
}

```

Recommendation: Add the sanity check for `EMERGENCY_DELAY`.

[L-43] Missing emergency unlock logic

The function `executeEmergencyUnlock()` correctly sets the flag `emergencyUnlockEnabled = true`, but nowhere in the contract is that flag consulted to permit withdrawals of locked positions.

```

File: veRAACToken.sol
425:     function scheduleEmergencyUnlock() external onlyOwner {
426:         _emergencyTimelock[EMERGENCY_UNLOCK_ACTION] = block.timestamp;
427:         emit EmergencyUnlockScheduled();
428:     }
429:
430:     function executeEmergencyUnlock()
431:     external
432:     onlyOwner
433:     withEmergencyDelay(EMERGENCY_UNLOCK_ACTION)
434:     {
435: @>         emergencyUnlockEnabled = true;
436:         emit EmergencyUnlockEnabled();
437:     }

```

It is recommended to incorporate `emergencyUnlockEnabled` so users are able to unlock positions.

[L-44] Legacy lock recalculation slashes voting power

When computing a user's voting power, the contract reads the **current** global `maxLockDuration`, even for locks created under a previous, longer duration. This allows a malicious or short-sighted governance vote to reduce all legacy lockers' power retroactively.

```
File: veRAACToken.sol
253:         uint256 newPower = LockManager._calculateVotingPower(
254:             userLock.amount,
255:             remainingTime,
256:@>         _lockState.maxLockDuration
257:         );
...
294:         uint256 newPower = LockManager._calculateVotingPower(
295:             newLock.amount,
296:             totalNewDuration,
297:@>         _lockState.maxLockDuration
298:         );
```

Record per-lock baseline

```
// LockManager.sol
struct Lock {
    uint256 amount;    // Amount of tokens locked
    uint256 start;     // Timestamp when lock starts
    uint256 end;       // Timestamp when lock expires
    bool exists;       // Flag indicating if lock exists
+   uint256 maxDurationAtLock; // NEW: snapshot of global cap
}
```

Use the stored cap for all calculations:

```
// veRAACToken.sol
uint256 newPower = LockManager._calculateVotingPower(
    userLock.amount,
    remainingTime,
-    _lockState.maxLockDuration
+    userLock.maxDurationAtLock
);
```

[L-45] Natspec and `getNormalizedDebt` implementation discrepancy

The NatSpec for `ReserveLibrary.getNormalizedDebt()` states it returns the normalized debt *in underlying asset units*, but the implementation actually returns a RAY-scaled index:


```
// File: ReserveLibrary.sol
461:
462:     /**
463:      * @notice Gets the normalized debt of the reserve.
464:      * @param reserve The reserve data.
465:      * @return The normalized debt (in underlying asset units).
466:      */
467:     function getNormalizedDebt(
        ReserveDataStorage reserve,
        ReserveRateDataStorage rateData
    ) internal view returns (uint256) {
468:         uint256 timeDelta = block.timestamp - reserve.lastUpdateTimestamp;
469:         if (timeDelta < 1) {
470:             return reserve.totalUsage;
471:         }
472:
473:         return calculateCompoundedInterest(
            rateData.currentUsageRate, timeDelta).rayMul(reserve.usageIndex);
474:     }
```

- **Line 470** returns `reserve.totalUsage` (raw tokens) only when no time has passed.
- **Line 473** compute `calculateCompoundedInterest(...)` (RAY) and multiply by `usageIndex` (RAY), yielding another RAY value. As a result, callers expecting an amount in tokens will instead receive a RAY (1e27)-scaled factor.

Clarify NatSpec if the intended output is the debt index or fix implementation to return true token amounts.

[L-46] Missing zero-amount check in

`LendingPool.borrowWithInsurance()`

The `borrowWithInsurance` function is missing the `onlyValidAmount(amount)` modifier, which means it can be called with a zero amount. This is likely unintended, as the regular `borrow` function does include this modifier to explicitly prevent zero-amount borrow calls to `_borrow` function:

```
File: LendingPool.sol
412:     function borrow(
        address adapter,
        bytes calldata data,
        uint256 amount
    ) external nonReentrant whenNotPaused onlyValidAmount(amount) {
413:         _borrow(adapter, data, amount);
414:     }
...
...
464:     function borrowWithInsurance(
        address adapter,
        bytes calldata data,
        uint256 amount
    ) external nonReentrant whenNotPaused onlySupportedAdapter(adapter) {
...
474:         _borrow(adapter, data, amount);
...
```

The following test demonstrates how `borrowWithInsurance` is allowed to be called with zero amount:

```
// File: LendingPool.test.js
it("represent01 call borrowWithInsurance zero amount", async function () {
  const borrowAmount = ethers.parseEther("50");
  await lendingPool.connect(user1).borrow
    (assetAdapter.target, assetData, borrowAmount);

  // approve insurance
  const insuranceFee = await lendingPool.calculateInsuranceFee
    (assetAdapter.target, user1.address, assetData, borrowAmount);
  await crvusd.connect(user1).approve(feeCollector.target, insuranceFee);

  // Call borrowWithInsurance with zero amount. The tx will not be
  // reverted.
  await lendingPool.connect(user1).borrowWithInsurance
    (assetAdapter.target, assetData, 0);
});
```

It's recommended to add the modifier to maintain consistent input validation.

[L-47] Queued proposals will not be executed if the timelock is changed

`Governance.setTimelock()` allows the owner to set a new timelock. This will provoke that when the queued proposals are executed, the execution will fail because the new timelock will not have the queued proposals.

Recommendations: Keep track of the queued proposals and queue them in the new timelock at the moment of switching.

[L-48] Altered `LendingPool` parameters may cause instant liquidations

`LendingPool.setParameter()` allows the protocol role to modify the `liquidationThreshold` and `healthFactorLiquidationThreshold`. As a result of changing this parameters, positions that were previously healthy can become unhealthy and get liquidated without time to react.

Recommendations: Make the changes of `liquidationThreshold` and `healthFactorLiquidationThreshold` subject to a timelock, so that users can adjust their positions accordingly.

[L-49] `RToken` fails if burn equals scaled balance increase

`RToken` reverts when the amount to burn is equal to the scaled balance increase.

```

if(scaledBalanceIncrease > amount){
    amountToMint = scaledBalanceIncrease - amount;
    _mint(from, amountToMint.toUint128());
} else {
    amountToBurn = amount - scaledBalanceIncrease;
    @> if (amountToBurn == 0) revert InvalidAmount();
    _burn(from, amountToBurn.toUint128());
}

```

There is no reason to deny users the ability to withdraw an amount equal to their scaled balance increase.

Recommendations:

```

amountToBurn = amount - scaledBalanceIncrease;
- if (amountToBurn == 0) revert InvalidAmount();
  _burn(from, amountToBurn.toUint128());

```

[L-50] **LendingPool** depositors might receive less than expected if fee is updated

On deposits to the lending pool, the deposit fee is subtracted from the deposit amount and the remaining amount is deposited into the pool.

If the deposit fee is updated without the depositor being aware, he will receive a smaller amount of tokens than expected.

Consider the following scenario:

- Current deposit fee is 0.
- Alice deposits 100 crvUSD, expecting to receive 100 RToken.
- Before Alice's transaction is confirmed, the deposit fee is updated to 10%.
- Alice receives 90 RToken instead of 100, which is not an amount she was willing to accept.

Recommendations: Receive an additional parameter on deposit to specify either the maximum deposit fee or the minimum amount of tokens to receive and enforce it.

[L-51] **Governance** owner can bypass timelock

The **Governance** contract uses a timelock mechanism to execute proposals. This ensures that users have a chance to react to the proposal before it is executed.

However, the timelock controller can be changed at any time by the owner of the contract, giving the owner total control over the execution of proposals and defeating the purpose of the timelock mechanism.

Recommendations: Make the change of the timelock controller subject to timelock itself. This can be done by either:

- Allowing only the current timelock controller to execute `setTimelock()`.
- Implementing a cooldown period between the proposal of the new timelock controller by the owner and the change.

[L-52] Incorrect check for total supply limit for `veRAACToken`

`veRAACToken.lock()` has the following check for the total supply limit:

```
if (totalSupply() + amount > MAX_TOTAL_SUPPLY)
    revert TotalSupplyLimitExceeded();
```

This is incorrect, as it is using the amount of RAAC tokens to be locked instead of the amount of veRAAC tokens to be minted, which will be lower if the duration is less than the maximum duration. As a result, users might not be able to lock their tokens even if the total supply resulting from the lock would be below the limit.

Recommendations:

```
-     if (totalSupply() + amount > MAX_TOTAL_SUPPLY)
-         revert TotalSupplyLimitExceeded();
(...)
uint256 newPower = uint256(uint128(bias));
+     if (totalSupply() + newPower > MAX_TOTAL_SUPPLY)
+         revert TotalSupplyLimitExceeded();
_checkpointState.writeCheckpoint(msg.sender, newPower);
_mint(msg.sender, newPower);
```

[L-53] Long liquidation grace period risks protocol solvency

The lending pool implements a liquidation grace period through the `BASE_LIQUIDATION_GRACE_PERIOD` constant set to 3 days. This grace period provides insured borrowers with a buffer time before their positions can be liquidated after becoming unhealthy. While grace periods can protect borrowers from temporary market volatility, a long duration exposes the protocol to insolvency risks.

```
if
    (block.timestamp > position.liquidationStartTime + parameters.liquidationGracePeriod) {
    revert GracePeriodExpired();
}
```

During the 3-day grace period, liquidators cannot close unhealthy positions even if they fall severely underwater. In volatile market conditions where collateral assets experience sharp price declines, the delay prevents timely liquidations that would otherwise protect the protocol. This creates a scenario where:

1. A borrower's position becomes unhealthy due to collateral price decline.
2. The 3-day grace period begins, blocking liquidations.
3. Collateral price continues falling significantly during this period.
4. By the time liquidations are enabled, the collateral value will be insufficient to cover the borrowed amount.
5. The stability pool absorbs the bad debt, negatively impacting protocol solvency.

The extended grace period effectively forces the protocol to maintain exposure to deteriorating positions. This design choice prioritizes borrower protection at the expense of protocol safety, particularly in stressed market conditions where swift liquidations are crucial for maintaining system health.

Recommendations: Consider reducing the `BASE_LIQUIDATION_GRACE_PERIOD` to a shorter timeframe (e.g., 4-12 hours, 1 day at max).

[L-54] Hardcoded borrow cap incompatible with low decimal tokens

The `LendingPool` contract implements a borrow cap mechanism to limit the maximum amount that can be borrowed for each asset. However, the borrow cap is hardcoded to use 18 decimals in the constructor, creating a significant mismatch for tokens with lower decimal places, particularly USDC which uses 6 decimals and is planned to be supported.

This decimal mismatch means that tokens with fewer decimals will have an effectively much larger borrow cap than intended. For example, if the borrow cap is set to $1,000,000 * 10^{18}$ (intended to be 1M tokens), for a 6 decimal token like USDC this would actually allow borrowing of $1,000,000 * 10^{18}$ USDC, which is 10^{30} USDC - an astronomically large amount.

```
constructor(  
    address _reserveAssetAddress,  
    // ... other parameters  
) {  
    // Hardcoded 18 decimals  
    borrowCap = 1_000_000 * 1e18;  
}
```

Recommendations: The borrow cap should be adjusted based on the decimals of the reserve asset:

```

constructor(
    address _reserveAssetAddress,
    // ... other parameters
) {
-   borrowCap = 1_000_000 * 1e18;
+   uint8 assetDecimals = IERC20Metadata(_reserveAssetAddress).decimals();
+   borrowCap = 1_000_000 * 10**assetDecimals;
}

```

[L-55] Decimal mismatch between **underlyingToken** and **RToken**

The **RToken** contract is designed to be a receipt token for deposits in the lending pool, supporting multiple underlying tokens including USDC. However, there is an issue in how token decimals are handled during minting. The **RToken** uses 18 decimals internally, but when minting tokens in response to deposits of underlying assets with different decimal places (like USDC with 6 decimals), the contract does not perform any decimal adjustment.

```

function decimals() public view virtual override(ERC20, IRToken) returns
(uint8) {
    return super.decimals();
}

```

This creates a significant mismatch between the value represented by the **RToken** and the actual underlying deposit.

1. Integration issues with other DeFi protocols expecting standard 18 decimal ERC20 tokens.
2. User experience problems as displayed values will be incorrect by several orders of magnitude.
3. Potential accounting errors in systems integrating with the protocol.
4. Similar issues with the **DebtToken** contract which is designed to maintain 1:1 parity with **RToken**.

Proof of Concept

1. User has 1,000,000 USDC (\$1 million).
2. User deposits 1,000,000 USDC to the lending pool.
3. Lending pool calls **mint()** on RToken with amount 1,000,000.
4. User receives 1,000,000 RTokens.
5. Due to 18 decimal places in RToken, this actually represents:
 - $1,000,000 * 10^{-18} = 0.000001$ RTokens.

Recommendations: Make RToken decimals match the underlying asset:

[L-56] Users must withdraw from the pool to claim rewards

The `StabilityPool` contract implements a reward mechanism to incentivize users to deposit and maintain stablecoin liquidity. However, in the current implementation there is no dedicated function for users to claim their accumulated rewards without withdrawing a portion of their deposit from the pool. The issue stems from reward distribution being tied exclusively to the `withdraw()` function, with no separate claim mechanism.

The inability to claim rewards without withdrawing from the Stability Pool severely undermines its core purpose of maintaining protocol stability and efficient liquidity provision. By forcing users to exit their positions to access rewards, the current design creates unnecessary liquidity volatility and increased gas costs, directly conflicting with the pool's stated goals of "aligning depositor incentives with protocol stability" and "compensating liquidity providers for risk exposure."

Recommendations: Add a dedicated reward claim function that allows users to collect their rewards without affecting their deposited position.

[L-57] Improper `minDy` calculation in LiquidationSwap

When one borrow position is unhealthy, the owner will liquidate this borrow position. We will deposit the liquidated collateral into index token and swap one part of index token for crvUSD token.

When we swap index token to crvUSD token, the `minDy` will be `(amount * (10000 - liquidityPoolFee)) / 10000`. The problem here is that index token is one fee-on-transfer token if we want to transfer from/to the swap pair. But when we calculate the `minDy`, we don't consider the transfer fee, this will cause actual swap token may be less than `minDy`.

```

function _swap(address sender, uint256 amount) internal {
    IERC20(vaultToken).safeTransferFrom(sender, address(this), amount);
    bool approveSuccessIndexToken = IERC20(vaultToken).approve(address
        (liquidityPool), amount);
    if (!approveSuccessIndexToken) revert ApprovalFailed();

    uint256 minDy = (amount * (10000 - liquidityPoolFee)) / 10000;
    liquidityPool.exchange(1, 0, amount, minDy);

    uint256 swappedAmount = IERC20(swapToken).balanceOf(address(this));
    IERC20(swapToken).safeTransfer(sender, swappedAmount);
}

function _swapAwareTransfer
(address sender, address recipient, uint256 amount) internal {
    if (isSwapPair[sender] || isSwapPair[recipient]) {
@>         uint256 feeAmount = (amount * swapFeeBP) / FEE_DENOM;
@>         uint256 burnAmount = (amount * burnFeeBP) / FEE_DENOM;
        uint256 rest = amount - feeAmount - burnAmount;
        if (burnAmount > 0) {
            _burn(sender, burnAmount);
        }
        if (feeAmount > 0) {
            super._transfer(sender, address(this), feeAmount);
            _approve(address(this), feeCollector, feeAmount);
            IFeeCollector(feeCollector).collectFee(address(this), address
                (this), feeAmount, keccak256("SWAP_FEE"));
        }
        super._transfer(sender, recipient, rest);
    } else {
        // standard no-fee move
        // If we don't transfer index token from or to the swap pair, we
        // don't need any fe
        super._transfer(sender, recipient, amount);
    }
}

```

Recommendations: Consider the possible transfer fee.

[L-58] Unbounded `distributionPools` array growth

The functions managing `distributionPools` array in `RAACMInter.sol` fail to maintain a tight list of distribution pools. In particular, `removeDistributionPool` deletes the pool's metadata and updates `totalWeight` but `distributionPools` is not adjusted. Over time, `distributionPools` accumulates “dead” entries, causing the minting loop to iterate over stale addresses:

```

File: RAACMinter.sol
368:     function removeDistributionPool(address pool) external onlyRole
        (UPDATER_ROLE) {
369:         if (!poolInfo[pool].exists) revert PoolDoesNotExist();
370:         poolInfo[pool].exists = false;
371:         delete poolInfo[pool];
372:         totalWeight = totalWeight - poolInfo[pool].weight;
373:         emit DistributionPoolRemoved(pool);
374:     }

```

When `_processMintedRewards` runs as part of the mint flow, it naively loops over every index in `distributionPools`:


```

File: RAACMinter.sol
244:         for (uint256 i = 0; i < distributionPools.length; i++) {
245:             address pool = distributionPools[i];
246:             DistributionPool memory poolData = poolInfo[pool];
247:             poolData.pendingRewards += rewardPerWeight * poolData.weight;
248:             _depositPendingReward(poolData);
249:         }

```

As `distributionPools.length` grows unchecked, this $O(n)$ loop will eventually consume more gas than the block limit, causing all mints to revert. Test:

```

it.only("0xbepresentremoveDistributionPoolArray", async function () {
    // initial pool from beforeEach
    const poolA = await lendingPool.getAddress();

    // add a second pool
    const poolB = await stabilityPool.getAddress();
    await raacMinter.addDistributionPool(poolB, 500);

    // sanity: now distributionPools = [poolA, poolB]
    expect(await raacMinter.distributionPools(0)).to.equal(poolA);
    expect(await raacMinter.distributionPools(1)).to.equal(poolB);

    // remove the first pool
    await raacMinter.removeDistributionPool(poolA);

    // AFTER removal, the array length is still 2:
    // index 0 is still the old entry (poolA) and index 1 is poolB
    expect(await raacMinter.distributionPools(0)).to.equal(poolA);
    expect(await raacMinter.distributionPools(1)).to.equal(poolB);

    // out-of-bounds at index 2
    await expect(raacMinter.distributionPools(2)).to.be.reverted;
});

```

Other examples of arrays are:

```

BaseGauge
- `distributorTokens`
- `rewardTokens`

GaugeController
- `_gaugeList`

GaugeRewardsDistributor
- `rewardTokens`

FeeCollector
- `feeTypeKeys`

RAACMinter
- `distributionPools`

veRAACToken
- `rewardTokens`

RWAVault
- `adapters`
- `redeemableERC721Adapters`
- `deposableAdapters`

StabilityPool
- `managerList`

LendingPool
- `adapters`

ERC721AssetAdapter
- `_depositedTokens`

ERC721VaultAdapter
- `_tokenIds` (acknowledged, but added for completeness)

```

Recommendations: Remove the pool's address from the `distributionPools` array (in `removeDistributionPool()`) to keep its length bounded. Additionally, `addDistributionPool()` and `manageDistributionPool()` should validate a `distributionPools` max length.

[L-59] Gauge functions inaccessible due to missing controller implementations

The gauge system is designed with a hierarchical control structure where the `GaugeController` contract is meant to have administrative control over individual gauge contracts through the `CONTROLLER_ROLE`. However, there is a flaw where several important gauge functions, despite being designed to be called by the controller, are effectively inaccessible because the controller contract lacks the necessary implementation to call them.

The `BaseGauge` contract grants the `CONTROLLER_ROLE` to the controller address and implements several administrative functions that are restricted to this role:

- `manageDistributor()`.
- `setMaxEmission()`.
- `setDistributorApproval()`.
- `setBoostAmplificationEnabled()`.
- `setPeriodDuration()`.

However, the `GaugeController` contract does not implement any functions to actually call these gauge functions, rendering them effectively inaccessible. Additionally, some controller functions like `updatePeriod()` are supposed to trigger corresponding updates in the gauges but fail to do so.

Proof of Concept

1. Admin needs to adjust the maximum emission rate for a gauge:
 - The gauge has `setMaxEmission()` restricted to `CONTROLLER_ROLE`.
 - The controller has the role but no function to call `setMaxEmission()`.
 - The emission rate cannot be updated.
2. Controller updates its period:
 - Calls its own `updatePeriod()`.
 - Never calls the gauge's `updatePeriod()`.
 - Periods become desynchronized.

Recommendations: Implement the missing functions in the controller.

[L-60] Proposal's execution can be blocked

In Governance, users can propose one proposal via `propose` function. If the proposal is voted on and succeeds, we can queue and execute this proposal.

When we queue one proposal, we will generate one proposal id via `targets`, `values`, `calldatas`, and `salt`. We don't allow to queue the same `id`. The problem here is that malicious users can frontrun the normal `propose` with the same parameters.

Now we have two proposals with different `proposalId` and total same parameters. If the normal proposer cancels the normal proposal, the malicious user can cancel the malicious proposal to block this proposal.

If two proposals are voted on and succeed, the malicious proposer can frontrun queue the malicious proposal. Then we will mark this `bytes32 id` as pending status. Then malicious users can cancel this proposal before we execute this proposal.

If we want to queue the normal proposal, the queue operation will be reverted because the `bytes32 id` is pending status.

```

function propose(
    address[] memory targets,
    uint256[] memory values,
    bytes[] memory calldatas,
    string memory description,
    ProposalType proposalType
) external override returns (uint256) {
    uint256 proposalId = _proposalCount++;
}

function _queueProposal(uint256 proposalId) internal {
    ProposalCore storage proposal = _proposals[proposalId];

    bytes32 salt = proposal.descriptionHash;
    bytes32 id = _timelock.hashOperationBatch(
        proposal.targets,
        proposal.values,
        proposal.calldatas,
        bytes32(0),
        salt
    );

    // Check if already queued
    @> if (_timelock.isOperationPending(id)) {
        revert ProposalAlreadyExecuted(proposalId, block.timestamp);
    }
    ...
}

```

Recommendations: When we generate the operation batch hash, consider adding the `proposalId`.

[L-61] Protocol will lose some profit from the curve vault

In VaultProxy, when the current buffer is not enough, we will withdraw some funds from the curve vault. The profit from the curve vault will be accrued into `unclaimedRewards`. And the fee collector will claim these rewards.

The problem here is that we make one incorrect yield calculation. When we calculate the `yield`, we will calculate the `yield` according to the `burnShares`. This will cause the remaining share's yield may fail to be accrued into `unclaimedRewards` because we've already updated the `lastSharePrice` to the latest share price.

For example:

1. We deposit 200 underlying token into the curve vault, and we get 200 curve vault shares.
2. After one period, the share's price increases, and we can withdraw 220 underlying tokens.
3. We withdraw 110 underlying tokens, and we will add 10 yield into the `unclaimedRewards`.
4. We withdraw the remaining underlying token, the yield will be 0 because `currentSharePrice` equals `vaultRewards.lastSharePrice`.
5. If we withdraw 220 underlying tokens once in step 3, we will have 20 yield token.

```

function withdrawFromVault(uint256 amount) internal {
    uint256 currentSharePrice = vault.pricePerShare(true);
    // withdraw from vault. Now crvUSD is in the Lending pool contract.
    uint256 burnedShares = vault.withdraw(amount, address(this), address
        (this));

    // Calculate yield based on share price appreciation and update to
    // unclaimedRewards
    uint256 yield;
    if (currentSharePrice > vaultRewards.lastSharePrice) {
        uint256 priceDiff = currentSharePrice - vaultRewards.lastSharePrice;
        yield = (burnedShares * priceDiff) / 1e18;
        vaultRewards.unclaimedRewards += yield;
    } else {
        yield = 0;
    }

    uint256 transferAmount = amount - yield;
    IERC20(reserve.reserveAssetAddress).safeTransfer
        (reserve.reserveRTokenAddress, transferAmount);

    // Total deposits can never be negative
    if (amount > vaultRewards.totalDeposits) {
        // @note-ok here. LOW.
        vaultRewards.totalDeposits = 0;
    } else {
        vaultRewards.totalDeposits -= amount;
    }

    @>    vaultRewards.lastSharePrice = currentSharePrice;
    vaultRewards.lastWithdrawalTime = block.timestamp;
}

```

Recommendations

Revisit the yield implementation logic.

[L-62] Duplicate reward application in

updateReward

In `BaseGauge.sol`, `_updateReward` invokes two separate reward-crediting mechanisms (`_calculatePendingRewards()` and `_updateUserReward()`) that compute and add the exact same amount to `rewards[token][account]`:

```

File: BaseGauge.sol
964:     function _updateReward(address token, address account) internal {
...
1012:@>         uint256 pending = _calculatePendingRewards
    (token, account);
1013:         if (pending > 0) {
1014:@>             rewards[token][account] += pending;
1015:         }
1016:@>         _updateUserReward(
1017:             token,
1018:             account,
1019:             rewardPerToken,
1020:             decimalAdjustment
1021:         );

```

```

File: BaseGauge.sol
929:     function _updateUserReward(
930:         address token,
931:         address account,
932:         uint256 rewardPerToken,
933:         uint256 decimalAdjustment
934:     ) internal {
...
947: @>         uint256 newReward = (workingBalance * userIntegralDelta) /
948:             (1e18 * decimalAdjustment);
949:
950:         userData.integral = rewardPerToken;
951:         userData.lastUpdate = block.timestamp;
952: @>         rewards[token][account] += newReward;

```

Both `_calculatePendingRewards` and the core logic inside `_updateUserReward` implement the exact same reward-calculation algorithm: they first derive how much reward has accrued since the user's last checkpoint—either by prorating the emission rate over elapsed time in `_calculatePendingRewards` or by computing the difference in `rewardPerTokenStored` values in `_updateUserReward`—and then scale that delta by the user's working balance, dividing by the protocol's precision and any decimal-adjustment factor. In both cases the computation boils down to:

```

earned = (workingBalance × rewardIntegralDelta) / (1e18 × decimalAdjustment)

```

which produces an identical amount (`pending` or `newReward`) to add to `rewards[token][account]`. By running this same formula twice, the contract double-counts each user's true entitlement.

Recommendations: It appears this went unnoticed because `_calculatePendingRewards` consistently produced a very low value in basis points due to the issue described in `Incorrect boost application overwrites user share`. Now that the `_calculatePendingRewards` function will return the correct amount due to the fix in that issue, it is advisable to simplify `_updateReward()` function by removing the call to `_calculatePendingRewards` entirely.

[L-63] Indefinite reward accrual via `emergencyWithdraw()`

The `emergencyWithdraw()` path fails to clear a user's reward checkpoints or zero out their voting power baseline. As a result, once "emergency mode" is enabled and a user calls `emergencyWithdraw()`, their `userVotingPowerAtNotify` remains nonzero and their `userRewardPerTokenPaid` remains "stale". Every new reward period or passage of time increases `rewardPerToken`, so `claimable()` for that user continues to grow indefinitely—even though they no longer hold any locked tokens. This allows a malicious actor to:

1. Lock once to gain initial voting power before the emergency mode is activated.
2. Trigger emergency mode.
3. Call `emergencyWithdraw()`.

4. Continue to earn and claim *all* future distributions without ever re-locking.

The following test shows how a user who retrieved their tokens by calling the `emergencyWithdraw` function can later call the `claimReward` function indefinitely and claim future rewards.

```

// File: veRAACToken.test.js
it("0xbepresentRewardsInEmergencyWithdrawal", async () => {
  const INITIAL_STAKE = ethers.parseEther("1000");
  const LOCK_DURATION = YEAR;
  const DAY = 24 * 3600;
  // 1) Two users lock tokens
  await approveAndLock(users[0], INITIAL_STAKE, LOCK_DURATION);
  await approveAndLock(users[1], INITIAL_STAKE, LOCK_DURATION);
  //
  // 2) Trigger emergency
  const EMERGENCY_ACTION = ethers.keccak256(
    ethers.toUtf8Bytes("enableEmergencyWithdraw")
  );
  await veRAACToken.connect(owner).scheduleEmergencyAction(
    EMERGENCY_ACTION);
  await time.increase(EMERGENCY_DELAY);
  await veRAACToken.connect(owner).enableEmergencyWithdraw();
  await time.increase(EMERGENCY_DELAY);
  //
  // 3) Send rewards
  await veRAACToken.addRewardToken(await rewardToken.getAddress());
  await rewardToken.mint(owner.address, ethers.parseEther("2000"));
  await rewardToken.connect(owner).approve(
    await veRAACToken.getAddress(),
    ethers.parseEther("2000"));
  await veRAACToken.notifyRewardAmount(
    await rewardToken.getAddress(),
    ethers.parseEther("1000")
  );
  //
  // 4) Let rewards accrue for a while
  await timeIncreaseWithMining(DAY);
  //
  // 5) User0 executes emergencyWithdraw
  await veRAACToken.connect(users[0]).emergencyWithdraw();
  //
  // 6) Check some claimable amount for user0, even when he has
  // already withdrawn
  let claimable0 = await veRAACToken.claimable(
    users[0].address,
    await rewardToken.getAddress()
  );
  expect(claimable0).to.be.gt(0);
  await timeIncreaseWithMining(DAY);
  //
  // 7) Check claimable for user0 after time increase.
  let claimable0After = await veRAACToken.claimable(
    users[0].address,
    await rewardToken.getAddress()
  );
  expect(claimable0After).to.be.gt(claimable0);
  //
  // 8) Claim rewards, assert claimable amount is zero
  await veRAACToken.connect(users[0]).claimReward(users[0].address);
  claimable0 = await veRAACToken.claimable(
    users[0].address,
    await rewardToken.getAddress()
  );
  expect(claimable0).to.equal(0);
  //
  // 9) Another reward distribution
  await rewardToken.mint(owner.address, ethers.parseEther("2000"));
  await rewardToken.connect(owner).approve(
    await veRAACToken.getAddress(),
    ethers.parseEther("2000"));
  await veRAACToken.notifyRewardAmount(
    await rewardToken.getAddress(),
    ethers.parseEther("1000")
  );
  //
  // 10) Let rewards accrue for a while
  await timeIncreaseWithMining(DAY);
  claimable0After = await veRAACToken.claimable(
    users[0].address,
    await rewardToken.getAddress()
  );
});

```



```

        expect(claimable0After).to.be.gt(claimable0);
        //
        // 11) Claim rewards, assert claimable amount is zero
        await veRAACToken.connect(users[0]).claimReward(users[0].address);
        claimable0After = await veRAACToken.claimable(
            users[0].address,
            await rewardToken.getAddress()
        );
        expect(claimable0After).to.equal(0);
    });

```

Recommendations: Mirror the cleanup logic from the standard `withdraw()` path inside `emergencyWithdraw()` to fully reset a user's reward state and `_checkpointState`.

[L-64] Missing decimals normalization between collateral and debt

The `LendingPool` contract implements a lending and borrowing system where users can deposit collateral assets and borrow other assets against their collateral. The system calculates a collateral value to determine the maximum amount a user can borrow:

```

function _validateBorrow(
    address adapter,
    bytes calldata data,
    uint256 amount
) internal view {
    // check that the amount does not exceed borrow
    require(
        IDebtToken(reserve.reserveDebtTokenAddress).totalSupply() +
        amount <=
        parameters.borrowCap,
        "borrow cap reached"
    ); // totalsupply because raw total supply just is balance of debtToken,
    // but total supply return the total amount borrowed so far.
    // For the borrow, we need to ensure that the user has enough collateral
    // to cover the new debt
    uint256 collateralValue = IAssetAdapter(adapter).getAssetValue(
        msg.sender,
        data
    );
    if (collateralValue == 0) revert NoCollateral();

    // We calculate the max debt that the user can have
    //(based on the collateral value and the liquidation threshold)
    uint256 maxDebt = collateralValue.percentMul(
        parameters.liquidationThreshold
    );

    // We ensure that the position has enough collateral to cover the new
    // debt or revert
    if (maxDebt < getPositionDebt(adapter, msg.sender, data) + amount) {
        revert NotEnoughCollateralToBorrow();
    }
}

```

The issue occurs in the collateral value calculation process. When determining the maximum debt a user can take on, the contract uses `adapter.getAssetValue()` which returns values in 18 decimal precision. However, the contract fails to account for assets with different decimal precisions when processing user-provided borrow amounts.

Specifically, when a user attempts to borrow an asset with fewer decimals (such as USDT or USDC which use 6 decimals), the contract incorrectly compares the 6-decimal user input directly against the 18-decimal collateral value. This decimal precision mismatch creates a significant vulnerability where users can borrow substantially more value than their collateral should allow.

For example, if a user has collateral worth 1000 tokens (represented as $1000 * 10^{18}$ internally), and attempts to borrow USDC (6 decimals), they could potentially borrow up to $1000 * 10^6$ USDC, which is actually 10^{12} times more value than the intended.

Recommendations: The contract should normalize decimal precision when comparing collateral values with borrow amounts.

[L-65] Inactive gauge blocks withdrawals

BaseGauge's withdrawal flow always invokes the boost checkpoint (withdraw() -> _handleBalanceUpdate() -> _updateWorkingBalance() -> _checkpoint()) and the `_applyBoost()`, even when the gauge has been deactivated:

```
// BaseGauge.sol
709 function _checkpoint(address user) internal {
...
730:         if (account != address(0)) {
731:             uint256 userCount = userCheckpointCounts[account];
732:             if (userCount < MAX_CHECKPOINTS) {
733:@>             uint256 boost = _applyBoost(account, _balances[account]);
...

```

Inside `_applyBoost`, the controller's `calculateBoost()` reverts by `GaugeNotFound()` if the gauge is marked inactive:

```
File: BaseGauge.sol
1070:     function _applyBoost(
        address account,
        uint256 baseWeight
    ) internal view virtual returns (uint256
1071:         if (baseWeight == 0) return 0;
1072:
1073:         // If boost amplification is disabled, return base multiplier
        //(10000 = 100%)
1074:         if (!boostAmplificationEnabled) {
1075:             return IGaugeController(controller).getMinBoost();
1076:         }
1077:
1078:         // Get boost from controller
1079:@>         (uint256 boost,) = IGaugeController(controller).calculateBoost
        (account, address(this), baseWeight);
1080:         return boost;
1081:     }
```

```

File: GaugeController.sol
325:     function calculateBoost(
326:         address user,
327:         address gauge,
328:         uint256 amount
329:     ) external view returns
    (uint256 boostBasisPoints, uint256 boostedAmount) {
330:@>         if (!isGauge(gauge)) revert GaugeNotFound();
...
772:     function isGauge(address gauge) public view returns (bool) {
773:@>         return gauges[gauge].lastUpdateTime != 0 && gauges[gauge].isActive;
774:     }

```

When `emergencyShutdown(gaugeAddr)` or `toggleGaugeStatus(gaugeAddr)` is called, `isGauge[gaugeAddr]` becomes `false`. Any subsequent call to `_applyBoost`, and thus any withdrawal, reverts at line `GaugeController#L330`, blocking withdrawals:

```

File: GaugeController.sol
371:     function toggleGaugeStatus(address gauge) external onlyRole
    (GAUGE_ADMIN) {
372:         if (!isGauge(gauge)) revert GaugeNotFound();
373:@>         gauges[gauge].isActive = !gauges[gauge].isActive;
374:         emit GaugeStatusUpdated(gauge, gauges[gauge].isActive);
375:     }

```

The following test demonstrates (`setBoostAmplificationEnabled is true`) how staking and reward accumulation work. Then, the `toggleGaugeStatus` function is called to deactivate the Gauge, which results in the user being unable to withdraw their stake anymore:

```
// File: BaseGauge.test.js
describe("Reward Distribution", () => {
  beforeEach(async () => {
    await advancePastPeriod(gauge);
    await gaugeController.vote(await gauge.getAddress(), 5000);
    await distributor.initializeRewardData(await gauge.getAddress(),
    await rewardToken.getAddress(), REWARD_AMOUNT);
    await rewardToken.approve(await distributor.getAddress(), REWARD_AMOUNT);
    await distributor.notifyRewardAmount(await rewardToken.getAddress(), REWARD_AMOUNT);
  });
  //

  it("0xbepresentshouldPreventWithdrawalsAfterShutdown", async () => {
    // setup boost amplification
    await gauge.connect(owner).setBoostAmplificationEnabled(true);
    //
    // 1. Setup initial staking
    await gauge.connect(user1).stake(ethers.parseEther("100"));
    // Verify initial stake
    const balance = await gauge.balanceOf(user1.address);
    expect(balance).to.equal(ethers.parseEther("100"));
    //
    // 2. Time pass to accumulate rewards
    await time.increase(WEEK);
    //
    // 4. Toggle gauge status to inactive
    await gaugeController.toggleGaugeStatus(await gauge.getAddress());
    // Verify gauge is inactive
    const isActive = await gaugeController.isGauge(
    await gauge.getAddress());
    expect(isActive).to.be.false;
    //
    // 5. user withdraw will be reverted
    await expect(
    gauge.connect(user1).withdraw(ethers.parseEther("100"))
    ).to.be.reverted;

  });
});
```

Recommendations: It is recommended that the `_applyBoost()` function return `minBoost()` when the `calculateBoost` call reverts.

[L-66] `WadRayMath.rayExp()` loses precision for large numbers

`WadRayMath.rayExp()` loses precision when `x` is large. For example a value of `x` equal to `10e27` returns `~4_850e27`, but the expected value should be `~22_026e27`.

This function is currently being used only in `ReserveLibrary.calculateCompoundedInterest()` and to produce a value greater than `1e27` it would require that the current rate is much greater than `1e27` and/or the time delta is very large (more than a year), which is unlikely to happen in practice.

However, it is recommended to document the limitation of this function to avoid its future use if it is expected to be used with values of `x` greater than `1e27`.

[L-67] Withdrawals in `RWAVault` may cause unexpected liquidations

The `withdrawAsset()` function in `RWAVault` allows emergency withdrawals of assets without burning vault tokens, which can cause sudden changes in the vault token price. Since the vault token price is derived from `totalAssets()` and used as collateral in the `LendingPool`, emergency withdrawals can trigger unexpected liquidations of borrowers' positions. Similarly, the `burnVaultToken()` function can artificially inflate the token price:

```
function withdrawAsset(
    address adapter,
    bytes calldata data,
    address receiver
) external onlyManager onlySupportedAdapter(adapter) nonReentrant {
    // note: this is kind of emergency withdraw of asset. No vault token is
    // burn
    if (receiver == address(0)) revert InvalidAddress();
    IVaultAssetAdapter(adapter).withdraw(data, receiver);
}

function burnVaultToken(address from, uint256 amount) external {
    if (!isManager(msg.sender) && msg.sender != stabilityPool)
        revert("only manager or stability pool");
    IVaultToken(vaultToken).burn(from, amount);
}
```

Recommendation: Implement a protocol-wide pause mechanism that should be activated before performing emergency withdrawals or vault token burns.